



TERM FINANCE

v2 Contracts
Security Assessment Report

Version: 2.0

May, 2026

Contents

Introduction	3
Disclaimer	3
Document Structure	3
Overview	3
Security Assessment Summary	4
Scope	4
Approach	6
Coverage Limitations	6
Findings Summary	6
Detailed Findings	7
Summary of Findings	8
Cross-Chain Transfers Break Redemptions Or DoS The Contract Before Maturity	12
OFT Contract Incorrectly Initialised	14
OFT Decimals Should Match decimalPlaces	16
Missing Admin Setter For approvedTermControllers Causes Global Reverts	17
Uninitialised AccessControl In TermDiamond	20
Rollover Bids Bypass Initial Collateral Ratio Enabling Self Liquidation	22
Arbitrary Function Execution In retrieveFunds Enables Complete Protocol Compromise	24
EIP-712 Nested Struct Hashing Non-Compliance Breaks Wallet Signature Validation	28
Missing Access Control In TermMorphoInterfaceFacet Allows Unauthorised Position Manipulation	30
Inverted Validation Logic In RetrieveFunds Allows Invalid Function Calls	32
Bid Update Collateral Array Mutation Enables Cross-User Locker Drain	34
Partial Liquidation Net Exposure Cap Guard Always Bypassed, Enabling Bad Debt Creation	37
Users Can Incur Full Aave/Morpho Debt While Receiving Only A Partial Term Fill	39
Wrong-Token retrieveFunds() Drains Diamond Balances Via Ignored Asset Check	41
Intent Minting Bypasses And Inflates Mint Exposure Cap	43
Blacklisted Borrower Can DoS Batch Liquidations Through Automatic Collateral Unlock	45
_debit() Allows Burning Tokens Due To Missing Modifier	47
Missing Slippage Protection In Strategy Sales	48
Missing Slippage Bounds In ERC4626 Deposits And Withdrawals	50
Uninitialised ERC-165 Mapping Causes supportsInterface To Always Return false	52
AccessControl Overrides Diamond supportsInterface	53
Missing Slippage Protection In Aave Supply Operations	54
User Tokens Can Be Locked In Facets	55
Fund Theft Via Flash-Loan Borrower Context Leak And Public swap()	56
Vault FulfillOrder Atomic Context Leaks To Malicious Vault Callback	58
Vault FulfillOrder Strands Purchase Tokens Extractable Via Batch Settlement	61
Collateral Dust From Liquidations Blocks Pro-Rata Redemption	64
Factory-Deployed Repos Permanently Locked Out Of Auction And Rollover-In Paths	66
Failed Or Partially Filled Rollover Auction Permanently Revokes Re-Election Right	68
Flash Loan Hooks Over-Pull On Partial Fill, Draining Taker Balance	70
Non-Auction Origination Uses Maintenance Ratios Instead Of Initial Ratios	72
Non-Standard ERC20 Tokens Are Not Supported	75
Partial Liquidation Leaves Locked Rollover Amount Above Live Debt	76
Repo-Token Liquidations Enable Preferential Recovery Against Selected Borrowers	79
Rollover Bids Bypass Initial Collateral Ratio Check	81
Silent Unlock Failures Strand Collateral And Double-Charge Borrowers	84
Unrevealed Rollover Bids Can Bypass The Public Auction Reveal Gate	87
Intent Settlement Bypasses Protocol servicingFee() Enforcement	89
WETHWrappingFacet Is Unusable Through The Diamond	92
Removal Of Minimum Tender Amount Check Allows Dust Rollover Bids	94
ERC20 Token Decimals Handling Vulnerabilities In usdValueOfTokens()	96

Fixed Precision Does Not Cater For Various Tokens	98
Incorrect <code>closureAmount</code> Emitted In Repo Token Liquidation Events	99
Leftover Collateral From Batch Default Functions Becomes Inaccessible After <code>mintOpenExposure</code>	101
<code>mintExposureCap</code> Is Defined Per Chain Causing Incorrect Accounting	102
Incomplete Facet Cleanup On Last Selector Removal Risks EIP2535 Noncompliance	103
Unnecessary Payable Entry Points Allow Stuck ETH	104
Permissionless Redemption Enables Front-Run Griefing	105
NatSpec Claims <code>SETTLER_ROLE</code> Restriction Which Is Not Implemented	106
Missing Oracle Validation	108
Unsafe <code>uint160</code> Casting In <code>Permit2</code> Transfers Enables Accounting Manipulation	110
Use Of Deprecated <code>transfer()</code> Method	112
Missing Input Validation Enables DoS and Unexpected Behaviour	113
Pro-Rata Rounding Creates Permanently Stale Encumbered Collateral	115
Rollover Approval Boundary Schedule Makes Rollover Assignment Impossible	117
Rollover Revocation Does Not Unwind Granted Roles	119
Strategy Facet Rejects Factory-Deployed Repos	122
<code>TermFlashLoanExecutorFacet</code> Requires Multi-Action Plans	124
Cached EIP-712 Domain Separator Can Become Stale	125
Caller-Sensitive <code>vault.asset()</code> Enables Undeclared Diamond Balance Spend	127
Preview Hooks Hardcode Primary Collateral Index, Breaking Multi-Collateral Flash Plans	129
Shared Salt In Deterministic Deployment Enables Address Griefing	131
No Default Admin Role For Permission Management After Deployment	133
Generic Custom Error Handling Obscures Underlying Revert Reason	134
Missing Duplicate Selector Check With Misleading Replace Revert	135
Diamond Lacks Pausing Mechanism	136
Owner Not Externally Queryable Via Loupe	137
Add Only Cuts Can Execute Unintended Initialisers Via <code>_init</code>	138
No Ownership Transfer Function In Diamond	140
Misleading NatSpec In <code>TermRouterFacet</code>	142
Borrowers' Excess Collateral Remains Locked On Partial Bid Fills	144
Potential Overlapping Offer Reveal And Borrower Bid Commit Times	146
Bid Slot Exhaustion Enables Single Borrower Manipulation And Auction Cancellation	148
Offer Slot Exhaustion Enables Single Lender Manipulation	150
Weak Commit Hash Allows Price Guessing With Low Entropy Nonces	152
Code Quality Issues In BSC Flashloan Implementation	153
Protocol Route Numbering System Issues	154
Incorrect Comments In <code>TermMorphoInterfaceFacet.sol</code>	155
Unused Local Variable	156
Documentation Claims Generic Gas Token Support But Code Is WETH-Specific	157
Insufficient Validation In <code>permit</code> Function	159
Factory Does Not Explicitly Reject Fee-On-Transfer Tokens	160
Inefficient Zero-Value Approvals in Collateral Flow	162
<code>settleLimitLendHook()</code> Omits <code>nonReentrant</code> , Diverging From The Hook Safety Pattern	163
Unsafe <code>Permit2</code> Cast In <code>RepoToken</code> Swap Silently Truncates Transfer Amount	165
Miscellaneous General Comments	166

A Vulnerability Severity Classification 170

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Term Finance components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Term Finance components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see ??), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Term Finance components in scope.

Overview

Term Finance is a fixed-rate onchain lending protocol that matches borrowers and lenders through sealed-bid auctions modeled on traditional repo markets. The protocol manages collateralised loans, repayments, liquidations, and ERC-20 repo tokens redeemable for principal plus interest at maturity.

This review covered Term Finance's migration to and expansion of an EIP-2535 Diamond-based architecture, including the modular facet system, upgradeability model, shared storage design, and selector-based routing. The assessment also reviewed the protocol's new intent-based execution framework, which enables signed off-chain limit orders, atomic settlement, multicall execution, Permit2 integration, and integrations with external protocols such as Aave and Morpho.

The review also included native flashloan execution through a new `TermFlashLoanExecutorFacet`, enabling leveraged and multi-step strategies to execute atomically in a single transaction. The review focused on the security and correctness of the new execution flows, cross-facet interactions, and flashloan orchestration logic.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [term-finance/term-finance](#) repository.

The assessment was conducted in four time-boxed phases, with the first phase's scope being strictly limited to the following PRs:

- [PR#1444](#)
- [PR#1446](#)
- [PR#1453](#)
- [PR#1454](#)
- [PR#1455](#)
- [PR#1456](#)
- [PR#1457](#)
- [PR#1458](#)
- [PR#1459](#)
- [PR#1460](#)

Additional assessment was performed on the OFT support that was added in this PR:

- [PR#1466](#)

The scope of the second phase was strictly limited to the following PRs:

- [PR#1443](#)
- [PR#1450](#)
- [PR#1451](#)
- [PR#1476](#)

The scope of the third phase was strictly limited to the following files and functions, assessed at commit [5437c09](#):

- `TermLoanIntentFacet.sol`
 - `settleLimitLend`
 - `cancelLimitLend`
 - `settleLimitBorrow`
 - `settleLimitBorrow`
 - `cancelLimitBorrow`
 - `setLimitOrderMakerTokenPairMinSaltValue`
 - `setPreSignedLendOrderHash`
 - `setPreSignedBorrowOrderHash`
 - `revokePreSignedLimitOrderHash`
- `TermRepoTokenIntentFacet.sol`
 - `swapRepoToken`
 - `cancelRepoToken`
 - `setSwapOrderMakerTokenPairMinSaltValue`
 - `setPreSignedSwapHash`
 - `revokePreSignedSwapOrderHash`
- `TransferFacet`
 - `erc20Transfer`
 - `erc20TransferFrom`
- `PermitFacet`
 - `permit`
- `Permit2Facet`
 - `approve2`
 - `transferFrom2`
- `TermAaveInterfaceFacet`
 - `aaveApproveDelegationWithSig`
 - `aaveSupply`
 - `aaveWithdrawOnBehalfOf`
 - `aaveBorrow`
 - `aaveRepay`
 - `availableFunds`
 - `availableBorrow`
- `TermMorphoInterfaceFacet`

- morphoSetAuthorizationWithSig
- morphoSupplyCollateral
- morphoSupply
- morphoWithdraw
- morphoWithdrawCollateral
- morphoBorrow
- morphoRepay
- availableFunds
- availableBorrow
- flashloan-aggregator diff
- TermFlashLoanCentralReceiverFacet.sol
- SwapRouterFacet.sol
- IPreSwap.sol
- IPostSwap.sol
- ERC4626InterfaceFacet
 - erc4626Withdraw
 - erc4626Redeem
- GasTokenWrapperFacet
 - wrapGasToken
 - unwrapGasToken
- e266ad9 Phase 2 changes
 - lockBidsWithReferral
 - lockOffersWithReferral

The scope of the fourth, the final phase, was strictly limited to the changes introduced between commits [4380d41](#) and [7ef08a6](#), assessed at the [4380d41...7ef08a6](#) diff.

Specifically, changes to the following files were considered in scope:

- TermAuction.sol
- TermAuctionBidLocker.sol
- TermAuctionOfferLocker.sol
- TermController.sol
- TermRepoCollateralManager.sol
- TermEventEmitter.sol
- TermRepoLocker.sol
- TermRepoRolloverManager.sol
- TermRepoServicer.sol
- TermRepoToken.sol
- facets/ERC4626InterfaceFacet.sol
- facets/Permit2Facet.sol
- facets/PermitFacet.sol
- facets/SwapRouterFacet.sol
- facets/TermLoanIntentFacet.sol
- facets/TermLoanIntentHookFacet.sol
- facets/TermRepoTokenIntentFacet.sol
- facets/TermRouterFacet.sol
- facets/TermStrategyFacet.sol
- facets/base/TermFlashBase.sol
- facets/base/TermFlashHookFacet.sol
- facets/base/TermMultiContextAuth.sol
- facets/external/TermAaveInterfaceFacet.sol
- facets/external/TermMorphoInterfaceFacet.sol
- facets/flashloan/TermFlashLoanExecutorFacet.sol
- factory/TermDiamondFactory.sol
- factory/TermRepoDeployerFactory.sol
- interfaces/ITermController.sol
- interfaces/ITermIntent.sol
- interfaces/ITermRepoCollateralManager.sol
- lib/ActionHookInput.sol
- lib/PreviewAction.sol
- lib/TermContractsPausable.sol
- libraries/LibTermStorage.sol

All of the above changes and corresponding fixes were consolidated and merged into [term-finance/term-finance-contracts](#) repository at commit [306bb3a](#).

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 86 issues during this assessment. Categorised by their severity:

- Critical: 9 issues.
- High: 6 issues.
- Medium: 24 issues.
- Low: 23 issues.
- Informational: 24 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Term Finance components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open:** the finding has not been addressed by the project team.
- **Resolved:** the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed:** the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
TRF-01	Cross-Chain Transfers Break Redemptions Or DoS The Contract Before Maturity	Critical	Closed
TRF-02	OFT Contract Incorrectly Initialised	Critical	Closed
TRF-03	OFT Decimals Should Match decimalPlaces	Critical	Closed
TRF-04	Missing Admin Setter For approvedTermControllers Causes Global Reverts	Critical	Closed
TRF-05	Uninitialised AccessControl In TermDiamond	Critical	Resolved
TRF-06	Rollover Bids Bypass Initial Collateral Ratio Enabling Self Liquidation	Critical	Closed
TRF-07	Arbitrary Function Execution In retrieveFunds Enables Complete Protocol Compromise	Critical	Resolved
TRF-08	EIP-712 Nested Struct Hashing Non-Compliance Breaks Wallet Signature Validation	Critical	Resolved
TRF-09	Missing Access Control In TermMorphoInterfaceFacet Allows Unauthorised Position Manipulation	Critical	Resolved
TRF-10	Inverted Validation Logic In RetrieveFunds Allows Invalid Function Calls	High	Resolved
TRF-11	Bid Update Collateral Array Mutation Enables Cross-User Locker Drain	High	Resolved
TRF-12	Partial Liquidation Net Exposure Cap Guard Always Bypassed, Enabling Bad Debt Creation	High	Closed
TRF-13	Users Can Incur Full Aave/Morpho Debt While Receiving Only A Partial Term Fill	High	Resolved
TRF-14	Wrong-Token retrieveFunds() Drains Diamond Balances Via Ignored Asset Check	High	Resolved
TRF-15	Intent Minting Bypasses And Inflates Mint Exposure Cap	High	Resolved
TRF-16	Blacklisted Borrower Can DoS Batch Liquidations Through Automatic Collateral Unlock	Medium	Resolved
TRF-17	_debit() Allows Burning Tokens Due To Missing Modifier	Medium	Closed
TRF-18	Missing Slippage Protection In Strategy Sales	Medium	Closed
TRF-19	Missing Slippage Bounds In ERC4626 Deposits And Withdrawals	Medium	Closed
TRF-20	Uninitialised ERC-165 Mapping Causes supportsInterface To Always Return false	Medium	Resolved
TRF-21	AccessControl Overrides Diamond supportsInterface	Medium	Resolved
TRF-22	Missing Slippage Protection In Aave Supply Operations	Medium	Resolved

TRF-23	User Tokens Can Be Locked In Facets	Medium	Resolved
TRF-24	Fund Theft Via Flash-Loan Borrower Context Leak And Public <code>swap()</code>	Medium	Resolved
TRF-25	Vault <code>FulfillOrder</code> Atomic Context Leaks To Malicious Vault Callback	Medium	Resolved
TRF-26	Vault <code>FulfillOrder</code> Strands Purchase Tokens Extractable Via Batch Settlement	Medium	Resolved
TRF-27	Collateral Dust From Liquidations Blocks Pro-Rata Redemption	Medium	Closed
TRF-28	Factory-Deployed Repos Permanently Locked Out Of Auction And Rollover-In Paths	Medium	Resolved
TRF-29	Failed Or Partially Filled Rollover Auction Permanently Revokes Re-Election Right	Medium	Closed
TRF-30	Flash Loan Hooks Over-Pull On Partial Fill, Draining Taker Balance	Medium	Closed
TRF-31	Non-Auction Origination Uses Maintenance Ratios Instead Of Initial Ratios	Medium	Closed
TRF-32	Non-Standard ERC20 Tokens Are Not Supported	Medium	Resolved
TRF-33	Partial Liquidation Leaves Locked Rollover Amount Above Live Debt	Medium	Closed
TRF-34	Repo-Token Liquidations Enable Preferential Recovery Against Selected Borrowers	Medium	Closed
TRF-35	Rollover Bids Bypass Initial Collateral Ratio Check	Medium	Closed
TRF-36	Silent Unlock Failures Strand Collateral And Double-Charge Borrowers	Medium	Closed
TRF-37	Unrevealed Rollover Bids Can Bypass The Public Auction Reveal Gate	Medium	Resolved
TRF-38	Intent Settlement Bypasses Protocol <code>servicingFee()</code> Enforcement	Medium	Closed
TRF-39	<code>WETHWrappingFacet</code> Is Unusable Through The Diamond	Medium	Resolved
TRF-40	Removal Of Minimum Tender Amount Check Allows Dust Rollover Bids	Low	Resolved
TRF-41	ERC20 Token Decimals Handling Vulnerabilities In <code>usdValueOfTokens()</code>	Low	Closed
TRF-42	Fixed Precision Does Not Cater For Various Tokens	Low	Resolved
TRF-43	Incorrect <code>closureAmount</code> Emitted In Repo Token Liquidation Events	Low	Resolved
TRF-44	Leftover Collateral From Batch Default Functions Becomes Inaccessible After <code>mintOpenExposure</code>	Low	Closed
TRF-45	<code>mintExposureCap</code> Is Defined Per Chain Causing Incorrect Accounting	Low	Closed
TRF-46	Incomplete Facet Cleanup On Last Selector Removal Risks EIP2535 Noncompliance	Low	Resolved
TRF-47	Unnecessary Payable Entry Points Allow Stuck ETH	Low	Resolved
TRF-48	Permissionless Redemption Enables Front-Run Griefing	Low	Closed
TRF-49	NatSpec Claims <code>SETTLER_ROLE</code> Restriction Which Is Not Implemented	Low	Resolved

TRF-50	Missing Oracle Validation	Low	Closed
TRF-51	Unsafe <code>uint160</code> Casting In <code>Permit2</code> Transfers Enables Accounting Manipulation	Low	Resolved
TRF-52	Use Of Deprecated <code>transfer()</code> Method	Low	Resolved
TRF-53	Missing Input Validation Enables DoS and Unexpected Behaviour	Low	Resolved
TRF-54	Pro-Rata Rounding Creates Permanently Stale Encumbered Collateral	Low	Closed
TRF-55	Rollover Approval Boundary Schedule Makes Rollover Assignment Impossible	Low	Resolved
TRF-56	Rollover Revocation Does Not Unwind Granted Roles	Low	Closed
TRF-57	Strategy Facet Rejects Factory-Deployed Repos	Low	Resolved
TRF-58	<code>TermFlashLoanExecutorFacet</code> Requires Multi-Action Plans	Low	Resolved
TRF-59	Cached EIP-712 Domain Separator Can Become Stale	Low	Resolved
TRF-60	Caller-Sensitive <code>vault.asset()</code> Enables Undeclared Diamond Balance Spend	Low	Resolved
TRF-61	Preview Hooks Hardcode Primary Collateral Index, Breaking Multi-Collateral Flash Plans	Low	Resolved
TRF-62	Shared Salt In Deterministic Deployment Enables Address Griefing	Low	Resolved
TRF-63	No Default Admin Role For Permission Management After Deployment	Informational	Closed
TRF-64	Generic Custom Error Handling Obscures Underlying Revert Reason	Informational	Resolved
TRF-65	Missing Duplicate Selector Check With Misleading Replace Revert	Informational	Resolved
TRF-66	Diamond Lacks Pausing Mechanism	Informational	Resolved
TRF-67	Owner Not Externally Queryable Via Loupe	Informational	Resolved
TRF-68	Add Only Cuts Can Execute Unintended Initialisers Via <code>_init</code>	Informational	Closed
TRF-69	No Ownership Transfer Function In Diamond	Informational	Closed
TRF-70	Misleading <code>NatSpec</code> In <code>TermRouterFacet</code>	Informational	Closed
TRF-71	Borrowers' Excess Collateral Remains Locked On Partial Bid Fills	Informational	Closed
TRF-72	Potential Overlapping Offer Reveal And Borrower Bid Commit Times	Informational	Closed
TRF-73	Bid Slot Exhaustion Enables Single Borrower Manipulation And Auction Cancellation	Informational	Closed
TRF-74	Offer Slot Exhaustion Enables Single Lender Manipulation	Informational	Closed
TRF-75	Weak Commit Hash Allows Price Guessing With Low Entropy Nonces	Informational	Closed
TRF-76	Code Quality Issues In BSC Flashloan Implementation	Informational	Resolved

TRF-77	Protocol Route Numbering System Issues	Informational	Closed
TRF-78	Incorrect Comments In <code>TermMorphoInterfaceFacet.sol</code>	Informational	Resolved
TRF-79	Unused Local Variable	Informational	Resolved
TRF-80	Documentation Claims Generic Gas Token Support But Code Is WETH-Specific	Informational	Resolved
TRF-81	Insufficient Validation In <code>permit</code> Function	Informational	Resolved
TRF-82	Factory Does Not Explicitly Reject Fee-On-Transfer Tokens	Informational	Closed
TRF-83	Inefficient Zero-Value Approvals in Collateral Flow	Informational	Resolved
TRF-84	<code>settleLimitLendHook()</code> Omits <code>nonReentrant</code> , Diverging From The Hook Safety Pattern	Informational	Closed
TRF-85	Unsafe <code>Permit2</code> Cast In <code>RepoToken</code> Swap Silently Truncates Transfer Amount	Informational	Resolved
TRF-86	Miscellaneous General Comments	Informational	Resolved

TRF-01 Cross-Chain Transfers Break Redemptions Or DoS The Contract Before Maturity			
Assets	contracts/TermRepoToken.sol contracts/TermRepoServicer.sol		
Status	Closed: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The `_debit()` function allows cross-chain transfers after the redemption period begins, causing critical accounting failures.

During redemptions, the contract relies on `totalRedemptionValue()` which depends on `totalSupply()`. When `_debit()` is called, it burns tokens and reduces `totalSupply()`, breaking fund accounting:

TermRepoToken.sol::_debit()

```
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
) internal override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    (amountSentLD, amountReceivedLD) = _debitView(_amountLD, _minAmountLD, _dstEid);

    // @dev In NON-default OFT, amountSentLD could be 100, with a 10% fee, the amountReceivedLD amount is 90,
    // therefore amountSentLD CAN differ from amountReceivedLD.

    // @dev Default OFT burns on src.
    _burn(_from, amountSentLD);
}
```

This creates two severe issues with cascading effects across the entire protocol:

- **Redemption value theft through timing attacks:** Early redeemers receive full value while late redeemers get nothing as `totalRedemptionValue()` shrinks with each cross-chain transfer. Since `totalRedemptionValue() = totalSupply() * redemptionValue`, when `_debit()` burns tokens during cross-chain transfers, it permanently reduces the total pool available for redemption. This allows attackers to:
 - Monitor the mempool for large redemption transactions
 - Front-run redeemers with cross-chain transfers to artificially deflate `totalSupply()`
 - Cause later redeemers to receive proportionally less value
 - Effectively steal redemption value from legitimate token holders
- **Complete protocol DoS through accounting invariant violations:** The `_isTermRepoBalanced()` function enforces critical accounting invariants that ensure the protocol's solvency. When cross-chain transfers burn tokens, they break the fundamental equation:

```
(totalOutstandingRepurchaseExposure + totalRepurchaseCollected) == termRepoToken
.totalRedemptionValue()
```

This causes systematic failures in:

- `mintOpenExposure()` - preventing new borrowing positions
- `submitRepurchasePayment()` - blocking debt repayments
- `burnCollapseExposure()` - disabling position closures

- All liquidation functions - preventing bad debt recovery

Once triggered, this creates an unrecoverable state where the protocol cannot process any new operations, effectively shutting down the entire protocol until addressed through manual intervention.

TermRepoServicer.sol::_isTermRepoBalanced()

```

if (shortfallHaircutMantissa == 0) {
    return
        (totalOutstandingRepurchaseExposure +
         totalRepurchaseCollected) /
        (10 ** 4) ==
        termRepoToken.totalRedemptionValue() / (10 ** 4); <<<
}

// @note in the case of shortfall, purchase currency in termRepoLocker must balance the pro rata redemption value of
//    ↳ remaining term repo tokens
return
    (totalRepurchaseCollected) / (10 ** 4) ==
    mul_ScalarTruncate(
        Exp({mantissa: shortfallHaircutMantissa}),
        termRepoToken.totalRedemptionValue() <<<
    ) /
    (10 ** 4);

```

Recommendations

Prevent cross-chain transfers after `redemptionTimeStamp` in the `_debit()` function.

Resolution

The development team has advised that they decided not to implement the OFT feature and, as such, the finding is now closed.

TRF-02	OFT Contract Incorrectly Initialised		
Assets	contracts/TermRepoToken.sol		
Status	Closed: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The `TermRepoToken` constructor initialises OFT state variables directly instead of using an initialiser function.

Since this contract follows the UUPS proxy pattern, the constructor executes only on the implementation contract, not on the proxy:

`TermRepoToken.sol::constructor()`

```

constructor(address _lzEndpoint, address _delegate) OFTCore(18, _lzEndpoint, _delegate) {
    _disableInitializers();
    _transferOwnership(_delegate);
}

```

The `OFTCore` constructor initialises LayerZero variables (`endpoint`, `delegate`, `decimals`) in the implementation contract's storage; however, the proxy contract's storage remains uninitialised.

Note, this is only a problem for the `_delegate` parameter, as it is the only mutable state variable. The other parameters are declared as `immutable` so they are stored directly in the implementation contract's bytecode and it is therefore safe to specify them in the constructor.

This oversight will cause the proxy contract to return `address(0)` for the `_delegate`, which is used as a parameter throughout the LayerZero OFT system for critical operations including cross-chain message validation, fee collection, and administrative functions.

When the proxy contract returns `address(0)` for the delegate:

- **Cross-chain message failures:** LayerZero's `lzReceive()` function may reject messages if the delegate validation fails, causing cross-chain transfers to permanently fail or become stuck.
- **Administrative lockout:** Many LayerZero administrative functions require delegate authorization, meaning the contract becomes unmanageable for cross-chain operations.
- **Security bypasses:** Some LayerZero security checks rely on delegate validation. A zero delegate may bypass intended access controls or cause unpredictable behaviour.

This creates a breakdown of cross-chain functionality, as the proxy contract (which users interact with) has an uninitialised delegate while the implementation contract (which is not directly accessible) has the correct delegate set.

Recommendations

Move the initialisation of the `_delegate` parameter from the constructor to the `initialize()` function to ensure proper proxy contract storage configuration.

Resolution

The development team has advised that they decided not to implement the OFT feature and, as such, the finding is now closed.

TRF-03	OFT Decimals Should Match decimalPlaces		
Assets	contracts/TermRepoToken.sol		
Status	Closed: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The `TermRepoToken` constructor hardcodes the `OFT` decimals to `18`, regardless of the underlying purchase token's actual decimals:

```
TermRepoToken.sol::constructor()
constructor(address _lzEndpoint, address _delegate) OFTCore(18, _lzEndpoint, _delegate) {
    _disableInitializers();
    _transferOwnership(_delegate);
}
```

This creates serious cross-chain accounting issues when the purchase token has different decimals on different chains. For example, USDC has 6 decimals on Ethereum Mainnet but 18 decimals on BNB Chain.

The hardcoded 18 decimal setting would create severe value discrepancies during cross-chain operations:

- **Value inflation:** When transferring from a 6-decimal chain (Ethereum USDC) to an 18-decimal chain, 1,000,000 tokens (representing \$1 USD) would be interpreted as 1,000,000,000,000,000 tokens (\$1 trillion USD) on the destination chain due to the 12-decimal difference.
- **Value destruction:** Conversely, transferring from an 18-decimal chain to a 6-decimal chain would cause 999,999,999,999 out of every 1,000,000,000,000 tokens to be lost due to truncation.
- **Redemption calculation failures:** The `totalRedemptionValue()` function multiplies `totalSupply()` by `redemptionValue`. With mismatched decimals, this calculation becomes meaningless and breaks the redemption mechanism.
- **Collateral accounting breakdown:** Since `TermRepoTokens` represent claims to repurchase obligations, decimal mismatches corrupt the fundamental 1:1 relationship between token supply and underlying collateral value.

Without decimal normalisation in `_debit()` and `_credit()`, cross-chain transfers will result in these massive value discrepancies.

Recommendations

Pass the purchase token's decimal value to the `OFTCore` constructor instead of hardcoding a value of 18.

Resolution

The development team has advised that they decided not to implement the `OFT` feature and, as such, the finding is now closed.

TRF-04	Missing Admin Setter For approvedTermControllers Causes Global Reverts		
Assets	contracts/TermRouterFacet.sol contracts/LibTermStorage.sol		
Status	Closed: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

All router validations read `approvedTermControllers` but nothing ever writes to this mapping, causing every call that reaches these checks to revert.

The validation functions perform a check against an empty mapping:

facets/TermRouterFacet.sol::_validateRepoServicer()

```
function _validateRepoServicer(ITermRepoServicer servicer) private view {
    TermStorage storage s = LibTermStorage.termStorage();
    ITermController termController = servicer.termController();
    if (!s.approvedTermControllers[address(termController)]) { // @audit Always false - never set
        revert InvalidTermController();
    }
    if (!termController.isTermDeployed(address(servicer))) {
        revert InvalidRepoId();
    }
}
```

facets/TermRouterFacet.sol::_validateCollateralManager()

```
function _validateCollateralManager(ITermRepoCollateralManager collateralManager) private view {
    TermStorage storage s = LibTermStorage.termStorage();
    ITermController termController = collateralManager.termController();
    if (!s.approvedTermControllers[address(termController)]) { // @audit Always false - never set
        revert InvalidTermController();
    }
    if (!termController.isTermDeployed(address(collateralManager))) {
        revert InvalidRepoId();
    }
}
```

facets/TermRouterFacet.sol::_validateRolloverManager()

```
function _validateRolloverManager(ITermRepoRolloverManager rolloverManager) private view {
    TermStorage storage s = LibTermStorage.termStorage();
    ITermController termController = rolloverManager.termController();
    if (!s.approvedTermControllers[address(termController)]) { // @audit Always false - never set
        revert InvalidTermController();
    }
    if (!termController.isTermDeployed(address(rolloverManager))) {
        revert InvalidRepoId();
    }
}
```

The storage mapping is declared but never written to:

libraries/LibTermStorage.sol

```
struct TermStorage {
    mapping(address => bool) approvedTermControllers; // @audit Never assigned to
}
```

This validation is called by multiple major router functions, including but not limited to:

- `submitRepurchasePayment()`
- `burnCollapseExposure()`
- `mintOpenExposure()`
- `redeemTermRepoTokens()`
- `externalLockCollateral()`
- `externalUnlockCollateral()`
- `electRollover()`
- `cancelRollover()`

If this mapping is never assigned to, there will be severe operational consequences:

- `TermRouterFacet` will be unusable, preventing all user interactions through the router interface.
- All router transactions will fail with confusing `InvalidTermController()` errors, regardless of the validity of the term controller.
- External systems expecting to interact through the router will be unable to function.
- Users will not be able to access core functionality such as repurchase payments, collateral management, or token redemptions through the router.
- The router facet will waste development effort and deployment costs as it cannot fulfil any of its intended purposes.

Recommendations

Implement proper initialisation and management of approved term controllers, setting values in `approvedTermControllers`.

Resolution

The development team has provided code outside the scope of the audit which allows a secure `DEV_OPS` role to call `revokeTermController()` as well as `approveTermController()`.

The testing team has informed the client that although this fix does address the issue of no writes to the controller mapping, revoking a role presents a new risk.

By allowing a term controller to be revoked at any point, all open positions and exposures will be affected. The `termRouterFacet` becomes unusable as many of the functions call `_validateRepoServicer()`, which reverts due to the fact that the mapping will return false for the specific term controller.

```
facets/TermRouterFacet.sol::_validateRepoServicer()
function _validateRepoServicer(ITermRepoServicer servicer) private view {
    TermStorage storage s = LibTermStorage.termStorage();
    ITermController termController = servicer.termController();
    if (!s.approvedTermControllers[address(termController)]) {
        revert InvalidTermController();
    }
    if (!termController.isTermDeployed(address(servicer))) {
        revert InvalidRepoId();
    }
}
```

The development team has acknowledged the risk and has provided the following response:

"revokeTermController is secured using the DEVOPS_ROLE, which will be held by a timelocked governance"

safe. The only situation a term controller will be revoked is if the controller admin wallet becomes compromised.

The controller admin wallet is a hot wallet that holds the `CONTROLLER_ADMIN_ROLE` which is responsible for `markTermDeployed` and `unmarkTermDeployed` function on the `TermController`. In that case, the `isTermDeployed` check we use to validate if a `TermRepoServicer` or `TermRepoToken` contract is legitimate is compromised, a malicious actor could start marking malicious contracts as `termDeployed`. That `termController` must be revoked to protect Router users from using malicious contracts. Legitimate `termDeployed` contracts may also be unmarked for term deployment and inaccessible.

Since revoking a term controller is timelocked, it will take 24 hours to execute and that will be a time period where we can direct users to repay their loans and remove their collateral. That is only possible of course if the compromised controller hasn't `unmarkTermDeployed` the `TermRepoServicer` and `TermRepoCollateral-Manager` contracts.

Those contracts will be available for direct access to extract assets for users, so in that situation we will direct them towards using those addresses."

TRF-05	Uninitialised AccessControl In TermDiamond		
Assets	contracts/TermDiamond.sol		
Status	Resolved: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The `TermDiamond` contract inherits OpenZeppelin's `AccessControl` but fails to properly initialise the access control system, rendering the role-based authentication non-functional.

The constructor does not call `__AccessControl_init()` or configure any roles, yet multiple Term contracts grant `SETTLER_ROLE` to the diamond address expecting it to function as an authenticated participant. This initialisation failure breaks the diamond's ability to validate roles when accessing role-protected functions across the protocol, potentially causing widespread functionality failures.

The diamond inherits access control but lacks proper initialisation:

TermDiamond.sol

```
contract TermDiamond is AccessControl {
    constructor(address _contractOwner, address _diamondCutFacet) payable {
        LibDiamond.setContractOwner(_contractOwner);

        // @audit
        // Missing: __AccessControl_init() or role setup
        // Missing: _setupRole(DEFAULT_ADMIN_ROLE, _contractOwner);

        // Diamond cut setup...
    }
}
```

Multiple protocol contracts grant roles to the diamond expecting it to be functional:

From various Term contracts

```
_grantRole(SETTLER_ROLE, diamond_); // @audit Expects diamond to validate this role
```

Without proper initialisation, the `AccessControl` system's internal mappings remain uninitialised:

- The `_roles` mapping is empty.
- `DEFAULT_ADMIN_ROLE` has no members.
- A role hierarchy is not established.
- Role validation will fail for all queries.

This particularly impacts facets that need to interact with role-protected functions across the protocol ecosystem.

The uninitialised access control system creates failures across the protocol:

- Role Validation Failures: The diamond cannot prove possession of granted roles when interacting with other protocol contracts.
- Function Call Rejections: Calls to role-protected functions in other contracts will revert, even when the diamond legitimately holds the required roles.

- Cross-Contract Integration Breakdown: The diamond cannot function as an authenticated protocol participant despite being designed for that purpose.
- Settler Functionality Disabled: Critical settlement operations that require `SETTLER_ROLE` will fail when routed through the diamond.
- Protocol Operational Failure: Core protocol workflows that depend on diamond-mediated operations will become non-functional.

This prevents the diamond from fulfilling its intended role in the protocol ecosystem.

Recommendations

Initialise the diamond's `AccessControl` system by calling `__AccessControl_init()` in an initialisation facet during deployment, using the `_init` parameter of `diamondCut()`.

Initialise a role management scheme. For example, grant `DEFAULT_ADMIN_ROLE` to an administrative address, then call `approveTermController()` for each valid `TermController` address.

Resolution

The development team has removed `AccessControl` as well as OpenZeppelin's `Pausable` from the `TermDiamond` and added them to the `DiamondCutFacet`.

The `TermDiamond` constructor now makes a call to `initDiamond` to grant any specified roles. This initialisation occurs atomically during deployment via a `delegatecall` within the diamond's `diamondCut` function, ensuring proper role setup without leaving any vulnerable initialisation windows.

To prevent replay attacks and unauthorised role initialisation, the implementation includes multiple security measures:

- A boolean flag in `LibDiamond` prevents multiple calls to `initDiamond`
- The function requires that `msg.sender` is the designated deployer wallet
- A block number check ensures initialisation can only occur in the same transaction as deployment

The fix can be found in the following commit: [f89fc36](#).

TRF-06 Rollover Bids Bypass Initial Collateral Ratio Enabling Self Liquidation			
Assets	TermAuctionBidLocker.sol TermAuction.sol		
Status	Closed: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

Rollover bids skip the initial collateral ratio validation applied to normal bids. A borrower can therefore elect rollover with an intentionally inflated bid price, win at an elevated clearing price, and have the new repurchase obligation become immediately liquidatable.

Normal term bids are checked against the initial collateral ratio at lock time:

TermAuctionBidLocker.sol

```
if (bidSubmission.amount < minimumTenderAmount) {
    revert BidAmountTooLow(bidSubmission.amount);
}
if (
    _isInInitialCollateralShortFall(
        bidSubmission.amount,
        bidSubmission.collateralTokens,
        bidSubmission.collateralAmounts
    )
) {
    revert CollateralAmountTooLow();
}
```

Rollover bids do not pass through this initial-ratio check. At reveal, they are only validated by `_isRolloverStillValid()`:

TermAuctionBidLocker.sol

```
if (!_isRolloverStillValid(revealedBid, pairOffServicer)) {
    ++unlockBidCount;
    // ... mark for unlock ...
}
```

This creates a profitable self-liquidation path:

1. A borrower with an existing term reaching maturity elects rollover with an inflated bid price.
2. The bid wins, raising the new clearing price and the borrower's repurchase obligation.
3. The resulting position falls below the maintenance collateral ratio immediately after assignment.
4. An alternate account completes the auction and liquidates the borrower in the same transaction, capturing liquidation incentives without relying on adverse market price movement.

The borrower can use this flow to leak value from the protocol by manufacturing a liquidatable position and then collecting the liquidation incentives through a controlled account.

Recommendations

Validate rollover bids against the initial collateral ratio using the final clearing price before assignment. Rollover bids that would be immediately liquidatable after assignment should be rejected or unlocked.

Resolution

The development team confirmed that this behaviour is intended design against `v2-release`.

TRF-07 Arbitrary Function Execution In <code>retrieveFunds</code> Enables Complete Protocol Compromise			
Assets	<code>contracts/TermRepoTokenIntentFacet.sol</code> <code>contracts/TermLoanIntentFacet.sol</code> <code>contracts/TermFlashLoanCentralReceiverFacet.sol</code>		
Status	Resolved: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The arbitrary function execution vulnerability affects multiple components in the Term Finance protocol:

- **Intent Facets:** The `retrieveFunds` mechanism in both the `TermRepoTokenIntentFacet.sol` and `TermLoanIntentFacet.sol` facets allows arbitrary function execution on the diamond contract during swapping, lending and borrowing operations, creating a critical backdoor that can bypass access controls and drain user funds.
- **Flash Loan Central Receiver:** The flash loan central receiver performs unconstrained `delegateCall()` execution to any registered diamond facet.

facets/TermRepoTokenIntentFacet.sol::_settleSwap()

```

if (retrieveFundsRequested){
    // generate retrieveFundsCalldata
    bytes memory retrieveFundsCalldata = _generateRetrieveFundsCalldata(
        order.retrieveFunds,
        order.makerAssetIsPurchaseToken ? purchaseTokenAmount : repoTokenAmount,
        order.makerAssetIsPurchaseToken ? purchaseToken : repoToken,
        order.maker
    );

    // execute retrieveFundsCalldata
    // @audit-issue Can execute any arbitrary call
    Address.functionCall(
        address(this),
        retrieveFundsCalldata
    );
    //... (end of function) ...
}

```

facets/TermLoanIntentFacet.sol::_settleLendOrder()

```

if (retrieveFundsRequested){
    // generate retrieveFundsCalldata
    bytes memory retrieveFundsCalldata = _generateRetrieveFundsCalldata(
        order.retrieveFunds,
        fillAmount,
        servicer.purchaseToken(),
        order.maker
    );

    // execute retrieveFundsCalldata
    Address.functionCall(
        address(this),
        retrieveFundsCalldata
    );
}

```

facets/TermLoanIntentFacet.sol::_settleBorrowOrder()

```

bytes memory retrieveFundsCalldata = _generateRetrieveFundsCalldata(
    order.retrieveFundsList[i],
    collateralAmounts[i],
    servicer.termRepoCollateralManager().collateralTokens(i),
    order.maker
);

// execute retrieveFundsCalldata
Address.functionCall(
    address(this),
    retrieveFundsCalldata
);

```

facets/flashloan/TermFlashLoanCentralReceiverFacet.sol::executeOperation()

```

(TermFlashLoanCallback memory callback, ) = abi.decode(data, (TermFlashLoanCallback, bytes));
// Validates selector exists and matches facet, but doesn't restrict WHICH functions
address actualFacet = IDiamondLoupe(address(this)).facetAddress(callback.selector);
if (actualFacet == address(o)) revert SelectorNotFound();
if (actualFacet != callback.callbackFacet) revert CallbackFacetSelectorMismatch();

bytes memory encodedData = abi.encodePacked(callback.selector, abi.encode/assets, amounts, premiums, initiator, data));

Address.functionDelegateCall(actualFacet, encodedData);

```

Attackers can embed malicious function calls within seemingly legitimate orders. When users fill these orders, the malicious functions execute automatically as part of the settlement.

Consider the following attack examples:

- Drain User Funds:

Attack Vector: Drain User Funds

```

// Attacker creates "innocent" swap order with hidden attack
RepoTokenSwapOrder memory order = RepoTokenSwapOrder({
    // ... normal swap parameters ...
    retrieveFunds: RetrieveFundsStruct({
        method: TermAaveInterfaceFacet.aaveWithdrawOnBehalfOf.selector,
        target: aavePool,
        additionalCalldata: abi.encode(asset, victim, largeAmount, false) // Drain victim's aTokens
    })
});

```

When a victim fills their swap, their `aTokens` are also withdrawn to contract.

- Admin Function Bypass (if contract has admin privileges):

Attack Vector: Admin Function Bypass

```

retrieveFunds: RetrieveFundsStruct({
    method: AdminFacet.setFeeRecipient.selector,
    target: address(o),
    additionalCalldata: abi.encode(attackerAddress)
})

```

Calls come from `address(this)`, bypassing admin checks.

- Cross-Facet Exploitation:

Attack Vector: Cross-Facet Exploitation

```

retrieveFunds: RetrieveFundsStruct({
    method: TermMorphoInterfaceFacet.morphoWithdraw.selector,
    // Exploit missing access controls in other facets
})

```

The allowance of arbitrary calls can cause a multitude of issues:

- **Complete protocol compromise:** Any diamond function can be executed
- **User fund theft:** Drain funds from Aave, Morpho, ERC4626 vaults via missing access controls in other facets
- **Access control bypass:** Execute admin functions if contract has elevated privileges
- **State manipulation:** Alter critical contract state during execution
- **Stealth attacks:** Malicious operations hidden within legitimate operations
- **Cross-facet exploitation:** Use vulnerabilities in any facet through this backdoor

The system is designed to allow arbitrary function calls for DeFi integration, but implements no meaningful restrictions on which functions can be called.

Recommendations

For intent facets, replace arbitrary execution with a controlled, purpose-built system that combines:

1. Dedicated Interface - Define exactly what `retrieveFunds` can do
2. Strict Whitelist - Only allow pre-approved operations
3. Parameter Validation - Ensure safe usage patterns

Create a single, controlled interface for all retrieve operations, e.g.:

interfaces/IRetrieveFunds.sol

```
interface IRetrieveFunds {
    function retrieveFromAave(address pool, address asset, uint256 amount, address user) external;
    function retrieveFromMorpho(address pool, bytes32 marketId, uint256 amount, address user) external;
    function retrieveFromVault(address vault, uint256 shares, address user) external;
}
```

These controls will introduce the following benefits:

- Explicit function signatures - No more arbitrary calldata
- Type safety - Compiler validates parameters
- Clear interface - Easy to audit what is possible
- Controlled execution - Each function has specific logic

For flash loan facet, implement function selector whitelist, e.g.:

facets/flashloan/TermFlashLoanCentralReceiverFacet.sol

```
mapping(bytes4 => bool) public allowedFlashLoanCallbacks;

modifier onlyAllowedCallback(bytes4 selector) {
    require(allowedFlashLoanCallbacks[selector], "Callback not whitelisted");
    _;
}
```

Resolution

The development team has addressed this vulnerability through multiple security layers - function selector whitelisting in `generateCalldata()` functions restricts execution to pre-approved operations, and the modifiers

added in the `TermAaveInterfaceFacet` and `TermMorphoInterfaceFacet` ensure interactions only occur with validated pool contracts.

This multi-layered approach eliminates arbitrary function execution while maintaining secure DeFi integration capabilities, reducing the attack surface to minimal, controlled operations on trusted contracts.

The fix can be found in the commit [876cac5](#).

TRF-08 EIP-712 Nested Struct Hashing Non-Compliance Breaks Wallet Signature Validation			
Assets	contracts/TermRepoTokenIntentFacet.sol contracts/TermLoanIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The `_getSwapOrderHash()`, `_getLendOrderHash()` and `_getBorrowOrderHash()` functions incorrectly hash the nested `RetrieveFundsStruct` using simple `abi.encode()` instead of proper EIP-712 specification, causing all signatures generated by standard wallets to fail validation.

Affected Functions:

- `TermRepoTokenIntentFacet._getSwapOrderHash()` line [210]
- `TermLoanIntentFacet._getLendOrderHash()` line [321]
- `TermLoanIntentFacet._getBorrowOrderHash()` line [339]

facets/TermRepoTokenIntentFacet.sol::_getSwapOrderHash()

```
return _getEIP712Hash(
    keccak256(abi.encode(
        _SWAP_ORDER_TYPEHASH,
        // ... other fields
    keccak256(abi.encode(order.retrieveFunds)) // ✗ INCORRECT: Not EIP-712 compliant
    ));
```

facets/TermLoanIntentFacet.sol::_getLendOrderHash()

```
return _getEIP712Hash(
    keccak256(abi.encode(
        _LEND_LIMIT_ORDER_TYPEHASH,
        // ... other fields
    keccak256(abi.encode(order.retrieveFunds)) // ✗ INCORRECT: Not EIP-712 compliant
    ));
```

facets/TermLoanIntentFacet.sol::_getBorrowOrderHash()

```
return _getEIP712Hash(
    keccak256(abi.encode(
        _BORROW_LIMIT_ORDER_TYPEHASH,
        // ... other fields
    keccak256(abi.encode(order.retrieveFunds)) // ✗ INCORRECT: Not EIP-712 compliant
    ));
```

According to EIP-712, nested structs must be hashed using their own type hash, not simple encoding. The current implementation fails to:

- Use a separate `RETRIEVE_FUNDS_STRUCT_TYPEHASH` for the nested struct
- Properly hash the bytes `additionalCalldata` field per EIP-712 requirements
- Follow the standard encoding format that wallets expect

The hash calculated by wallets using `signTypedData()` differs from the hash calculated by the contract, causing `_validateOrderSignature()` to always revert with `invalidSignature()`, leading to the following:

- **Complete system dysfunction:** All orders signed by standard wallets (Ethers, Viem, MetaMask) fail signature validation
- **No wallet integration possible:** Users cannot interact with the protocol through normal web3 wallets
- **Testing/development issues:** Signatures generated in development environments do not work in production
- **User experience breakdown:** Protocol appears broken to all standard wallet users

Recommendations

Implement proper EIP-712 nested struct hashing for both single structs and arrays. Define the spec-compliant type hash and implement correct hashing functions:

Recommended Fix for All Intent Facets

```
bytes32 private constant RETRIEVE_FUNDS_STRUCT_TYPEHASH = keccak256("RetrieveFundsStruct(bytes4 method,address target,bytes
↳ additionalCalldata)");
```

Recommended Fix for All Intent Facets

```
/// @notice Hashes a single RetrieveFundsStruct per EIP-712 specification
function _hashRetrieveFundsStruct(RetrieveFundsStruct memory item) private pure returns (bytes32) {
    return keccak256(abi.encode(
        RETRIEVE_FUNDS_STRUCT_TYPEHASH,
        item.method,
        item.target,
        keccak256(item.additionalCalldata) // bytes are hashed per EIP-712
    ));
}

/// @notice Hashes an array of RetrieveFundsStruct per EIP-712 specification
function _hashRetrieveFundsArray(RetrieveFundsStruct[] memory items) private pure returns (bytes32) {
    bytes32[] memory hashes = new bytes32[](items.length);
    for (uint256 i = 0; i < items.length; i++) {
        hashes[i] = _hashRetrieveFundsStruct(items[i]);
    }
    return keccak256(abi.encodePacked(hashes));
}
```

Then update the order hash functions to use proper EIP-712 encoding:

Recommended Fix for All Intent Facets

```
// For single struct (swap/lend orders)
_hashRetrieveFundsStruct(order.retrieveFunds)

// For array of structs (borrow orders)
_hashRetrieveFundsArray(order.retrieveFundsList)
```

Resolution

The development team resolved this issue by implementing EIP-712 compliant nested struct hashing with proper typehashes and encoding functions for `RetrieveFundsStruct`, ensuring wallet-generated signatures are correctly validated.

The fix can be found in [PR#1519](#).

TRF-09	Missing Access Control In TermMorphoInterfaceFacet Allows Unauthorised Position Manipulation		
Assets	contracts/TermMorphoInterfaceFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

All functions except `morphoSetAuthorizationWithSig()` lack access control, allowing any user to call functions on behalf of someone else. This enables attackers to completely control other users' Morpho positions, steal their funds, and manipulate their debt positions without permission.

The lack of access control opens the door to multiple manipulation attacks on users where funds can get locked in the facet, users could gain unexpected debt, and user positions could be forced into liquidation.

This attack vector exploits two weak points in the facet:

- The separation between Morpho's authorisation system (which correctly validates that `TermMorphoInterfaceFacet.sol` is authorised to act on behalf of users) and the facet's missing validation of who can initiate these authorised operations; AND
- The lack of any functionality to withdraw any stuck funds from the facet.

Consider the following attack scenarios:

• Locked User Funds

The `morphoWithdraw()` function withdraws a user's positions from the Morpho contracts. Due to the lack of access control, anyone is able to call this function on behalf of a user. Usually, allowing an attacker to pay the gas for a user's withdraw would not be an issue, however, due to the `payoutUser` parameter as well as the lack of recovery methods, this causes a critical vulnerability.

Attack Flow:

1. Alice authorises Term facet for legitimate use: `morphoSetAuthorizationWithSig()`
2. Attacker calls withdraw on the users behalf setting `payoutUser` to false:
`morphoWithdraw(pool, marketParams, assets, alice, false)`
3. Morpho validates that Term facet is authorised for the user.
4. Operation succeeds, user's funds transferred to Term facet
5. With `payoutUser=false`, funds become trapped in the facet

An attacker can monitor users positions and continuously withdraw and lock up the users funds inside the `TermMorphoInterfaceFacet.sol` facet.

• Increasing User Debt

Attackers can force users into unwanted debt positions by calling `morphoBorrow()` on their behalf without consent. If a user has authorised the `TermMorphoInterfaceFacet.sol` to manage their Morpho positions (required for Term Finance usage), attackers can force them to borrow maximum amounts against their collateral while directing the borrowed funds to the Term facet rather than the user. This creates several critical issues:

1. Users acquire substantial debt they never requested

2. Immediate interest accrual on forced loans
3. Increased liquidation risk from higher leverage
4. Potential unawareness of the new debt position until significant interest has accumulated or liquidation occurs.

- **Possible Liquidation**

Attackers can exploit this vulnerability to push users' positions to the edge of liquidation thresholds by forcing maximum borrowing against their collateral. In Morpho's system, liquidating undercollateralised positions is financially rewarding through built-in liquidation bonuses, creating strong economic incentives for attackers to manipulate healthy positions toward liquidation.

```
facets/external/TermMorphoInterfaceFacet.sol::morphoSupplyCollateral()
```

```
IERC20(marketParams.collateralToken).safeTransfer(msg.sender, seizedAssets);
```

- **Additional Scenario**

In conjunction with [TRF-51](#), an attacker could combine the two and create largely profitable positions.

1. Attacker withdraws multiple user positions with asset A.
2. Attacker then calls `aaveSupply()` with a very large input amount (`type(uint160).max + X`);
3. Attacker only spends `type(uint160).max` but the positions created is worth `type(uint160).max + X`, where X is money stolen from user positions the attacker has withdrawn.

Recommendations

The testing team recommends a two part fix:

1. Implement a recovery mechanism for locked ERC20s. By implementing this provides a recovery mechanism for any user based attacks or even general user error.
2. Add access control to allow only whitelisted actors to call the functions.

```
facets/external/TermMorphoInterfaceFacet.sol
```

```
modifier() {
  require(
    msg.sender == user ||
    msg.sender == multicalInitiator() ||
    isApprovedOperator(user, msg.sender),
    "Unauthorized caller"
  );
  _;
}
```

Resolution

The development team implemented access control via the `onlyUserOrActiveContext` modifier, preventing unauthorized users from manipulating other users' positions. The fix includes validation for direct calls, flash loan contexts, and active settlement maker operations, effectively addressing the unauthorized position manipulation vulnerability.

The fixes can be found in commits [8413f0f](#) and [e562f8e](#).

TRF-10	Inverted Validation Logic In RetrieveFunds Allows Invalid Function Calls		
Assets	contracts/TermRepoTokenIntentFacet.sol contracts/TermLoanIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

The `_validateRetrieveFunds()` function contains inverted validation logic that allows calls to non-existent functions while blocking calls to valid diamond facets.

The function checks if a method selector maps to a valid facet address and reverts if it finds one, which is the opposite of intended behaviour.

```
facets/TermLoanIntentFacet.sol::_validateRetrieveFunds()
if (retrieveFunds.method != bytes4(0)) {
    address facetAddress = IDiamondLoupe(address(this)).facetAddress(retrieveFunds.method);

    // Confirm that method belongs to specified facet
    if (facetAddress != address(0)) {
        revert InvalidRetrieveFundsFunction();
    }
    return true;
}
return false;
```

This logic flaw means:

- Valid diamond facet functions are blocked (when `facetAddress != address(0)`)
- Invalid/non-existent functions are allowed (when `facetAddress == address(0)`)
- Attackers could potentially specify malicious function selectors that don't exist in the diamond

The validation occurs before the function call in `_generateRetrieveFundsCalldata()` at line [369] inside `TermRepoTokenIntentFacet.sol`, where the same `facetAddress` lookup is performed but no additional validation prevents the subsequent `Address.functionCall()` execution.

This could allow attackers to specify invalid function selectors in swap orders, potentially leading to unexpected contract behaviour or bypassing intended access controls when the `retrieveFunds` mechanism is used.

Recommendations

Fix the validation logic to properly validate that the method exists in a legitimate facet.

```
facets/TermLoanIntentFacet.sol::_validateRetrieveFunds()
// Confirm that method belongs to specified facet
if (facetAddress == address(0)) {
    revert InvalidRetrieveFundsFunction();
}
```

Resolution

The finding has been resolved - the development team updated the inverted statement in commits [a72a4f9](#) and [244fdd5](#).

TRF-11 Bid Update Collateral Array Mutation Enables Cross-User Locker Drain			
Assets	contracts/TermAuctionBidLocker.sol contracts/TermRepoCollateralManager.sol contracts/TermRepoLocker.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

A bidder who updates an existing bid with a mutated collateral array can corrupt the stored bid's collateral accounting, drain other participants' collateral from the shared `TermRepoLocker` pool, and permanently strand their own collateral. Any update that changes the token identity at an existing array index, including reordering and duplicate-token substitutions, triggers the same root cause and leads to direct loss of funds for other auction participants.

`TermAuctionBidLocker._lock()` handles both initial creation and updates through a single path. When a bid already exists, the function computes a delta by comparing old and new `collateralAmounts` by array index without verifying that the token at each index matches. The lock and unlock calls operate on the token from the new submission, not the old one, and the stored bid is overwritten with the new arrays:

TermAuctionBidLocker.sol::_lock()

```
for (uint256 i = 0; i < bidSubmission.collateralTokens.length; ++i) {
    if (bidExists) {
        oldCollateralAmount = bids[bidId].collateralAmounts[i]; // @audit reads old amount at index i - but old token[i] may
        // differ from new token[i]
        if (oldCollateralAmount < bidSubmission.collateralAmounts[i]) {
            termRepoCollateralManager.auctionLockCollateral(
                sender,
                address(bidSubmission.collateralTokens[i]), // @audit operates on new token at index i
                bidSubmission.collateralAmounts[i] - oldCollateralAmount
            );
        } else if (oldCollateralAmount > bidSubmission.collateralAmounts[i]) {
            termRepoCollateralManager.auctionUnlockCollateral(
                bidSubmission.bidder,
                address(bidSubmission.collateralTokens[i]), // @audit operates on new token at index i
                oldCollateralAmount - bidSubmission.collateralAmounts[i]
            );
        }
    }
    // ... snip ...
}

bids[bidId] = TermAuctionBid({
    // ... snip ...
    collateralAmounts: bidSubmission.collateralAmounts, // @audit overwrites with mutated arrays
    collateralTokens: bidSubmission.collateralTokens,
    // ... snip ...
});
```

The simplest manifestation is a reorder. A bidder initially locks `[tokenA: 100, tokenB: 50]`, then submits an update in reverse order: `[tokenB: 50, tokenA: 100]`. The diff proceeds as follows:

- **Index 0:** old amount is 100 (which originally corresponded to token A), while the new amount is 50 for token B. `100 > 50`, so `auctionUnlockCollateral()` is called for token B with amount 50. Because unlocks are paid from the locker's pooled token B balance rather than a bidder-specific sub-ledger, the protocol releases 50 token B even though the 100-unit comparison came from the old token A slot.

- **Index 1:** old amount is 50 (which originally corresponded to token B), while the new amount is 100 for token A. $50 < 100$, so `auctionLockCollateral()` is called for **token A** with amount 50. The bidder sends 50 more token A to the locker.

After the update, the locker holds 150 of token A and 0 of token B from this bid, but the stored bid claims `[tokenB: 50, tokenA: 100]`. If another participant has deposited token B into the shared locker, the attacker can later receive 100 units of token B despite only ever contributing 50 – via pre-reveal withdrawal, post-cancellation withdrawal, or the auction's own unlock path. The other participant is left with a shortfall and may find their own unlock reverting once the shared token B balance is exhausted.

The same corruption appears in non-reorder updates. A bid starting as `[tokenA: 1, tokenB: 1, tokenC: 1]` updated to `[tokenA: 9, tokenB: 9, tokenB: 9]` succeeds but leaves the stored bid claiming `tokenA: 9, tokenB: 18, tokenC: 0` while the actual contribution is `tokenA: 9, tokenB: 17, tokenC: 1`.

When the corrupted bid is later returned, the protocol pays out according to the corrupted stored pairs. In the withdrawal path, `unlockBids()` calls `_unlock()`, iterating stored arrays to transfer out of the locker. In the auction path, `completeAuction()` / `cancelAuction()` obtain the same corrupted arrays from `getAllBids()` and forward them through `auctionUnlockBid()` to `auctionUnlockCollateral()`:

TermRepoLocker.sol::transferTokenToWallet()

```
function transferTokenToWallet(
    address destinationWallet,
    address token,
    uint256 amount
) external override ... {
    IERC20 tokenInstance = IERC20(token);
    tokenInstance.safeTransfer(destinationWallet, amount); // @audit draws from the shared pool – no per-bidder balance
    ↳ tracking
}
```

Because `TermRepoLocker` holds a single pooled balance per token across all bidders, the corrupted payout draws from whatever liquidity is present at unlock time. If other participants have locked the target token, the attacker exits with tokens this bid never contributed – turning accounting corruption into a cross-user drain. If insufficient liquidity is present, the unlock reverts and the attacker's displaced collateral becomes stranded. The duplicate-token example above demonstrates this directly: the stored bid requests 18 units of token B while only 17 were ever locked, so the remaining 1 unit of token C is orphaned.

No validation in `_lock()` enforces that the collateral token array is preserved between submissions or that entries are reconciled by token address. The only update-path checks – approved collateral type and initial collateral ratio – do not detect reordering, duplication, or token-identity mutation across indices. The router facet contains a comment stating that bid submissions should use the same ordering, but this is not enforced, and `TermAuctionBidLocker.lockBids()` remains directly callable by users who have approved the shared locker.

This issue has a high impact because the attack leads to direct loss of funds for other auction participants whenever the shared locker holds enough of the target token to satisfy the corrupted unlock. This issue has a medium likelihood because the exploit path is deterministic and reachable through normal bid-update flows whenever a repo accepts at least two distinct collateral tokens. If the relevant token liquidity is unavailable, the exploit degrades into a denial-of-withdrawal outcome rather than theft.

Recommendations

Either enforce that bid updates preserve the exact collateral token array (same length, same order, same token identity at each index) or normalise and diff collateral by token address rather than array index. At minimum, reject any update where `bidSubmission.collateralTokens[i] != bids[bidId].collateralTokens[i]` for any index `i`, require that the array lengths match, and reject duplicate collateral token entries within a single bid submission.

Resolution

The development team has fixed this issue in [PR#1597](#) by checking that the collateral tokens in the submission match the original bid's collateral tokens as follows:

TermAuctionBidLocker.sol::lockBids()

```
function lockBids(
    TermAuctionBidSubmission[] calldata bidSubmissions
    bidSubmission.collateralTokens[i]
);
}
// ... snip ...
if (bidExists) {
    if (bids[bidId].collateralTokens[i] != bidSubmission.collateralTokens[i]) {
        revert CollateralTokenMismatch(
            bids[bidId].collateralTokens[i],
            bidSubmission.collateralTokens[i]
        );
    }
}
// ... snip ...
}
```

TRF-12	Partial Liquidation Net Exposure Cap Guard Always Bypassed, Enabling Bad Debt Creation		
Assets	contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

A liquidator can call `batchLiquidation()` twice in sequence, first to selectively seize the highest-value collateral bucket and push the position into raw insolvency, then immediately again to seize all remaining collateral at a liquidation discount, fully draining a borrower who was only in haircut shortfall, while leaving lenders with a loss across the entire position rather than a marginal shortfall.

`batchLiquidation()` in `TermRepoCollateralManager.sol` allows any caller to partially repay a borrower's debt in exchange for a portion of their locked collateral plus a liquidation bonus. For positions that are not eligible for full liquidation (raw collateral market value exceeds the repurchase obligation), a post-liquidation guard is invoked to prevent the partial liquidation from leaving the borrower in an unacceptable state:

TermRepoCollateralManager.sol::batchLiquidation()

```
if (!allowFullLiquidation) {
    if (!_withinNetExposureCapOnLiquidation(borrower)) { // @audit guard is intended to block over-liquidation
        revert ExceedsNetExposureCapOnLiquidation();
    }
}
```

The guard function `_withinNetExposureCapOnLiquidation()` contains an early-return branch that makes the remainder of the function, including the `netExposureCapOnLiquidation` cap check, permanently unreachable for any position eligible for liquidation:

TermRepoCollateralManager.sol::_withinNetExposureCapOnLiquidation()

```
function _withinNetExposureCapOnLiquidation(address borrower) internal view returns (bool) {
    // ... snip ...

    // haircutUSDTotalCollateralValue computed using initialCollateralRatios

    if (
        lessThanExp(haircutUSDTotalCollateralValue, borrowerRepurchaseValue)
    ) {
        return true; // @audit unconditionally allows any partial liquidation
    }

    // @audit the code below – including the netExposureCapOnLiquidation check – is dead
    Exp memory excessEquity = sub_(haircutUSDTotalCollateralValue, borrowerRepurchaseValue);
    return lessThanOrEqualExp(
        div_(excessEquity, borrowerRepurchaseValue),
        Exp({mantissa: netExposureCapOnLiquidation})
    );
}
```

The early return triggers whenever the haircut-adjusted (using `initialCollateralRatios`) collateral value is below the repurchase obligation. This is exactly the condition that must hold for any position eligible for liquidation: `isBorrowerInShortfall()` requires the haircut-adjusted collateral (using `maintenanceCollateralRatios`) to be less than the obligation. Since `initialCollateralRatio >= maintenanceCollateralRatio` by protocol invariant, the initial-ratio haircut is always equal to or smaller than the maintenance-ratio haircut. Whenever a position is in shortfall, the early return fires unconditionally, returning `true` and bypassing the cap entirely.

The concrete two-call attack path is:

1. A borrower is in haircut shortfall but raw-solvent: `isBorrowerInShortfall() == true` and `getCollateralMarketValue() > usdValueOfTokens(purchaseToken, repurchaseObligation)`. This is the normal liquidation-window state for a well-collateralised multi-token position.
2. **Call 1** — the attacker calls `batchLiquidation(borrower, [X, 0, ...])`, passing a non-zero amount only for the highest-value collateral token. The `_withinNetExposureCapOnLiquidation()` guard returns `true` (early exit). After this call, `getCollateralMarketValue(borrower) < usdValueOfTokens(purchaseToken, remainingDebt)`: the position is now raw-insolvent and `allowFullLiquidation` becomes `true`.
3. **Call 2** — the attacker immediately calls `batchLiquidation(borrower, [0, Y, ...])`, targeting remaining collateral. Because `allowFullLiquidation == true`, the guard is skipped by design and remaining collateral is seized at a further liquidation discount.
4. The borrower is fully drained across both calls. Lenders receive less than par, the loss is the cumulative liquidation bonus extracted across all collateral.

The `netExposureCapOnLiquidation` parameter, which the factory validates as non-zero (`> 1e16`), is rendered permanently inert by this flaw. The identical guard is invoked by `batchLiquidationWithRepoToken()` and is equally affected.

This issue has a high impact because a two-call liquidation sequence can fully drain a borrower who was only in haircut shortfall, converting a marginal shortfall into a complete collateral loss for lenders. This issue has a medium likelihood because the attack path is deterministic and permissionless for any liquidator, but requires a multi-collateral position in the haircut-shortfall-but-raw-solvent state, which is the normal liquidation window for well-collateralised positions.

Recommendations

Remove the early-return branch in `_withinNetExposureCapOnLiquidation()`. Post-liquidation haircut-adjusted collateral below the remaining obligation indicates an over-aggressive partial liquidation, not a reason to permit it. Additionally verify that post-liquidation raw collateral market value does not fall below the remaining repurchase obligation, ensuring partial liquidations cannot push a raw-solvent position into insolvency.

Resolution

This issue has been acknowledged. The development team has confirmed that this is the intended design:

"Lenders would prefer to cut their losses than try to hang on indefinitely while the situation can get materially worse."

TRF-13	Users Can Incur Full Aave/Morpho Debt While Receiving Only A Partial Term Fill		
Assets	contracts/facets/external/TermAaveInterfaceFacet.sol contracts/facets/external/TermMorphoInterfaceFacet.sol contracts/facets/ERC4626InterfaceFacet.sol contracts/facets/TermLoanIntentFacet.sol contracts/TermRepoServicer.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

The Aave and Morpho fulfill-order helpers execute the full external borrow or withdrawal before settlement knows how much of the target order is still fillable. If the order has only partial remaining capacity, the user still takes the full external debt or collateral reduction while only receiving a partial Term fill.

All four affected entry points follow the same pattern: they perform the external protocol action first, pass the full requested amount into settlement, and rely on settlement to clamp the fill if the order is already partially filled.

In `aaveBorrowFulfillOrder()`, the full borrow is committed to Aave before settlement begins:

TermAaveInterfaceFacet.sol::aaveBorrowFulfillOrder()

```
_aaveBorrowInternal(aavePool, asset, borrowAmount, msg.sender, false);
// ... snip ...
TermLoanIntentFacet(address(this)).settleLimitBorrow(
    params.order,
    msg.sender,
    borrowAmount,
    params.signature
);
```

`_aaveBorrowInternal()` creates real variable-rate debt on the caller's position. When `payoutToUser_` is false and an atomic transaction is active, the borrowed assets remain in the diamond:

TermAaveInterfaceFacet.sol::_aaveBorrowInternal()

```
function _aaveBorrowInternal(
    address aavePool_, address asset_, uint256 amount_,
    address onBehalfOf_, bool payoutToUser_
) internal {
    // ... snip ...
    IPool(aavePool_).borrow(asset_, amount_, 2, 0, onBehalfOf_);
    if ((atomicTxInitiator() == address(0) && getFlashLoanBorrower() == address(0))
        && msg.sender != address(this)) || payoutToUser_ {
        IERC20(asset_).safeTransfer(onBehalfOf_, amount_);
    }
}
```

Settlement then caps the fill via `_calculateAndValidateFillAmount()`:

TermLoanIntentFacet.sol::_calculateAndValidateFillAmount()

```
function _calculateAndValidateFillAmount(
    uint256 fillAmount,
    uint256 totalAmount,
    uint256 filledAmount
) private pure returns (uint256) {
    uint256 maxFill = totalAmount - filledAmount;
    uint256 actualFillAmount = Math.min(fillAmount, maxFill);
    if (actualFillAmount == 0) { revert NothingToFill(); }
    return actualFillAmount;
}
```

The same pattern exists in:

- `aaveBorrowFulfillOrder()` in `TermAaveInterfaceFacet.sol`: borrows full amount at line [382], passes full borrowAmount at line [392].
- `aaveWithdrawFulfillOrder()` in `TermAaveInterfaceFacet.sol`: withdraws full amount at line [308], passes full withdrawAmount at line [318].
- `morphoBorrowFulfillOrder()` in `TermMorphoInterfaceFacet.sol`: borrows full amount at line [378], passes full assets at line [389].
- `morphoWithdrawCollateralFulfillOrder()` in `TermMorphoInterfaceFacet.sol`: withdraws full collateral at line [301], passes full assets at line [312].

There is no cleanup path after the fill is capped. If a user calls `aaveBorrowFulfillOrder()` for 500 USDC against an order with only 100 USDC of remaining capacity, the user still incurs 500 USDC of Aave debt while settlement only uses 100 USDC. The remaining 400 USDC stays in the diamond.

For the withdraw variants, the same bug reduces the user's external collateral by the full requested amount even though settlement only uses a partial fill. That worsens the user's health factor without a matching Term position increase.

The partial-fill condition can arise naturally or be MEV-induced by reducing the order's remaining capacity before the transaction executes.

This issue has a high impact because users suffer both direct token loss (stranded in diamond) and cross-protocol liquidation risk (unmatched external debt or reduced external collateral). This issue has a medium likelihood because exploitation requires partial-fill conditions that can be MEV-induced, and the entry points are public.

Recommendations

Cap the external borrow or withdrawal to the order's remaining fill capacity before calling the external protocol. If settlement can still clamp the fill, the function must immediately repay, re-deposit, or refund any excess before returning.

Resolution

The development team has resolved this issue in [PR#1598](#).

The vulnerable fulfill-order helpers were removed and replaced with settlement-with-hook entry points that check remaining order capacity before retrieving funds from external protocols. Direct intent settlement now also reverts when the requested fill exceeds the order's remaining capacity, preventing external debt or withdrawals from being incurred for amounts that settlement will not use.

TRF-14 Wrong-Token retrieveFunds() Drains Diamond Balances Via Ignored Asset Check			
Assets	contracts/facets/ERC4626InterfaceFacet.sol contracts/facets/external/TermMorphoInterfaceFacet.sol contracts/facets/TermLoanIntentFacet.sol contracts/facets/TermRepoTokenIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

The settlement layer treats any successful `retrieveFunds()` callback as proof that the expected token reached the diamond, but it never verifies that token actually arrived. Morpho and ERC4626 both accept an `asset` parameter for this path, yet neither adapter enforces it.

A maker can therefore source the wrong token through `retrieveFunds()`, have settlement mark the leg as routed anyway, and make downstream settlement spend pre-existing diamond balances of the expected token.

In the Morpho adapter, `generateCalldata()` accepts `asset` but ignores it and derives the token from a maker-chosen `marketId`:

TermMorphoInterfaceFacet.sol::generateCalldata()

```
function generateCalldata(
    bytes4 selector,
    address morphoPool,
    address asset,
    address user,
    uint256 amount,
    bool payoutUser,
    bytes calldata data
) external view approvedMorphoPoolOnly(morphoPool) returns (bytes memory) {
    bytes32 marketIdBytes = abi.decode(data, (bytes32));
    Id marketId = Id.wrap(marketIdBytes);
    MarketParams memory marketParams = IMorpho(morphoPool).idToMarketParams(marketId);
    if (selector == WITHDRAW_COLLATERAL_SELECTOR) {
        return abi.encodeWithSelector(selector, morphoPool, marketParams, amount, user, payoutUser);
    } else if (selector == BORROW_SELECTOR) {
        return abi.encodeWithSelector(selector, morphoPool, marketParams, amount, user, payoutUser);
    }
    // ... snip ...
}
```

The ERC4626 adapter has the same gap:

ERC4626InterfaceFacet.sol::generateCalldata()

```
function generateCalldata(
    bytes4 selector,
    address vault,
    address asset,
    address user,
    uint256 amount,
    bool payoutUser,
    bytes calldata data
) external pure returns (bytes memory) {
    if (selector == WITHDRAW_SELECTOR) {
        return abi.encodeWithSelector(selector, vault, amount, user, false, payoutUser);
    }
    // ... snip ...
}
```

Execution later re-reads `vault.asset()`:

ERC4626InterfaceFacet.sol::_withdrawFromVault()

```
function _withdrawFromVault(
    address vault_,
    uint256 assets_,
    address user_,
    address receiver_,
    bool usePermit2_
) internal returns (uint256) {
    IERC4626 vaultContract = IERC4626(vault_);
    IERC20 asset = IERC20(vaultContract.asset());
    uint256 shares = vaultContract.withdraw(assets_, receiver_, address(this));
}
```

Settlement never checks that the expected token actually arrived before marking the leg as routed:

TermRepoTokenIntentFacet.sol::_settleSwap()

```
Address.functionCall(
    address(this),
    retrieveFundsCalldata
);
// ...
if (order.makerAssetIsPurchaseToken) {
    isRoutedPurchaseToken = true;
} else {
    isRoutedRepoToken = true;
}
```

Once the routed flag is set, downstream settlement uses the diamond's balance instead of `safeTransferFrom()` from the maker. An attacker can exploit this as follows:

1. A pre-existing `purchaseToken` balance is present in the diamond.
2. The attacker creates a `retrieveFunds()` route that sources a different token through a Morpho market or ERC4626 vault.
3. The attacker, acting as borrower, settles against the lend order.
4. Settlement executes the callback and sets `isRoutedPurchaseToken = true` without checking `purchaseToken` arrival.
5. `_settlePurchaseTokensAndFees()` transfers `purchaseToken` to the borrower from the diamond's balance.
6. The wrong token remains in the diamond while the correct token balance is depleted.

This issue has a high impact because pre-existing diamond-held balances of the expected token can be consumed to satisfy maker obligations using only maker-signed order parameters. This issue has a medium likelihood because exploitation requires the diamond to already hold that token balance, but this is realistic given the protocol's composite flows and known stranding paths.

Recommendations

Verify the expected token arrived before marking a `retrieveFunds()` leg as routed.

Resolution

This issue has been fixed in [PR#1622](#).

A balance check has been added to make sure that the expected token has arrived at the expected address.

TRF-15 Intent Minting Bypasses And Inflates Mint Exposure Cap			
Assets	contracts/TermRepoServicer.sol contracts/TermRepoToken.sol contracts/facets/TermRouterFacet.sol contracts/facets/TermLoanIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

A self-matched intent cycle can inflate `mintExposureCap` without bound, defeating the governance-set risk limit on repo token minting. By minting through the intent path (which does not decrement the cap), transferring the resulting repo tokens to the borrower, and burning them via `burnCollapseExposure()` (which increments the cap), an attacker ratchets the cap upward by the minted amount each iteration. Positions remain collateral-backed throughout, so this is a risk-limit bypass rather than an insolvency vector.

`TermRepoToken` maintains a `mintExposureCap` that caps how many repo tokens can be minted for a given Term Repo deployment. The auction-based minting path in `_mintOpenExposureInternal()` correctly decrements this cap after minting:

TermRepoServicer.sol::_mintOpenExposureInternal()

```
function _mintOpenExposureInternal(
    address borrower, address sender, uint256 amount, uint256[] calldata collateralAmounts
) internal {
    // ... snip ...

    uint256 minterTokensValue = termRepoToken.mintTokens(msg.sender, minterTokens);
    termRepoToken.decrementMintExposureCap(amount); // @audit cap enforced on auction path

    // ... snip ...
}
```

However, the new intent-based minting path in `mintOpenExposureFromIntent()` has this call commented out:

TermRepoServicer.sol::mintOpenExposureFromIntent()

```
function mintOpenExposureFromIntent(
    address borrower, address lender, uint256 purchaseTokenAmount,
    uint256[] calldata collateralAmounts, uint256 offerRate, bool isRoutedCollateral
) external onlyRole(DIAMOND_ROLE) returns (uint256) {
    // ... snip ...

    uint256 totalMintValue = termRepoToken.mintTokens(lender, amountOfRepoTokenToMint);

    // TODO: assess whether this is needed for intents
    //termRepoToken.decrementMintExposureCap(amountOfRepoTokenToMint); // @audit cap NOT decremented

    // ... snip ...
}
```

`mintTokens()` performs no cap validation. It only checks pause state and the `MINTER_ROLE`, then calls `_mint()` directly. The cap is enforced only through `decrementMintExposureCap()`, which the intent path omits.

This omission alone allows intent-originated exposure to exceed the cap. The issue is amplified by the burn path: both `burn()` and `burnAndReturnValue()` in `TermRepoToken` increment the cap when tokens are destroyed:

TermRepoToken.sol::burnAndReturnValue()

```
function burnAndReturnValue(
    address account,
    uint256 amount
) external override onlyRole(BURNER_ROLE) whileBurningNotPaused
    whileTermContractsNotPaused(ITermRepoServicer(config.termRepoServicer).termController())
    returns (uint256)
{
    _burn(account, amount);
    mintExposureCap += amount; // @audit cap restored on every burn
    // ... snip ...
}
```

The `burnCollapseExposure()` function is publicly accessible via `TermRouterFacet` and calls `burnAndReturnValue()` internally. This creates a deterministic cap inflation loop:

1. **Account A** (maker) signs a lend order. **Account B** (taker) settles it via `settleLimitBorrow()`, providing collateral. `mintOpenExposureFromIntent()` mints repo tokens to Account A. Cap change: **0** (no decrement).
2. Account A **transfers** the repo tokens to Account B (standard ERC20 transfer).
3. Account B calls `burnCollapseExposure()` via `TermRouterFacet`, burning the repo tokens and collapsing the debt. `burnAndReturnValue()` executes `mintExposureCap += amount`. Cap change: **+amount**.
4. Account B's debt reaches zero, collateral is **unlocked** and returned.
5. **Repeat** with the same collateral. Each cycle inflates the cap by `amount`.

After N cycles, `mintExposureCap = original + (N * amount)`. The cap grows without bound, making the governance-set limit unenforceable. Any future auction-path minting will pass the `decrementMintExposureCap()` check regardless of the original limit. The cost per cycle is gas plus any intent settlement fees, negligible relative to the governance control being defeated.

This issue has a medium impact because the cap bounds tail-risk exposure (oracle failure, collateral depeg) independently of collateral backing; defeating it removes a protocol-level safety control but does not directly cause insolvency. This issue has a high likelihood because both intent settlement and `burnCollapseExposure()` are permissionless, and nothing prevents the maker and taker from being the same party.

Recommendations

Uncomment the `decrementMintExposureCap()` call in `mintOpenExposureFromIntent()` to enforce the same exposure cap on the intent path as on the auction path.

Resolution

The development team has fixed this issue in [PR#1632](#).

The corresponding line has been uncommented as recommended to decrease the mint exposure cap.

TRF-16	Blacklisted Borrower Can DoS Batch Liquidations Through Automatic Collateral Unlock		
Assets	contracts/TermRepoCollateralManager.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

Batch liquidation functions automatically unlock remaining collateral to borrowers when their repurchase obligation reaches zero, causing liquidations to fail if the borrower is blacklisted in blacklistable tokens like `USDC` or `USDT`.

Both `batchLiquidation()` and `batchLiquidationWithRepoToken()` in `TermRepoCollateralManager` contain identical vulnerable logic:

TermRepoCollateralManager.sol::batchLiquidation()

```
// ... function logic
// unencumber all collateral tokens owned by borrower if balance paid off
if (termRepoServicer.getBorrowerRepurchaseObligation(borrower) == 0) {
    _unencumberRemainingBorrowerCollateralOnZeroObligation(borrower);
    for (uint256 i = 0; i < collateralTokens.length; ++i) {
        if (lockedCollateralLedger[borrower][collateralTokens[i]] > 0) {
            _unlockCollateral( // @audit - will revert if borrower is blacklisted for USDT/USDC collateral
                borrower,
                collateralTokens[i],
                lockedCollateralLedger[borrower][collateralTokens[i]],
                false
            );
        }
    }
}
```

TermRepoCollateralManager.sol::batchLiquidationWithRepoToken()

```
// ... function logic
// unencumber all collateral tokens owned by borrower if balance paid off
if (termRepoServicer.getBorrowerRepurchaseObligation(borrower) == 0) {
    _unencumberRemainingBorrowerCollateralOnZeroObligation(borrower);
    for (uint256 i = 0; i < collateralTokens.length; ++i) {
        if (lockedCollateralLedger[borrower][collateralTokens[i]] > 0) {
            _unlockCollateral( // @audit - will revert if borrower is blacklisted for USDT/USDC collateral
                borrower,
                collateralTokens[i],
                lockedCollateralLedger[borrower][collateralTokens[i]],
                false
            );
        }
    }
}
```

The `_unlockCollateral()` function attempts to transfer tokens directly to the borrower:

TermRepoCollateralManager.sol::_unlockCollateral()

```
// ... requirements
termRepoLocker.transferTokenToWallet(borrower, collateralToken, amount); // @audit - attempts direct transfer
// ... event emitted
```

When a borrower is blacklisted in `USDC` (or other blacklistable tokens), the `transferTokenToWallet` call reverts, causing the entire batch liquidation transaction to fail.

A borrower using `USDC` collateral can get themselves blacklisted by `USDC` (through various means like mixing services or regulatory issues) after their position becomes liquidatable. When liquidators attempt to pay off the borrower's debt via full batch liquidation, the automatic collateral unlock will fail, preventing any liquidation from completing and potentially causing the protocol to accumulate bad debt if the position becomes severely underwater.

Recommendations

Remove the token transfers from these liquidation functions. Instead, separate collateral that is meant to be returned to borrowers after liquidation from the rest of the collateral and allow it to be claimed in a separate transaction. By doing so, users will be free to open new positions without mixing collaterals.

Resolution

The development team has addressed this vulnerability in [PR#1462](#) by implementing a try-catch mechanism around collateral transfer operations. When a transfer to a blacklisted borrower fails, the implementation redirects the funds to the protocol's reserve address instead of causing the entire liquidation transaction to revert. This prevents the accumulation of bad debt while ensuring liquidation operations can complete successfully.

However, it remains critical that `termController.getProtocolReserveAddress()` consistently returns a valid reserve address. Failure to maintain this invariant could result in funds being misdirected to an incorrect address or transaction failures that would reintroduce the original bad debt accumulation vulnerability.

TRF-17 <code>_debit()</code> Allows Burning Tokens Due To Missing Modifier			
Assets	contracts/TermRepoToken.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

The `_debit()` function lacks the `whileBurningNotPaused` modifier, creating a gap in the pause mechanism.

When burning is paused, cross-chain transfers can still proceed because `_debit()` directly calls `_burn()` without the pause check.

TermRepoToken.sol::_debit()

```
(amountSentLD, amountReceivedLD) = _debitView(_amountLD, _minAmountLD, _dstEid);

// @dev In NON-default OFT, amountSentLD could be 100, with a 10% fee, the amountReceivedLD amount is 90,
// therefore amountSentLD CAN differ from amountReceivedLD.

// @dev Default OFT burns on src.
_burn(_from, amountSentLD);
```

This allows tokens to be burned during cross-chain operations even when the contract administrator has explicitly paused burning functionality.

Recommendations

Add the `whileBurningNotPaused` modifier to the `_debit()` function:

TermRepoToken.sol::_debit()

```
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
- ) internal override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
+ ) internal override whileBurningNotPaused returns (uint256 amountSentLD, uint256 amountReceivedLD) {
```

Resolution

The development team has advised that they decided not to implement the OFT feature and, as such, the finding is now closed.

TRF-18	Missing Slippage Protection In Strategy Sales		
Assets	contracts/TermStrategyFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

The `sellRepoToken()` function in `TermStrategyFacet` does not implement slippage protection through minimum proceeds validation, exposing users to potential value extraction through manipulated exchange rates, adverse market timing, or MEV attacks.

The function performs repo token sales without user-defined slippage bounds:

facets/TermStrategyFacet.sol::sellRepoToken()

```
function sellRepoToken(
    IStrategy strategy,
    address repoToken,
    uint256 repoTokenAmount
) external nonReentrant returns (uint256 proceeds) {
    address asset = strategy.asset();
    uint256 preProceedsBalance = IERC20(asset).balanceOf(address(this));

    IERC20(repoToken).safeTransferFrom(msg.sender, address(this), repoTokenAmount);
    IERC20(repoToken).approve(address(strategy), repoTokenAmount);

    strategy.sellRepoToken(repoToken, repoTokenAmount);
    uint256 postProceedsBalance = IERC20(asset).balanceOf(address(this));

    if (postProceedsBalance <= preProceedsBalance) {
        revert NoProceedsReceived();
    }
    proceeds = postProceedsBalance - preProceedsBalance;
    IERC20(asset).safeTransfer(msg.sender, proceeds); // @audit No minimum output validation
}
```

The only protection is ensuring some proceeds are received (`postProceedsBalance > preProceedsBalance`), but this provides no meaningful slippage protection as even minimal proceeds (e.g., 1 wei) would satisfy this condition.

The absence of slippage protection can open attack vectors and user experience issues. Sandwich attacks can manipulate strategy pricing before and after the user's transaction to extract value. Users cannot set acceptable loss thresholds, reducing their ability to manage risk.

This particularly affects larger transactions where slippage and manipulation risks are amplified, potentially causing significant financial losses for users.

Recommendations

Implement slippage protection by adding a minimum proceeds parameter:

facets/TermStrategyFacet.sol::sellRepoToken()

```

function sellRepoToken(
    IStrategy strategy,
    address repoToken,
    uint256 repoTokenAmount
    uint256 minProceeds // @audit Add slippage protection parameter
) external nonReentrant returns (uint256 proceeds) {
    address asset = strategy.asset();
    uint256 preProceedsBalance = IERC20(asset).balanceOf(address(this));

    IERC20(repoToken).safeTransferFrom(msg.sender, address(this), repoTokenAmount);
    IERC20(repoToken).approve(address(strategy), repoTokenAmount);

    strategy.sellRepoToken(repoToken, repoTokenAmount);
    uint256 postProceedsBalance = IERC20(asset).balanceOf(address(this));

    if (postProceedsBalance <= preProceedsBalance) {
        revert NoProceedsReceived();
    }
    proceeds = postProceedsBalance - preProceedsBalance;

    if (proceeds < minProceeds) { // @audit Check minimum output
        revert InsufficientProceeds(proceeds, minProceeds);
    }

    IERC20(asset).safeTransfer(msg.sender, proceeds);
}

```

This allows users to specify their acceptable minimum return and protects against various forms of value extraction.

Resolution

The development team has stated that the slippage protection will not be needed due to the token price being calculated through a known time weighted fixed rate and have provided the following response.

"Slippage protection is not needed for sellRepoToken since repoToken prices are known time weighted fixed rates."

TRF-19	Missing Slippage Bounds In ERC4626 Deposits And Withdrawals		
Assets	contracts/ERC4626InterfaceFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

The ERC4626 wrapper functions in `ERC4626InterfaceFacet` lack caller-provided slippage guards, exposing users to unfavourable share/asset outcomes due to vault fees or rounding differences.

facets/ERC4626InterfaceFacet.sol::depositToVault()

```
function depositToVault(address vault, uint256 assets, address receiver)
    external nonReentrant returns (uint256 shares)
{
    // ...
    shares = vaultContract.deposit(assets, receiver); // @audit No minShares bound
    // ...
}
```

facets/ERC4626InterfaceFacet.sol::withdrawFromVault()

```
function withdrawFromVault(address vault, uint256 assets, address receiver)
    external nonReentrant returns (uint256 shares)
{
    uint256 sharesToRedeem = vaultContract.previewWithdraw(assets);
    // ...
    shares = vaultContract.withdraw(assets, receiver, address(this)); // @audit No maxShares bound

    if (shares != sharesToRedeem) {
        revert SharesRedeemedMismatch(shares, sharesToRedeem);
    }
}
```

The absence of slippage protection creates several user experience and financial risks:

- **Deposit Slippage:** Users may receive significantly fewer shares than expected due to vault fee changes, price movements, or MEV attacks occurring between transaction submission and execution.
- **Withdrawal Slippage:** Users may spend significantly more shares than expected due to vault fee changes, price movements, or MEV attacks occurring between transaction submission and execution.
- **Usability Degradation:** Users cannot express their slippage tolerance, creating a reduced user experience for advanced users.

These issues particularly affect users during volatile market conditions.

Recommendations

- Add slippage-aware overloads and enforce bounds:
 - `depositToVaultWithMin(vault, assets, receiver, minShares)` requiring `actualSharesReceived >= minShares`.
 - `withdrawFromVaultWithMin(vault, assets, receiver, maxShares, minAssets)` requiring `actualAssetsReceived >= minAssets` and `sharesToRedeem <= maxShares`.

- Alternatively, support `redeem(shares, receiver, owner)` with `minAssets` to bound proceeds.
- Document that callers should compute `previewDeposit/previewWithdraw` to set their slippage arguments.

Resolution

The development team updated the NatSpec in the `ERC4626InterfaceFacet` to inform readers that `previewDeposit()` and `previewWithdraw()` should be used to account for slippage.

The team has acknowledged the slippage risk and has decided not to implement the suggested slippage protection, providing the following response:

"We want to keep the logic wrapped around calling the vault simple for easier integration with other Term operations, so we won't add the slippage protection suggested."

TRF-20	Uninitialised ERC-165 Mapping Causes supportsInterface To Always Return false		
Assets	contracts/DiamondLoupeFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Low	Likelihood: High

Description

The `supportsInterface()` function implementation always returns `false` because the `supportedInterfaces` mapping in diamond storage is never initialised with any interface IDs. This breaks ERC-165 introspection functionality that external tools, wallets, and integrations rely upon to detect contract capabilities, potentially causing compatibility issues with systems that expect proper interface detection.

The current implementation queries an uninitialised mapping:

```
facets/DiamondLoupeFacet.sol::supportsInterface()

function supportsInterface(bytes4 _interfaceId) external override view returns (bool) {
    LibDiamond.DiamondStorage storage ds = LibDiamond.diamondStorage();
    return ds.supportedInterfaces[_interfaceId]; // @audit Always false - never populated
}
```

The `supportedInterfaces` mapping in `LibDiamond.DiamondStorage` is declared but never written to:

```
libraries/LibDiamond.sol

struct DiamondStorage {
    mapping(bytes4 => FacetAddressAndPosition) selectorToFacetAndPosition;
    mapping(address => FacetFunctionSelectors) facetFunctionSelectors;
    address[] facetAddresses;
    mapping(bytes4 => bool) supportedInterfaces; // @audit Never populated
    address contractOwner;
}
```

No initialisation logic or administrative functions exist to populate this mapping with the interfaces actually supported by the diamond's facets.

Recommendations

Initialise ERC-165 interface IDs during deployment via an initialisation facet executed through the `_init` parameter of `diamondCut()` and set the mapping to `true` for supported interfaces.

Resolution

The recommended fix has been applied and can be found in the following commit: [4269e3c](#).

TRF-21	AccessControl Overrides Diamond supportsInterface		
Assets	contracts/TermDiamond.sol contracts/DiamondLoupeFacet.sol contracts/LibDiamond.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Low	Likelihood: High

Description

The `supportsInterface(bytes4)` function exists in both the `TermDiamond` base contract (inherited from `AccessControl`) and `DiamondLoupeFacet`, creating a function selector collision that prevents proper ERC-165 introspection for facet interfaces.

Because the diamond contract directly inherits `AccessControl.supportsInterface`, calls resolve to the base implementation rather than being routed through the diamond's fallback mechanism to the loupe facet, causing the diamond to report only `AccessControl` interfaces whilst omitting facet interfaces expected by EIP-2535 tooling and integrations.

TermDiamond.sol

```
contract TermDiamond is AccessControl {
```

facets/DiamondLoupeFacet.sol::supportsInterface()

```
function supportsInterface(bytes4 _interfaceId) external override view returns (bool) {
    LibDiamond.DiamondStorage storage ds = LibDiamond.diamondStorage();
    return ds.supportedInterfaces[_interfaceId];
}
```

Recommendations

- Override `supportsInterface()` in `TermDiamond` to use diamond storage and include `AccessControl`.
- Initialise ERC-165 registrations via an initialisation facet executed through `_init` in `diamondCut()`.

Resolution

This issue was resolved as part of the `AccessControl` initialisation fix. The development team removed `AccessControl` inheritance from `TermDiamond` and moved it to `DiamondCutFacet`, eliminating the function selector collision entirely.

With `AccessControl` no longer directly inherited by the diamond contract, calls to `supportsInterface()` now properly route through the diamond's fallback mechanism to `DiamondLoupeFacet.supportsInterface()`, which correctly reports all registered facet interfaces from diamond storage.

This architectural change resolves the ERC-165 introspection issue while maintaining all access control functionality through the diamond's routing system.

The fix can be found in the following commit: [f89fc36](#).

TRF-22	Missing Slippage Protection In Aave Supply Operations		
Assets	contracts/TermAaveInterfaceFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

The `aaveSupply()` function lacks validation to ensure the user receives the expected amount of aTokens, potentially causing fund lockup when Aave accepts partial supplies.

facets/external/TermAaveInterfaceFacet.sol::aaveSupply()

```
// Approve Aave pool to spend the tokens
IERC20(asset).approve(aavePool, amount);

// Supply to Aave on behalf of the specified user
IPool(aavePool).supply(asset, amount, onBehalfOf, 0);
```

The function transfers the full amount of tokens from the user but does not validate that Aave actually supplied the complete amount. Unlike `aaveWithdrawOnBehalfOf()` and `aaveRepay()` which check returned amounts, `aaveSupply()` has no slippage protection.

When Aave's supply operation is partial (due to supply caps, market conditions, or other constraints), users pay the full amount but receive fewer aTokens than expected. The remaining tokens remain in the facet.

Recommendations

Implement slippage protection by comparing aToken balances before and after the supply operation:

facets/external/TermAaveInterfaceFacet.sol::aaveSupply()

```
// Get aToken balance before supply
uint256 aTokenBalanceBefore = IERC20(aTokenAddress).balanceOf(onBehalfOf);

// Execute supply
IPool(aavePool).supply(asset, amount, onBehalfOf, 0);

// Validate actual supply amount
uint256 aTokenBalanceAfter = IERC20(aTokenAddress).balanceOf(onBehalfOf);
uint256 actualSupplied = aTokenBalanceAfter - aTokenBalanceBefore;

if (actualSupplied != amount) revert SupplyAmountMismatch();
```

This ensures users receive exactly the aTokens they paid for or the transaction reverts.

Resolution

The development team has implemented the suggested fix in the commit [f9ff127](#).

TRF-23	User Tokens Can Be Locked In Facets		
Assets	contracts/TermMorphoInterfaceFacet.sol contracts/TermAaveInterfaceFacet.sol contracts/ERC4626InterfaceFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Several interface facets, `TermMorphoInterfaceFacet.sol`, `TermAaveInterfaceFacet.sol`, and `ERC4626InterfaceFacet.sol`, do not implement token recovery functionality, creating permanent fund lockup scenarios.

These facets can accumulate tokens through normal operations but provide no mechanism for retrieval.

- Intentional Token Retention:** Functions like `morphoWithdraw()`, `aaveWithdrawOnBehalfOf()`, `erc4626Withdraw()`, and `erc4626Redeem()` include `payoutToUser` parameters that allow tokens to remain in the facet rather than being transferred to users.
- Partial Operation Failures:** The `aaveSupply()` function may only partially succeed due to protocol constraints (supply caps, liquidity limits), leaving the difference between transferred and actually supplied tokens stranded in the facet.
- Accidental Transfers:** Direct token transfers to these facets have no recovery pathway.

Recommendations

Add a recovery function where an admin can retrieve funds locked in the facet:

Recommended Recovery Function

```
function recoverERC20(address token, uint256 amount, address to) external onlyAdmin {
    require(to != address(0), "Invalid recipient");
    IERC20(token).safeTransfer(to, amount);
    emit TokenRecovered(token, amount, to);
}
```

Resolution

The development team restricted the `erc20Transfer` function to admin-only access, eliminating the risk of unauthorized fund drainage while maintaining administrative recovery functionality for legitimate stuck fund scenarios.

The fix can be found in the commit [8332892](#).

TRF-24	Fund Theft Via Flash-Loan Borrower Context Leak And Public swap()		
Assets	contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol contracts/facets/base/TermMultiContextAuth.sol contracts/facets/external/TermMorphoInterfaceFacet.sol contracts/facets/external/TermAaveInterfaceFacet.sol contracts/facets/SwapRouterFacet.sol contracts/TermController.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

Three independent weaknesses compose into a direct theft path: the global `activeFlashLoanBorrower` context leaks the victim's authority to any reentrant caller during a flash-loan execution; the on-behalf Morpho and Aave borrow functions accept `payoutUser = false`, routing borrowed assets into the diamond while leaving the victim with the debt; and the public `swap()` has no access control, allowing any caller to redirect diamond-held tokens to an arbitrary receiver.

`flashExecute()` sets `activeFlashLoanBorrower` before any external interaction and holds it live for the duration of every planned action callback. `onlyUserOrActiveContext()` grants authority to any address while that flag is set, regardless of who the actual caller is. The on-behalf borrow and collateral-withdrawal paths are guarded only by this modifier, so a reentrant attacker inside a vault or Aave callback can invoke them as the victim:

TermFlashLoanExecutorFacet.sol

```
_setFlashLoanBorrower(msg.sender); // @audit victim authority is globally active for the entire callback sequence
```

TermMultiContextAuth.sol

```
modifier onlyUserOrActiveContext(address user) {
    require(
        msg.sender == user ||
        (tflc.activeFlashLoanBorrower != address(0) &&
         tflc.activeFlashLoanBorrower == user), // @audit any reentrant caller satisfies this check
        "Unauthorized caller"
    );
    _;
}
```

The exploit chain proceeds as follows. An attacker controls or influences a callback surface reachable from the victim's flash plan, for example, a user-approved ERC4626 vault. Inside that callback, while `activeFlashLoanBorrower == victim`, the attacker calls `morphoBorrow()` or `aaveBorrow()` with `payoutUser = false`. The modifier passes, and the borrow routes assets into the diamond instead of to the victim while the victim receives the debt:

TermMorphoInterfaceFacet.sol

```
function _morphoBorrowInternal(...) internal {
    // ... snip ...
    IMorpho(morphoPool_).borrow(
        marketParams_,
        assets_,
        0,
        user_,
        payoutUser_ ? user_ : address(this) // @audit debt assigned to victim; borrowed assets land in diamond
    );
}
```

After the callback returns, the attacker calls `SwapRouterFacet.swap()` directly. The function has no access control and decodes the receiver from attacker-supplied calldata, converting the diamond-held assets into output sent to the attacker:

SwapRouterFacet.sol

```
function swap(address tokenIn, uint256 amountIn, SwapRouterData calldata data) external {
    _swapInternal(tokenIn, amountIn, data); // @audit no access control; receiver decoded from attacker calldata
}
```

A fourth path arises from the shared approval registry. `TermController.approvedExternalAddresses` is checked identically by `approvedAavePoolOnly()` and `approvedERC4626VaultOnly()`, so any controller-approved address passes the Aave pool gate. A malicious contract passed as `aavePool` receives the diamond's token approval via `_aaveSupplyInternal()`, calls `transferFrom()` to drain the asset, and mints worthless `aTokens` to satisfy the supply delta check, no flash-loan context is required for this path.

This issue has a high impact because an attacker can manufacture victim debt, seize Morpho collateral, or drain approved token balances in a single transaction with no recovery path for the victim. This issue has a low likelihood because all paths except the fake Aave supply require the victim to be inside an active flash-loan execution, and the attacker must control or influence a reachable callback surface.

Recommendations

Replace the global flash-loan borrower context with a scoped execution model that restricts `onlyUserOrActiveContext()` to planned internal self-calls only. Restrict `swap()` to trusted execution contexts and require the receiver to be the diamond. Replace the shared `approvedExternalAddresses` mapping with per-integration registries to prevent ERC4626 approvals from satisfying the Aave pool gate.

Resolution

This issue has been fixed in [PR#1599](#). The modifier `onlyUserOrActiveContext()` now only allows self-calls as follows:

```
modifier onlyUserOrActiveContext(address user) {
    // ... snip ...
    (ts.activeSettlementMaker != address(0)) &&
    ts.activeSettlementMaker == user && msg.sender == address(this),
    // ... snip ...
}
```

TRF-25 Vault FulfillOrder Atomic Context Leaks To Malicious Vault Callback			
Assets	contracts/facets/ERC4626InterfaceFacet.sol contracts/facets/base/TermAtomicTxProtection.sol contracts/facets/base/TermMultiContextAuth.sol contracts/facets/TermLoanIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

A malicious ERC4626 vault can abuse `withdrawFromVaultFulfillOrder()` or `redeemFromVaultFulfillOrder()` to settle an arbitrary order on behalf of a victim. The bug is that both flows expose the victim's atomic batch context before making an external call to the vault.

Both entry points use `initiateAtomicTxProtection()`, which stores `msg.sender` as `atomicTxInitiator` before any external call and clears it only after the function returns:

base/TermAtomicTxProtection.sol

```

modifier initiateAtomicTxProtection() {
    TermStorage storage ts = LibTermStorage.termStorage();
    require(ts.atomicTxInitiator == UNSET_ATOMIC_TX_INITIATOR, "AtomicTx already initiated");
    ts.atomicTxInitiator = msg.sender;
    _;
    ts.atomicTxInitiator = UNSET_ATOMIC_TX_INITIATOR;
}

```

The external vault call happens while that context is live:

ERC4626InterfaceFacet.sol::withdrawFromVaultFulfillOrder()

```

function withdrawFromVaultFulfillOrder(
    address vault,
    uint256 assets,
    ITermIntent.OrderType orderType,
    bytes calldata orderData
) external initiateAtomicTxProtection approvedERC4626VaultOnly(vault, msg.sender) {
    _withdrawFromVault(vault, assets, msg.sender, address(this), false);
}

// Inside _withdrawFromVault():
uint256 shares = vaultContract.withdraw(assets_, receiver_, address(this));

```

`requireBatchContext()` in `TermMultiContextAuth` accepts any caller while `atomicTxInitiator` is set to the target user:

base/TermMultiContextAuth.sol

```

modifier requireBatchContext(address user) {
    TermStorage storage ts = LibTermStorage.termStorage();
    TermFlashLoanContext storage tflc = LibTermStorage.termFlashLoanContext();
    require(
        (ts.multicallInitiator != address(0) &&
         ts.multicallInitiator == user) ||
        (ts.atomicTxInitiator != address(0) &&
         ts.atomicTxInitiator == user) || // @audit passes while vault callback is active
        (tflc.activeFlashLoanBorrower != address(0) &&
         tflc.activeFlashLoanBorrower == user),
        "Batch context required"
    );
    _;
}

```

The batch-only settlement overloads rely on `requireBatchContext(taker)`:

TermLoanIntentFacet.sol::settleLimitLend()

```

function settleLimitLend(
    LimitLendOrder memory order,
    address taker,
    uint256 fillAmount,
    uint256[] memory collateralAmounts,
    Signature memory signature
) external requireBatchContext(taker) nonReentrant {
    _settleLimitLendInternal(order, taker, fillAmount, collateralAmounts, signature, true, false);
}

```

`nonReentrant()` on `settleLimitLend()` and `settleLimitBorrow()` does not stop this. The outer fulfill-order path does not hold the reentrancy lock, so the vault's callback is the first lock acquisition.

An attacker can exploit this as follows:

1. A victim calls `withdrawFromVaultFulfillOrder(maliciousVault, assets, LEND, orderData)` against a vault they have previously approved.
2. `initiateAtomicTxProtection()` sets `atomicTxInitiator = victim`.
3. `_withdrawFromVault()` calls `maliciousVault.withdraw()`, transferring execution to attacker-controlled code while `atomicTxInitiator == victim`.
4. The malicious vault reenters the Diamond and calls `settleLimitLend(attackerOrder, victim, fillAmount, collateralAmounts, attackerSig)` with `taker = victim`.
5. `requireBatchContext(victim)` passes because `atomicTxInitiator == victim`. The `nonReentrant()` check passes because no lock is held.
6. The victim is forced into the attacker's chosen settlement before their own flow resumes.

The precondition is that the vault is victim-approved via `userApproveVault()` or controller-approved. The impact is high because the victim can be pushed into arbitrary borrowing or lending state in a single transaction. The likelihood is low because the attack still depends on a malicious approved vault and the victim entering one of the affected fulfill-order flows.

This root cause is distinct from the flash-loan borrower context leak described separately in [TRF-24](#). That finding exploits `activeFlashLoanBorrower` set by `flashExecute()` via `onlyUserOrActiveContext()`. This finding exploits `atomicTxInitiator` set by `initiateAtomicTxProtection()` via `requireBatchContext()`, through a different entry point with different preconditions.

Recommendations

Do not expose `atomicTxInitiator` across the external vault call. Execute the vault withdrawal or redemption first, then set the atomic context only around the internal settlement call and clear it immediately after.

Resolution

The development team has resolved this issue in [PR#1598](#).

The vulnerable ERC4626 fulfill-order helpers were removed, and batch settlement authorization now requires a Diamond self-call with `activeAtomicTxSettlementTaker` set for the relevant user rather than trusting the broader atomic transaction initiator.

TRF-26 Vault FulfillOrder Strands Purchase Tokens Extractable Via Batch Settlement			
Assets	contracts/facets/ERC4626InterfaceFacet.sol contracts/facets/TermLoanIntentFacet.sol contracts/facets/MulticallFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The `withdrawFromVaultFulfillOrder()` and `redeemFromVaultFulfillOrder()` functions contain a two-stage accounting bug - they first move the caller's full vault withdrawal into the diamond, then settle against only the order's remaining capacity without refunding the surplus. A later batch borrow settlement using the same purchase token can then treat that leftover diamond balance as valid routed lender funds, allowing an attacker to obtain a lender-side repo-token position without supplying the corresponding purchase tokens.

Both `withdrawFromVaultFulfillOrder()` and `redeemFromVaultFulfillOrder()` pass the full withdrawn amount into `settleLimitBorrow()`:

ERC4626InterfaceFacet.sol::redeemFromVaultFulfillOrder()

```
_withdrawFromVault(vault, assets, msg.sender, address(this), false);
// ... snip ...
TermLoanIntentFacet(address(this)).settleLimitBorrow(
    params.order,
    msg.sender,
    assets, // @audit full vault withdrawal passed into settlement
    params.signature
);
```

Settlement later caps that fill to the order's remaining capacity:

TermLoanIntentFacet.sol::_calculateAndValidateFillAmount()

```
function _calculateAndValidateFillAmount(
    uint256 fillAmount,
    uint256 totalAmount,
    uint256 filledAmount
) private pure returns (uint256) {
    uint256 maxFill = totalAmount - filledAmount;
    uint256 actualFillAmount = Math.min(fillAmount, maxFill); // @audit settlement only consumes remaining order capacity
    if (actualFillAmount == 0) { revert NothingToFill(); }
    return actualFillAmount;
}
```

If `assets > maxFill`, only `actualFillAmount` is used in settlement. The remainder stays in the diamond and is never refunded to the vault user.

Those stranded tokens can be extracted later because the batch `settleLimitBorrow()` path hardcodes `isRoutedPurchaseToken = true`:

TermLoanIntentFacet.sol::settleLimitBorrow()

```
function settleLimitBorrow(
    LimitBorrowOrder memory order,
    address taker,
    uint256 fillAmount,
    Signature memory signature
) external requireBatchContext(taker) nonReentrant {
    _settleLimitBorrowInternal(
        order,
        taker,
        fillAmount,
        signature,
        true, // @audit hardcodes isRoutedPurchaseToken = true for batch settlement
        false
    );
}
```

When `isRoutedPurchaseToken` is true, settlement pays directly from the diamond's balance:

TermLoanIntentFacet.sol::_executeTokenTransfer()

```
function _executeTokenTransfer(
    address token, bool isRouted, address from, address to,
    address feeRecipient, uint256 amount, uint256 fee, bool useTransfer2
) private {
    if (isRouted) {
        IERC20(token).safeTransfer(to, amount - fee); // @audit spends existing diamond balance instead of pulling from
        // ↳ current lender
        if (fee > 0) {
            IERC20(token).safeTransfer(feeRecipient, fee); // @audit fees also come out of the diamond's existing balance
        }
    }
    // ... snip ...
}
```

The later extraction works because the batch `settleLimitBorrow()` path hardcodes `isRoutedPurchaseToken = true`, so settlement treats any matching purchase-token balance already sitting in the diamond as valid routed lender funds without binding that balance to the current lender, current order, or original funding source.

An attacker can exploit this as follows:

1. The attacker creates a borrow order for 500 purchase tokens.
2. Using a separate address as lender, the attacker front-runs and fills 400 of that order. The attacker-controlled borrower receives 400 purchase tokens and the attacker-controlled lender receives the repo-token position for that 400 fill.
3. That leaves only 100 purchase tokens of remaining capacity on the order.
4. A victim lender then calls `withdrawFromVaultFulfillOrder(vault, 500, BORROW, orderData)` against the same order.
5. The protocol first withdraws the victim lender's full 500 purchase tokens from the vault into the diamond.
6. Settlement can now use only the remaining 100 capacity, because the attacker's earlier 400 fill already consumed the rest of the order.
7. That remaining 100 is used to fill the order. The attacker-controlled borrower receives 100 purchase tokens, and the victim lender receives the repo-token position for that 100 fill.
8. The victim lender's extra 400 purchase tokens are not refunded and remain stranded in the diamond.
9. An innocent borrower later creates a different borrow order denominated in the same purchase token for at least 400, with an open lender slot (`order.taker == address(0)`) or the attacker explicitly authorized as lender/taker.
10. The attacker acts as lender/taker on that second order through the batch `settleLimitBorrow()` path via `multicall()`.
11. Because `isRoutedPurchaseToken = true`, that later settlement pays the innocent borrower from the diamond's stranded 400 instead of pulling fresh purchase tokens from the attacker.
12. The attacker still receives the corresponding lender-side repo-token position for that later fill without contributing new purchase tokens.

The same issue exists in `redeemFromVaultFulfillOrder()`. The partial-fill condition can arise naturally or be MEV-induced by reducing the order's remaining capacity before the victim's transaction lands.

The attacker can later monetize that repo-token position by redeeming it for purchase tokens after the redemption period opens, or by selling it earlier via the repo-token swap market.

This issue has a high impact because a victim's stranded purchase tokens can be converted into an attacker-controlled lender-side repo-token position, creating direct economic loss even though the attacker does not directly withdraw the stranded balance. This issue has a low likelihood because monetization requires a two-stage sequence: the attacker must first induce a partial-fill mismatch by reducing an order's remaining capacity before a victim's vault-fulfill transaction executes, and then a later compatible borrow order using the same purchase token must exist with an open or attacker-authorized lender slot so the attacker can reuse the stranded balance through batch settlement.

Recommendations

Refund any surplus purchase tokens to the caller when settlement fills less than the vault withdrawal. The fulfill-order path should not leave unused purchase tokens in the diamond.

Resolution

The development team has resolved this issue in [PR#1598](#).

The ERC4626 fulfill-order helpers were removed in favor of settlement-with-hook entry points that check remaining capacity before retrieving funds, and direct settlement now reverts on overfills. Batch settlement authorization was also tightened so `multicall()` cannot reuse pre-existing Diamond balances as routed lender funds.

TRF-27 Collateral Dust From Liquidations Blocks Pro-Rata Redemption			
Assets	contracts/TermRepoCollateralManager.sol contracts/TermRepoServicer.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`batchDefault()` and `batchDefaultWithRepoToken()` round down the amount of collateral they try to seize. If a borrower is left with a tiny leftover amount after one default, the next default can fail instead of clearing the rest of the position.

TermRepoCollateralManager.sol::_transferLiquidationCollateral()

```
// @audit rounding down drops any leftover fraction
return (
    collateralAmountWithDiscount.mantissa /
    10 ** (18 - tokenInstance.decimals()),
    protocolSeizureAmount.mantissa /
    10 ** (18 - tokenInstance.decimals())
);
```

Once the leftover collateral is smaller than the next seizure amount, `_transferLiquidationCollateral()` reverts instead of taking the rest:

TermRepoCollateralManager.sol::_transferLiquidationCollateral()

```
if (
    collateralSeizureAmount >
    lockedCollateralLedge[borrower][collateralAddress] // @audit reverts instead of seizing remainder; dust collateral
    ↪ cannot be cleared
) {
    revert InsufficientCollateralForLiquidationRepayment(
        collateralAddress
    );
}
```

That leaves some debt unpaid and some collateral still locked. The whole term then stays stuck because `redeemTermRepoTokens()` only allows pro-rata redemption once all locked collateral has been cleared:

TermRepoServicer.sol::redeemTermRepoTokens()

```
if (
    termRepoToken.totalRedemptionValue() <=
    totalRepurchaseCollected + 10 ** 4
) {
    _parRedemption(redeemer, amountToRedeem);
} else if (!termRepoCollateralManager.encumberedCollateralRemaining()) {
    _proRataRedemption(redeemer, amountToRedeem);
} else {
    revert EncumberedCollateralRemaining();
}
```

Because the liquidation path cannot clear the leftover amount, `encumberedCollateralRemaining()` stays `true` and repo token holders in that term cannot redeem on the fair split path.

This issue has a medium impact because one borrower can block everyone else in the term from redeeming fairly. It has a medium likelihood because it only needs normal public liquidation calls, but it does require a large price move and a partial default that leaves a small remainder behind.

Recommendations

Make `batchDefault()` and `batchDefaultWithRepoToken()` take the full remaining collateral instead of reverting when the requested seizure is larger than what is left. Alternatively, add a cleanup path that clears the borrower once the remaining collateral is too small to seize normally.

Resolution

The development team has acknowledged the finding with the following comment:

"We assume liquidators will act up to this boundary; if not, our own liquidator can ensure it is reached."

TRF-28 Factory-Deployed Repos Permanently Locked Out Of Auction And Rollover-In Paths			
Assets	contracts/factory/TermRepoDeployerFactory.sol contracts/TermRepoServicer.sol contracts/TermRepoCollateralManager.sol contracts/TermInitializer.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Every repo deployed through `TermRepoDeployerFactory` is locked out of V1 auction entry and auction-based rollover. The factory keeps sole `INITIALIZER_ROLE` on each component, but it does not expose any way to call `reopenToNewAuction()`, and the role cannot be delegated because `DEFAULT_ADMIN_ROLE` is never set.

`TermRepoDeployerFactory` deploys each component with `address(this)` as the `INITIALIZER_ROLE` holder, then calls `pairTermContracts()` once, passing `address(0)` for all auction and offer-locker arguments and consuming the one-shot `notTermContractPaired` modifier:

TermRepoDeployerFactory.sol::_pairAllContracts()

```
// NOTE: auction/offerLocker/termDiamond addresses must be set later when an auction is deployed
TermRepoServicer(deployed.termRepoServicer).pairTermContracts(
    deployed.termRepoLocker,
    deployed.termRepoCollateralManager,
    deployed.termRepoToken,
    termDiamond,
    address(0), // @audit termAuctionOfferLocker permanently address(0)
    address(0), // @audit termAuction permanently address(0)
    deployed.rolloverManager,
    // ... snip ...
);
```

`pairTermContracts()` grants `AUCTION_LOCKER` and `AUCTIONEER` to these zero addresses, which can never be exercised:

TermRepoServicer.sol::pairTermContracts()

```
function pairTermContracts(
    // ... snip ...
    address termAuctionOfferLocker_,
    address termAuction_,
    // ... snip ...
) external onlyRole(INITIALIZER_ROLE) notTermContractPaired {
    // ... snip ...
    _grantRole(AUCTION_LOCKER, termAuctionOfferLocker_); // @audit grants to address(0)
    _grantRole(AUCTIONEER, termAuction_);                 // @audit grants to address(0)
    // ... snip ...
}
```

`notTermContractPaired` sets `termContractPaired = true` and reverts on any subsequent call. The only other function that can grant these roles is `reopenToNewAuction()`:

TermRepoServicer.sol::reopenToNewAuction()

```
function reopenToNewAuction(
    TermAuctionGroup calldata termAuctionGroup
) external onlyRole(INITIALIZER_ROLE) { // @audit only the factory can call this
    _grantRole(AUCTION_LOCKER, address(termAuctionGroup.termAuctionOfferLocker));
    _grantRole(AUCTIONEER, address(termAuctionGroup.auction));
    // ... snip ...
}
```

The factory exposes no function to invoke `reopenToNewAuction()` on a previously deployed repo. The remaining paths are also blocked:

- `TermInitializer.setupAuction()`, the only other callsite, gates entry with `controller.isTermDeployed()`, which returns `false` for factory repos (registered via `markTermFactoryDeployed()`). Even if an admin called `markTermDeployed()` to bridge this gap, `TermInitializer` lacks `INITIALIZER_ROLE` on factory-deployed components.
- `INITIALIZER_ROLE` cannot be delegated: neither `TermRepoServicer` nor `TermRepoCollateralManager` sets `DEFAULT_ADMIN_ROLE` during `initialize()`. In `OpenZeppelin AccessControl`, `grantRole(INITIALIZER_ROLE, x)` requires `getRoleAdmin(INITIALIZER_ROLE)`, which defaults to `DEFAULT_ADMIN_ROLE`. Since nobody holds it, the admin chain is permanently broken.

The result is that factory repos cannot use `AUCTION_LOCKER`-gated flows, `AUCTIONEER`-gated flows, or `openExposureOnRolloverNew()`. The intent and strategy path via the diamond still works, so the impact is bounded to medium. The likelihood is medium because this appears as soon as auction wiring or auction-based rollover is attempted on a factory repo.

Recommendations

Add an admin-controlled path for factory repos to call `reopenToNewAuction()` after deployment.

Resolution

This issue has been fixed in [PR#1634](#) by adding the function `deployAuctionAndReopenTerm()` that deploys and pairs auction contracts.

TRF-29	Failed Or Partially Filled Rollover Auction Permanently Revokes Re-Election Right		
Assets	contracts/TermRepoRolloverManager.sol contracts/TermAuction.sol contracts/TermRepoServicer.sol contracts/lib/TermRepoRolloverElection.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

A borrower whose rollover bid is not filled, or only partially filled, has the `processed` flag set to `true` regardless of outcome, permanently blocking re-election and stranding residual debt with no rollover escape, potentially forcing liquidation or default.

`TermRepoRolloverManager` tracks a per-borrower `processed` flag. The struct documentation states it should be `false` on failure:

lib/TermRepoRolloverElection.sol

```
/// @dev A boolean testing whether rollover has been successfully processed:
///      false if bid fails to lock or is not successful in rollover auction
bool processed; // @audit - comment says false on failure; implementation sets true unconditionally
```

Despite this, `fulfillRollover()` sets `processed = true` unconditionally:

TermRepoRolloverManager.sol::fulfillRollover()

```
function fulfillRollover(
    address borrower
) external onlyRole(ROLLOVER_BID_FULFILLER_ROLE) {
    rolloverElections[borrower].processed = true; // @audit - no check on fill outcome
    emitter.emitRolloverProcessed(termRepoId, borrower);
}
```

`_markRolloverAsProcessed()` in `TermAuction` invokes this across three failure paths:

Path 1 – Failed auction (no clearing price). All rollover bids are marked processed with zero debt settled:

TermAuction.sol

```
} else {
    // Return sorted bid funds.
    for (uint256 i = 0; i < sortedBids.length; ++i) {
        if (sortedBids[i].isRollover) {
            _markRolloverAsProcessed( // @audit - marks processed; zero debt settled
                sortedBids[i].rolloverPairOffTermRepoServicer,
                sortedBids[i].bidder
            );
        } else {
            termAuctionBidLocker.auctionUnlockBid(
                sortedBids[i].id,
                sortedBids[i].bidder,
                sortedBids[i].collateralTokens,
                sortedBids[i].collateralAmounts
            );
        }
    }
}
```

Path 2 – Bid price below clearing price. The "No assignment" branch immediately marks rollover bids processed:

TermAuction.sol

```

} else {
    // No assignment.
    if (sortedBids[i].isRollover) {
        _markRolloverAsProcessed( // @audit - marks processed; bid never filled
            sortedBids[i].rolloverPairOffTermRepoServicer,
            sortedBids[i].bidder
        );
    } else {
        termAuctionBidLocker.auctionUnlockBid(
            sortedBids[i].id,
            sortedBids[i].bidder,
            sortedBids[i].collateralTokens,
            sortedBids[i].collateralAmounts
        );
    }
}
}

```

Path 3 — Partial fill. `_partiallyAssignBid()` settles only the covered fraction via `closeExposureOnRolloverExisting()`, then unconditionally calls `fulfillRollover()`:

TermRepoServicer.sol

```
termRepoRolloverManager.fulfillRollover(borrower); // @audit - called after partial close; residual debt remains
```

In all three paths the borrower retains outstanding debt in `repurchaseExposureLedger[borrower]` while `processed` is `true`. When the borrower attempts to re-elect rollover, `_electRolloverInternal()` reverts:

TermRepoRolloverManager.sol

```

if (rolloverElections[borrower].processed) {
    revert RolloverProcessedToTerm(); // @audit - blocks re-election; no admin reset path
}

```

`cancelRollover()` contains the same guard and no privileged reset function exists, so the borrower must repay in cash or face liquidation.

This issue has a medium impact because borrowers with residual debt are forced into cash repayment or face liquidation risk. This issue has a medium likelihood because thin order books, high clearing rates, or auction cancellations are plausible triggering conditions.

Recommendations

Conditionalise the `_markRolloverAsProcessed()` call so it executes only when a rollover bid is fully assigned via the `_assignRolloverBid()` path. For no-assignment, failed-auction, unrevealed-bid, and cancelled-auction paths, leave `processed` unchanged so the borrower retains the ability to re-elect before the repurchase window closes. The partial-fill path also requires attention: `closeExposureOnRolloverExisting()` in `TermRepoServicer` unconditionally calls `fulfillRollover()` after settling the covered fraction, so the same conditional treatment must apply there.

Resolution

The development team has acknowledged the finding with the following comment:

“To keep the system simple, we limit rollovers to a single attempt. In v2, if a rollover attempt fails, users can instead refinance via Aave, Morpho, or TermRouter.”

TRF-30	Flash Loan Hooks Over-Pull On Partial Fill, Draining Taker Balance		
Assets	contracts/facets/TermLoanIntentHookFacet.sol contracts/facets/TermRepoTokenIntentFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`settleLimitLendHook()` and `swapRepoTokenHook()` pull the taker's payment from the requested fill amount rather than the amount actually settled. If the underlying order was partially filled after the flash plan was built, the taker pays for the full request while receiving only the remaining capacity.

`settleLimitLendHook()` computes `totalBorrowAmount` from the caller-supplied `fillAmounts` before settlement:

TermLoanIntentHookFacet.sol

```
(
    bool usePermit2,
    LimitLendOrder[] memory orders,
    Signature[] memory signatures,
    uint256[] memory fillAmounts
) = abi.decode(
    input.additionalCalldata,
    (bool, LimitLendOrder[], Signature[], uint256[])
);

uint256 totalBorrowAmount;
for (uint256 i = 0; i < len; ++i) {
    totalBorrowAmount += _calculateBorrowAmountFromFillAmount(
        fillAmounts[i],
        orders[i].borrowFee,
        maturityTimestamp
    );
}
```

But each `settleLimitLend()` can clamp the fill to the remaining order capacity:

TermLoanIntentFacet.sol

```
function _calculateAndValidateFillAmount(
    uint256 fillAmount, uint256 totalAmount, uint256 filledAmount
) private pure returns (uint256) {
    uint256 maxFill = totalAmount - filledAmount;
    uint256 actualFillAmount = Math.min(fillAmount, maxFill);
    if (actualFillAmount == 0) { revert NothingToFill(); }
    return actualFillAmount;
}
```

After settlement, the hook still pulls the full pre-computed `totalBorrowAmount`:

TermLoanIntentHookFacet.sol

```
if (usePermit2) {
    Permit2Lib.PERMIT2.transferFrom(
        taker, address(this), totalBorrowAmount.toUint160(), purchaseToken
    );
} else {
    IERC20(purchaseToken).safeTransferFrom(taker, address(this), totalBorrowAmount);
}
```

If the order was partially filled, the taker receives less than `totalBorrowAmount` but still pays the full amount. The excess stays in the diamond. The executor does not detect this because it only measures the diamond's balances:

TermFlashLoanExecutorFacet.sol

```
function _executeCallAndGetTokenBalances(bytes memory encodedCalldata, address tokenIn, address tokenOut)
    private returns (uint256, uint256)
{
    uint256 tokenOutBalanceBefore = IERC20(tokenOut).balanceOf(address(this));
    Address.functionCall(address(this), encodedCalldata);
    uint256 tokenOutBalanceAfter = IERC20(tokenOut).balanceOf(address(this));
}
```

The same bug exists in `swapRepoTokenHook()`:

`_swapRepoTokenInternal()` can clamp the fill, but the hook still pulls the full `takerSwapOutputAmount` from the taker.

This issue has a medium impact because the loss equals the difference between the requested and actual fill, bounded by the partial fill size. This issue has a medium likelihood because partial fills are a normal protocol condition and the MEV-induction path is viable on any chain with a public mempool.

Recommendations

After settlement, pull only the amount actually delivered. The hook should derive its transfer amount from the realised fill, not the requested fill.

Resolution

The development team has resolved this issue in [PR#1624](#).

The hooks now validate remaining order capacity before settlement and revert if the requested fill exceeds the available amount, preventing silent over-pull on partial fills.

TRF-31 Non-Auction Origination Uses Maintenance Ratios Instead Of Initial Ratios			
Assets	contracts/TermRepoCollateralManager.sol contracts/TermRepoServicer.sol contracts/facets/TermRouterFacet.sol contracts/facets/TermStrategyFacet.sol contracts/facets/TermLoanIntentFacet.sol contracts/TermAuctionBidLocker.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

A borrower can open fresh non-auction positions using only the maintenance collateral ratio rather than the higher initial ratio. This weakens origination standards across router, strategy, and intent-based minting paths.

`calculateMintableExposure()` now divides by `maintenanceCollateralRatios[collateralToken]`:

TermRepoCollateralManager.sol::calculateMintableExposure()

```
function calculateMintableExposure(
    address collateralToken,
    uint256 amountToLock
) external view returns (Exp memory) {
    return
        div_(
            div_(
                termPriceOracle.usdValueOfTokens(
                    collateralToken,
                    amountToLock
                ),
                Exp({mantissa: maintenanceCollateralRatios[collateralToken]})
            ),
            termPriceOracle.usdValueOfTokens(purchaseToken, 1)
        );
}
```

`TermRepoServicer._handleCollateral()` uses this value to compute `maxMintValue` for both normal and intent-based minting:

TermRepoServicer.sol::_handleCollateral()

```

function _handleCollateral(
    address borrower,
    address sender,
    uint256[] calldata collateralAmounts
) private returns (uint256 maxMintValue) {
    for (uint256 i = 0; i < collateralAmounts.length; ++i) {
        termRepoCollateralManager.mintOpenExposureLockCollateral(
            borrower,
            sender,
            termRepoCollateralManager.collateralTokens(i),
            collateralAmounts[i]
        );
        uint256 maxTokensFromCollateral = truncate(
            div_(
                termRepoCollateralManager.calculateMintableExposure(
                    termRepoCollateralManager.collateralTokens(i),
                    collateralAmounts[i]
                ),
                Exp({mantissa: termRepoToken.redemptionValue()})
            )
        );
        maxMintValue += maxTokensFromCollateral;
    }
}

```

Both minting entrypoints rely on this `maxMintValue` check:

TermRepoServicer.sol::_mintOpenExposureInternal()

```

function _mintOpenExposureInternal(
    address borrower,
    address sender,
    uint256 amount,
    uint256[] calldata collateralAmounts
) internal {
    uint256 maxMintValue = _handleCollateral(borrower, sender, collateralAmounts);
    if (amount > maxMintValue) {
        revert InsufficientCollateral(); // @audit - rejects amount > maintenance-ratio cap, not initial-ratio cap
    }
    // ... snip ...
}

```

TermRepoServicer.sol::mintOpenExposureFromIntent()

```

function mintOpenExposureFromIntent(
    address borrower,
    address lender,
    uint256 purchaseTokenAmount,
    uint256[] calldata collateralAmounts,
    uint256 offerRate,
    bool isRoutedCollateral
) external onlyRole(DIAMOND_ROLE) returns (uint256) {
    // ... snip ...
    uint256 maxMintValue;
    if (isRoutedCollateral) {
        maxMintValue = _handleCollateral(borrower, msg.sender, collateralAmounts); // @audit - maxMintValue computed against
        // maintenance ratio
    } else {
        maxMintValue = _handleCollateral(borrower, borrower, collateralAmounts); // @audit - maxMintValue computed against
        // maintenance ratio
    }
    if (amountOfRepoTokenToMint > maxMintValue) {
        revert InsufficientCollateral();
    }
    // ... snip ...
}

```

The same servicer path is used by router minting, strategy minting, and intent-based minting. The auction path still enforces the higher initial ratio:

TermAuctionBidLocker.sol::_isInInitialCollateralShortFall()

```
function _isInInitialCollateralShortFall(
    uint256 bidAmount,
    address[] memory collateralTokens_,
    uint256[] memory collateralAmounts
) internal view returns (bool) {
    uint256 initialCollateralRatio = termRepoCollateralManager
        .initialCollateralRatios(collateralToken);
}
```

A borrower can therefore choose collateral in the band between the maintenance and initial thresholds: too weak for auction origination, but still sufficient for router, strategy, or intent-based minting. That leaves newly opened positions with less buffer against shortfall and liquidation.

This issue has a medium impact because it weakens a core risk control across multiple minting flows and can increase credit losses during volatility. This issue has a medium likelihood because the path is directly reachable onchain, but exploitation depends on a collateral configuration that falls between the two thresholds.

Recommendations

Use `initialCollateralRatios[collateralToken]` for fresh minting checks.

Resolution

The development team has acknowledged the finding with the following comment:

"The time between submitting an auction bid and clearing it can range from hours to days. Given this delay, initial ratios are set conservatively. For atomic minting, however, such a large buffer is unnecessary, and imposing auction-level constraints on an atomic transaction is overly restrictive. Our UI will provide sufficient guidance for prospective borrowers."

TRF-32	Non-Standard ERC20 Tokens Are Not Supported		
Assets	/*		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Users cannot use non-standard ERC20 tokens through the term finance protocol.

A token which does not strictly follow the ERC20 token interface might not be supported by several facets. Specifically, this is true when the token's `approve()` function does not adhere to the ERC20 interface. One prominent example for such tokens is the stablecoin `USDT`.

For example in the contract `ERC4626InterfaceFacet`, when `ERC4626InterfaceFacet._depositToVault()` is called with such a token, the transaction reverts at the following line:

```
ERC4626InterfaceFacet.sol::_depositToVault()
```

```
asset.approve(vault_, assets_);
```

Since this contract expects `approve()` to return a `bool`, calling it on a token like `USDT`, which returns nothing, results in an execution error and causes the transaction to revert.

In the case of `USDT`, the function does not return any value, which causes an execution error as a `bool` is expected.

The impact of this issue is medium because it prevents users from interacting with the protocol using widely adopted, non-standard ERC20 tokens such as `USDT`. While the protocol remains functional for fully compliant ERC20 tokens, the inability to support popular assets significantly reduces its practical usability and limits user participation. This can lead to reduced liquidity and adoption, especially given that tokens like `USDT` are commonly used in DeFi.

The likelihood is also medium because non-standard ERC20 implementations are well-known and prevalent in the ecosystem, yet not all users will attempt to use them. However, given the popularity of such tokens, it is reasonably likely that users will encounter this issue in practice, particularly when interacting with vaults or deposit flows that rely on strict ERC20 compliance.

Recommendations

Consider using `SafeERC20` for all ERC20 interactions, and replace direct calls to `approve()` with `SafeERC20.forceApprove()`.

Resolution

This issue has been fixed in [PR#1623](#).

All `approve()` calls have been replaced by `SafeERC20.forceApprove()`.

TRF-33 Partial Liquidation Leaves Locked Rollover Amount Above Live Debt			
Assets	contracts/TermRepoServicer.sol contracts/TermRepoRolloverManager.sol contracts/TermAuctionBidLocker.sol contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

After a borrower elects rollover and the rollover auction passes reveal, a partial liquidation can reduce the borrower's live debt without reducing the stored rollover reservation. The protocol then treats more debt as reserved for rollover than actually exists.

`TermRepoRolloverManager electRollover()` stores a `rolloverAmount` in the borrower's election. `TermRepoServicer._getMaxRepaymentAroundRollover()` uses that stored amount to reserve part of the borrower's debt from manual repayment. Debt-reducing liquidation paths mutate `repurchaseExposureLedger[borrower]` without synchronising `rolloverElections[borrower].rolloverAmount`.

`batchLiquidation()` reduces live debt through `liquidatorCoverExposure()` and `_repay()`:

TermRepoCollateralManager.sol

```
termRepoServicer.liquidatorCoverExposure( // @audit - does not update rolloverElections[borrower].rolloverAmount
  borrower,
  msg.sender,
  closureAmounts[i]
);
```

TermRepoServicer.sol::liquidatorCoverExposure()

```
function liquidatorCoverExposure(
  address borrower,
  address liquidator,
  uint256 amountToCover
) external onlyRole(COLLATERAL_MANAGER) {
  _repay(borrower, liquidator, amountToCover); // @audit - reduces live debt but leaves rolloverElections stale
}
```

Neither function updates the borrower's outstanding rollover election. The same unsynchronised debt reduction exists in `liquidatorCoverExposureWithRepoToken()` and default flows.

`_getMaxRepaymentAroundRollover()` then performs an uncapped subtraction against the stale stored amount:

TermRepoServicer.sol::_getMaxRepaymentAroundRollover()

```
if (
  rolloverElection.rolloverAmount == 0 || rolloverElection.processed
) {
  outstandingRolloverAmount = 0;
} else {
  outstandingRolloverAmount = rolloverElection.rolloverAmount;
}

return repurchaseExposureLedger[borrower] - outstandingRolloverAmount; // @audit - panics when live debt < stale
↳ rolloverAmount after partial liquidation
```

When `repurchaseExposureLedger[borrower] < outstandingRolloverAmount`, this panics with Solidity's checked-arithmetic error (0x11). `submitRepurchasePayment()` calls this helper before any repayment guard:

TermRepoServicer.sol::submitRepurchasePayment()

```
uint256 maxRepurchaseAmount = _getMaxRepaymentAroundRollover(borrower); // @audit - panics here if borrower's debt < stale
    ↳ rolloverAmount
if (amount > maxRepurchaseAmount) {
    revert RepurchaseAmountTooHigh();
}
```

Even `submitRepurchasePayment(1)` panics. `burnCollapseExposure()` is blocked by the same helper, so both manual repayment paths can be denied.

Cancellation via `cancelRollover()` rewrites the rollover bid through `_processRollover()`, which must pass the bid locker's auction-phase gate:

TermRepoRolloverManager.sol::cancelRollover()

```
rolloverElections[borrower].rolloverAmount = 0;

_processRollover(borrower);
```

TermAuctionBidLocker.sol

```
if (
    // solhint-disable-next-line not-rely-on-time
    block.timestamp > revealTime || block.timestamp < auctionStartTime
) {
    revert AuctionNotOpen(); // @audit - blocks cancelRollover() once past revealTime, trapping the stale election
}
```

Once past `revealTime`, `cancelRollover()` is blocked regardless of bid amount. The `DIAMOND_ROLE` overload follows the same `_cancelRolloverInternal()` path and fails identically. No separate reset path clears the stale `rolloverAmount`.

The deadlock is:

1. A borrower elects rollover for their full debt.
2. The rollover auction passes `revealTime`, so the bid is phase-locked.
3. A liquidator partially reduces the borrower's debt.
4. `submitRepurchasePayment()` and `burnCollapseExposure()` revert because the repayment bound underflows against the stale reservation.
5. `cancelRollover()` reverts with `AuctionNotOpen()`.

This issue has a medium impact because the borrower is denied both manual repayment paths while carrying genuine residual debt, leaving the position exposed to further liquidation or default. This issue has a medium likelihood because it only requires an ordinary partial liquidation after rollover election and after `revealTime`.

Recommendations

Keep `rolloverElections[borrower].rolloverAmount` in sync with live debt on every debt-reducing path.

Resolution

The development team has addressed the arithmetic underflow in [PR#1635](#) and acknowledged the deadlock with the following comment:

"Once a borrower elects rollover, the corresponding amount is treated as fully reserved for the auction, and repayments that would reduce or interfere with that reservation are disallowed. The stale accounting intro-

duced by post-election liquidations is safely and deterministically resolved at auction clearing, where the rollover amount is reconciled to the borrower's actual outstanding debt. Auction clearing is permissionless and can be executed by any actor immediately after the auction concludes—and has historically occurred within minutes."

TRF-34	Repo-Token Liquidations Enable Preferential Recovery Against Selected Borrowers		
Assets	contracts/TermRepoToken.sol contracts/TermRepoServicer.sol contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

TermRepoToken is meant to be a fungible claim on the whole term. But `batchLiquidationWithRepoToken()` and `batchDefaultWithRepoToken()` let a holder burn repo tokens against a chosen borrower and receive the same liquidation premium as a cash liquidator. That turns a pool claim into borrower-specific recovery.

TermRepoServicer.sol::liquidatorCoverExposureWithRepoToken()

```
function liquidatorCoverExposureWithRepoToken(
    address borrower,
    address liquidator,
    uint256 amountOfRepoToken
) external onlyRole(COLLATERAL_MANAGER) returns (uint256) {
    uint256 burnValue = termRepoToken.burnAndReturnValue(
        liquidator,
        amountOfRepoToken
    );
    if (burnValue > repurchaseExposureLedger[borrower]) {
        revert RepurchaseAmountTooHigh();
    }
    repurchaseExposureLedger[borrower] -= burnValue; // @audit - reduces borrower exposure using pooled claim value, not
    ↪ fresh external capital
    totalOutstandingRepurchaseExposure -= burnValue;
    return burnValue;
}
```

`burnAndReturnValue()` always burns at fixed par:

TermRepoToken.sol::burnAndReturnValue()

```
/// @notice This is an ERC-20 contract to track claims to the aggregate repurchase obligations due on the repurchase date
    ↪ across all borrowers to a Term Repo

function burnAndReturnValue(
    address account,
    uint256 amount
) external onlyRole(BURNER_ROLE) returns (uint256) {
    _burn(account, amount);
    mintExposureCap += amount;
    uint256 valueBurned = truncate(
        mul_(
            Exp({mantissa: amount * expScale}),
            Exp({mantissa: redemptionValue})
        )
    );
    return valueBurned;
}
```

The collateral manager then pays out borrower-specific collateral and liquidation premium:

TermRepoCollateralManager.sol::batchDefaultWithRepoToken()

```
function batchDefaultWithRepoToken(
    address borrower,
    uint256[] calldata closureRepoTokenAmounts
) external {
    // ... snip ...
    closureValue = termRepoServicer.liquidatorCoverExposureWithRepoToken(
        borrower,
        msg.sender,
        closureRepoTokenAmounts[i]
    );

    (
        collateralSeizureAmount,
        collateralSeizureProtocolShare
    ) = _collateralSeizureAmounts(closureValue, collateralTokens[i]);

    _transferLiquidationCollateral(
        borrower,
        msg.sender,
        collateralTokens[i],
        closureValue,
        collateralSeizureAmount,
        collateralSeizureProtocolShare,
        true
    );
}
```

That lets an active holder choose the healthiest delinquent borrower or the best collateral bucket, then recover ahead of passive holders. The same pattern exists in `batchLiquidationWithRepoToken()`. The default path is the clearest case because any delinquent borrower can be targeted once the repurchase window expires.

This issue has a medium impact because it redistributes value among holders in distressed terms rather than draining unrelated protocol funds. It has a medium likelihood because the path is permissionless once shortfall or default conditions exist, but it still depends on multiple borrowers with different collateral quality and an active holder willing to choose a target.

Recommendations

Remove the liquidation premium when the liquidator covers exposure with repo tokens, or settle repo-token liquidations against a pool-wide shortfall-adjusted value instead of fixed `redemptionValue`. If repo tokens are meant to be fungible aggregate claims, they should not provide a path to selective recovery.

Resolution

The development team has acknowledged the finding with the following comment:

"This behavior was intentionally considered in the design. The ability for repo token holders to extinguish borrower-specific exposure by burning their claim was introduced to ensure liveness in distressed scenarios, particularly where no external liquidators are available or willing to act. This path is permissionless and equally accessible to all repo token holders, which preserves fairness at the mechanism level. The tradeoff is justified to guarantee liquidation liveness and avoid reliance on external capital during periods of stress."

TRF-35 Rollover Bids Bypass Initial Collateral Ratio Check			
Assets	contracts/TermAuctionBidLocker.sol contracts/TermRepoRolloverManager.sol contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Rollover bids are accepted into a new term's auction without verifying that the borrower's collateral meets the new term's initial collateral ratio, allowing a borrower whose collateral has declined into the maintenance-to-initial band to roll into a position that would be rejected as a fresh bid: exposing new-term lenders to weaker origination standards.

`_lockRolloverBid()` in `TermAuctionBidLocker.sol` registers rollover bids with the following validation:

TermAuctionBidLocker.sol::_lockRolloverBid()

```
function _lockRolloverBid(TermAuctionBid memory bid) internal {
    // reject rollover bid if auction is full
    if (bidCount >= MAX_BID_COUNT) {
        revert MaxBidCountReached();
    }
    if (!bid.isRollover) {
        revert NonRolloverBid(bid.id);
    }
    if (bid.amount < minimumTenderAmount / MINIMUM_ROLLOVER_TENDER_DIVISOR) {
        revert BidAmountTooLow(bid.amount);
    }
    // @audit no _isInInitialCollateralShortFall() check – compare with _lock() below
    bool bidExists = bids[bid.id].amount != 0;
    if (purchaseToken != bid.purchaseToken) {
        revert InvalidPurchaseToken();
    }
    bids[bid.id] = bid;
    // ... snip ...
}
```

By contrast, `_lock()` for fresh bids gates on the initial collateral ratio:

TermAuctionBidLocker.sol::_lock()

```
if (
    /// @dev check to see if bid has enough collateral to meet initial ratio for purchase price
    _isInInitialCollateralShortFall(
        bidSubmission.amount,
        bidSubmission.collateralTokens,
        bidSubmission.collateralAmounts
    )
) {
    revert CollateralAmountTooLow(); // @audit absent from _lockRolloverBid()
}
```

The rollover bid's `collateralAmounts` array is initialised to all-zeros in `_processRollover()`:

TermRepoRolloverManager.sol::_processRollover()

```
uint256[] memory collateralAmounts = new uint256[](
    collateralTokens.length
); // @audit all zero – no collateral at lock time

TermAuctionBid memory termAuctionBid = TermAuctionBid({
    // ... snip ...
    collateralAmounts: collateralAmounts,
    isRollover: true,
    // ... snip ...
});
termAuctionBidLocker.lockRolloverBid(termAuctionBid);
```

At auction clearing, `_processRevealedBidsForValidity()` scales old-term collateral onto the rollover bid and checks against `_isInMaintenanceCollateralShortFall()`:

TermAuctionBidLocker.sol::_processRevealedBidsForValidity()

```
if (revealedBid.isRollover) {
    // ... snip ...
    if (!_isRolloverStillValid(revealedBid, pairOffServicer)) {
        // mark for unlocking
        continue;
    }
}
// Both fresh and rollover bids reach this check – but it uses maintenanceCollateralRatios
if (
    _isInMaintenanceCollateralShortFall( // @audit maintenance ratio, not initial ratio
        revealedBid.amount,
        revealedBid.bidPriceRevealed,
        revealedBid.collateralTokens,
        revealedBid.collateralAmounts
    )
) { /* mark for unlocking */ }
```

Since `initialCollateralRatios >= maintenanceCollateralRatios` by protocol invariant, a borrower whose collateral sits between the two thresholds can roll over despite being ineligible for a fresh bid. The gap is typically 20-30 percentage points — a meaningful reduction in the collateral buffer. Borrowers are economically incentivised to exploit this since rollover is their only path once the initial threshold is breached.

This issue has a medium impact because new-term lenders receive weaker collateral backing than the initial ratio guarantees, increasing their exposure to liquidation shortfall. This issue has a medium likelihood because it requires collateral in the maintenance-to-initial band, a realistic market condition the borrower is economically incentivised to exploit.

Recommendations

Apply the initial collateral ratio check at `_isRolloverStillValid()` where the proportional collateral amounts are recomputed. The lock-time check alone is insufficient because `_isRolloverStillValid()` recalculates collateral proportions later, potentially producing amounts that no longer satisfy the initial ratio even if they passed at lock time.

Resolution

The development team has acknowledged the finding with the following comment:

“Rollover bids are intentionally not required to meet the new term’s initial collateral ratio. Borrowers may reduce collateral to the maintenance ratio immediately after an auction, so lenders cannot rely on positions remaining at the initial ratio post-origination. The initial margin primarily protects borrowers from

immediate liquidation rather than serving as a persistent guarantee to lenders. For existing positions being rolled over, we do not require borrowers to top up collateral solely to extend the loan, as this would impose unnecessary friction without materially improving lender protections beyond the enforced maintenance threshold."

TRF-36 Silent Unlock Failures Strand Collateral And Double-Charge Borrowers			
Assets	contracts/TermRepoCollateralManager.sol contracts/TermRepoServicer.sol contracts/facets/TermRouterFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

When a collateral token transfer fails silently during the zero-obligation cleanup flow, collateral can remain stuck in `TermRepoLocker` for the borrower even though the protocol has already treated it as unencumbered. In the flash-repurchase path this becomes direct user loss: `submitRepurchasePaymentHook()` can still pull the same collateral token from the borrower wallet, so a fully repaid borrower can lose fresh wallet collateral while the original locked collateral remains stranded.

The root cause is the empty `catch` block in `_unlockCollateral()` in `TermRepoCollateralManager`. If `termRepoLocker.transferTokenToWallet()` reverts, the function swallows the failure and leaves the borrower's locked balance untouched:

```
TermRepoCollateralManager.sol::_unlockCollateral()

function _unlockCollateral(
    address borrower,
    address collateralToken,
    uint256 amount,
    bool decrementEncumberedCollateral
) internal {
    if (amount > lockedCollateralLedger[borrower][collateralToken]) {
        revert UnlockAmountGreaterThanCollateralBalance();
    }

    try termRepoLocker.transferTokenToWallet(borrower, collateralToken, amount) {
        lockedCollateralLedger[borrower][collateralToken] -= amount;
        if (decrementEncumberedCollateral) {
            encumberedCollateralBalances[collateralToken] -= amount;
        }
        emitter.emitCollateralUnlocked(
            termRepoId,
            borrower,
            collateralToken,
            amount
        );
    } catch {
        // Transfer failed
    }
}
```

This behaviour compounds in the repurchase flow. `TermRepoServicer._submitRepurchasePaymentInternal()` repays first and then unconditionally attempts to unlock all collateral once the borrower's exposure reaches zero:

TermRepoServicer.sol::_submitRepurchasePaymentInternal()

```
function _submitRepurchasePaymentInternal(address borrower, uint256 amount) internal {
    ...
    _repay(borrower, msg.sender, amount);

    if (repurchaseExposureLedger[borrower] == 0) {
        termRepoCollateralManager.unlockCollateralOnRepurchase(borrower);
    }

    emitter.emitRepurchasePaymentSubmitted(termRepoId, borrower, amount);
}
```

`unlockCollateralOnRepurchase()` pre-decrements `encumberedCollateralBalances` before calling `_unlockCollateral()`, so a failed transfer leaves the protocol in a state where the collateral is still locked for the borrower but no longer counted as encumbered:

TermRepoCollateralManager.sol::unlockCollateralOnRepurchase()

```
function unlockCollateralOnRepurchase(address borrower) external onlyRole(SERVICER_ROLE) {
    for (uint256 i = 0; i < collateralTokens.length; ++i) {
        address collateralToken = collateralTokens[i];
        uint256 collateralAmount = lockedCollateralLedger[borrower][collateralToken];

        if (collateralAmount > 0) {
            encumberedCollateralBalances[collateralToken] -= collateralAmount;
            _unlockCollateral(borrower, collateralToken, collateralAmount, false);
        }
    }
}
```

This alone strands collateral. The impact escalates via the flash hook in `TermRouterFacet`. `submitRepurchasePaymentHook()` validates the requested output against the borrower's pre-repayment locked balance, submits the repayment, and then unconditionally transfers `minOutputAmount` of the collateral token from the borrower wallet:

TermRouterFacet.sol::submitRepurchasePaymentHook()

```
function submitRepurchasePaymentHook(
    ActionHookInput calldata input
) external onlyFlashLoanContext(input.user) nonReentrant {
    address borrower = input.user;
    address collateralToken = input.outputToken;
    uint256 collateralAmount = input.minOutputAmount;

    ITermRepoServicer repoServicer = ITermRepoServicer(input.targetAddress);
    _validateCollateralAmount(repoServicer, borrower, collateralToken, collateralAmount);
    ...
    repoServicer.submitRepurchasePayment(borrower, repaymentAmount);
    ...
    IERC20(collateralToken).safeTransferFrom(borrower, address(this), collateralAmount);
}
```

Because `_validateCollateralAmount()` checks `getCollateralBalance()` before repayment, it approves an output amount that only becomes available after `unlockCollateralOnRepurchase()` succeeds. If the unlock silently fails, the hook does not notice. The borrower's obligation drops to zero, the original collateral remains locked, and the hook still removes `collateralAmount` from the borrower wallet to satisfy the flash step.

The resulting state is:

- `repurchaseExposureLedger[borrower]` is zero
- the originally locked collateral is still recorded in `lockedCollateralLedger`
- the hook recipient has received `collateralAmount`
- the borrower wallet has lost `collateralAmount` of the same token

This issue has a medium impact because it does not create protocol-wide insolvency but does result in direct borrower loss when the flash hook pulls replacement collateral while the original locked collateral remains stranded.

This issue has a medium likelihood because the path only needs a token that can fail on transfer while the borrower still has approved wallet balance. This is a realistic edge condition for blacklistable, pausable, or policy-constrained collateral assets, and once it occurs the flash hook deterministically converts the silent unlock failure into same-transaction wallet loss.

Recommendations

Remove the silent catch behaviour in `_unlockCollateral()`. Either remove the try/catch entirely so `transferTokenToWallet()` reverts normally, or keep the try/catch and explicitly revert in the catch branch.

Resolution

The development team has resolved the hook escalation path in [PR#1625](#).

The `submitRepurchasePaymentHook()` now reverts if collateral remains locked after repayment, preventing the double-charge scenario where the hook pulls fresh collateral from the borrower wallet while the original collateral stays stranded.

TRF-37	Unrevealed Rollover Bids Can Bypass The Public Auction Reveal Gate		
Assets	contracts/TermAuction.sol contracts/TermAuctionBidLocker.sol contracts/TermRepoRolloverManager.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Any third party can bypass `completeAuction()`'s non-admin "all prices revealed" guard by placing an unrevealed rollover bid in the revealed-bid array. `TermAuctionBidLocker` accepts it without checking `isRevealed`, so the auction completes against a rollover bid whose `bidPriceRevealed` is still zero. The bid is then irreversibly marked processed, blocking the borrower from re-electing rollover.

`TermAuction.completeAuction()` is public after `auctionEndTime`. For non-admin callers it rejects calls that include unrevealed tenders:

TermAuction.sol

```
if (!hasRole(ADMIN_ROLE, msg.sender)) {
  if (
    completeAuctionInput.unrevealedBidSubmissions.length > 0 ||
    completeAuctionInput.unrevealedOfferSubmissions.length > 0
  ) {
    revert InvalidParameters(
      "All tender prices must be revealed for auction to be complete"
    );
  }
}
```

Bid revelation remains open after `auctionEndTime`, `revealBids()` only checks `block.timestamp >= revealTime`, with no upper bound:

TermAuctionBidLocker.sol

```
modifier onlyWhileAuctionRevealing() {
  if (block.timestamp < revealTime) {
    revert AuctionNotRevealing();
  }
  _;
}
```

However, in `TermAuctionBidLocker._processRevealedBidsForValidity()`, rollover bids skip the `isRevealed` check:

TermAuctionBidLocker.sol

```
if (revealedBid.isRollover) { // @audit - isRevealed not checked for rollover bids; unrevealed bid passes with
    // bidPriceRevealed == 0
    // ... snip ...
    if (!_isRolloverStillValid(revealedBid, pairOffServicer)) {
        // ... snip ...
    }
} else {
    if (!revealedBid.isRevealed) {
        revert BidNotRevealed(revealedBid.id);
    }
}
```

A public caller can place an unrevealed rollover bid ID in `completeAuctionInput.revealedBidSubmissions` instead of `unrevealedBidSubmissions`, bypassing the public reveal gate without admin privileges.

From there the rollover is processed using a zero revealed price:

- if the auction does not clear, `completeAuction()` walks `sortedBids` and calls `_markRolloverAsProcessed()` for rollover bids in the cancellation path;
- if the auction does clear at a positive price, the zero-priced rollover falls into `_assignBids()`'s "No assignment" branch, which also calls `_markRolloverAsProcessed()`.

TermAuction.sol

```
if (sortedBids[i].isRollover) {
  _markRolloverAsProcessed( // @audit - permanently marks rollover processed even when bid was never revealed
    sortedBids[i].rolloverPairOffTermRepoServicer,
    sortedBids[i].bidder
  );
}
```

Once marked processed, the borrower cannot re-elect rollover:

TermRepoRolloverManager.sol

```
if (rolloverElections[borrower].processed) {
  revert RolloverProcessedToTerm(); // @audit - re-election permanently blocked once processed, regardless of how it
  ↳ occurred
}
```

The rollover bid ID is not secret, it is emitted in rollover-related events and deterministically derived from the rollover manager and borrower address. An attacker can front-run the borrower's reveal after auction close and force the rollover down the failure path.

This is distinct from the general "processed on failure" rollover issue: here the failure becomes attacker-triggerable by any public caller while the non-admin completion guard is supposed to preserve the borrower's ability to reveal first.

This issue has a medium impact because a borrower's rollover can be irreversibly marked processed without assignment, stranding residual debt and potentially forcing liquidation or default. This issue has a medium likelihood because the rollover bid ID is publicly observable, the attack window exists after every auction close, and no privileges are required.

Recommendations

Require rollover bids passed through the revealed-bid path to satisfy the same reveal invariant as normal bids:

- reject any rollover bid in `revealedBidSubmissions` unless `isRevealed == true`; or
- alternatively, treat unrevealed rollover bids as unrevealed for the purpose of the public `completeAuction()` gate.

If rollover bids intentionally bypass manual reveal, the protocol should remove the misleading price-hash / reveal workflow for rollovers and redesign the public completion guard, because the current hybrid design lets arbitrary callers force completion against a still-unrevealed rollover.

Resolution

The development team has fixed this issue in [PR#1626](#) by adding in `_processRevealedBidsForValidity()` the `isRevealed` check for rollover bids.

TRF-38 Intent Settlement Bypasses Protocol servicingFee() Enforcement			
Assets	contracts/TermRepoServicer.sol contracts/facets/TermLoanIntentFacet.sol contracts/facets/TermRouterFacet.sol contracts/facets/TermStrategyFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Low	Likelihood: High

Description

Intent-based loan settlement lets makers set `borrowFee` to zero and open repo exposure without paying the protocol fee enforced by the normal minting paths. That makes the intent path a fee bypass for new origination.

`mintOpenExposureFromIntent()` does not apply `servicingFee()` or mint any fee share to the treasury:

TermRepoServicer.sol::mintOpenExposureFromIntent()

```
function mintOpenExposureFromIntent(
    address borrower,
    address lender,
    uint256 purchaseTokenAmount,
    uint256[] calldata collateralAmounts,
    uint256 offerRate,
    bool isRoutedCollateral
) external onlyRole(DIAMOND_ROLE) returns (uint256) {
    uint256 totalMintValue = termRepoToken.mintTokens(
        lender,
        amountOfRepoTokenToMint // @audit no protocol servicingFee deducted; full amount minted to lender
    );
}
```

Instead, the intent settler trusts maker-supplied fee data:

```
if (!s.approvedFeeRecipients[order.feeRecipient]) {
    revert InvalidFeeRecipient();
}
```

TermLoanIntentFacet.sol::_settlePurchaseTokensAndFees()

```

function _settlePurchaseTokensAndFees(
    ITermRepoServicer servicer,
    address borrower,
    address lender,
    uint256 fillAmount,
    uint256 borrowFee,
    address feeRecipient,
    bool isRoutedPurchaseToken,
    bool useTransfer2
) private returns (uint256) {
    uint256 proratedFee = mul_ScalarTruncate(
        mul_(
            Exp({mantissa: borrowFee}), // @audit borrowFee is maker-supplied; can be zero, bypassing protocol fee
            div_(
                Exp({mantissa: (servicer.maturityTimestamp() - block.timestamp)}),
                Exp({mantissa: YEAR_SECONDS})
            )
        ),
        fillAmount
    );

    address purchaseToken = servicer.purchaseToken();

    _executeTokenTransfer(
        purchaseToken,
        isRoutedPurchaseToken,
        lender,
        borrower,
        feeRecipient,
        fillAmount,
        proratedFee,
        useTransfer2
    );
}

```

The normal mint path does charge the protocol fee and mints it to `termController.getTreasuryAddress()`:

TermRepoServicer.sol

```

Exp memory protocolShareProRated = mul_(
    Exp({mantissa: servicingFee}),
    proRate
);

uint256 protocolMintTokens = mul_ScalarTruncate(
    protocolShareProRated,
    amount
);
uint256 minterTokens = amount - protocolMintTokens;

uint256 protocolMintTokensValue = termRepoToken.mintTokens(
    termController.getTreasuryAddress(),
    protocolMintTokens
);
uint256 minterTokensValue = termRepoToken.mintTokens(
    msg.sender,
    minterTokens
);

```

The normal mint path also deducts via:

```
Exp({mantissa: servicingFee}), // @audit this protocol fee deduction is absent from the intent path
```

Because `borrowFee` is part of the signed order, any maker can publish intent orders with `borrowFee = 0` and still satisfy the onchain checks.

This issue has a low impact because it does not create unbacked exposure or transfer existing protocol funds, but it suppresses fee revenue on all intent-originated positions and creates a persistent incentive to avoid fee-enforced flows. This issue has a high likelihood because setting `borrowFee` to zero requires no special permissions

and validation checks only recipient approval.

Recommendations

Enforce the protocol minimum fee inside the intent settlement path. `borrowFee` should only be able to add to the protocol fee, not replace it.

Resolution

The development team has acknowledged the finding with the following comment:

"We explicitly bypass the servicing fee to make the borrowFee more dynamic. The borrowFee will be set by the frontend. While it is possible that a user can create their own calldata with borrowFee set to zero, we are willing to accept that risk."

TRF-39	WETHWrappingFacet Is Unusable Through The Diamond		
Assets	contracts/TermDiamond.sol contracts/facets/WETHWrappingFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Low	Likelihood: High

Description

The diamond cannot receive native ETH, but `WETHWrappingFacet.wrapETH()` and `WETHWrappingFacet.unwrapETH()` both require the proxy to accept it. Both entrypoints revert when called through the diamond, making the ETH wrapping interface unusable.

The proxy only exposes a non-payable `fallback()` with no `receive()` function:

TermDiamond.sol

```
fallback() external { // @audit - not payable; any call with msg.value reverts before delegation
    LibDiamond.DiamondStorage storage ds;
    bytes32 position = LibDiamond.DIAMOND_STORAGE_POSITION;
    // ... snip ...
    address facet = ds.selectorToFacetAndPosition[msg.sig].facetAddress;
    require(facet != address(0), "Diamond: Function does not exist");
    assembly {
        let result := delegatecall(gas(), facet, 0, calldatasize(), 0, 0)
        // ... snip ...
    }
}
```

`wrapETH()` fails because calls carrying `msg.value` revert before delegation to the facet:

WETHWrappingFacet.sol::wrapETH()

```
function wrapETH(
    address wrappedTokenAddr
) external payable { // @audit - payable but unreachable; diamond's non-payable fallback() rejects ETH before this executes
    IWETH wrappedToken = IWETH(wrappedTokenAddr);
    wrappedToken.deposit{value: msg.value}();
    wrappedToken.safeTransfer(msg.sender, msg.value);
}
```

`unwrapETH()` fails because `wrappedToken.withdraw(amount)` sends native ETH to `msg.sender` — the diamond during `delegatecall` — which cannot receive it:

WETHWrappingFacet.sol::unwrapETH()

```
function unwrapETH(
    uint256 amount,
    address wrappedTokenAddr,
    bool usePermit2
) external {
    IWETH wrappedToken = IWETH(wrappedTokenAddr);
    // ... snip ...
    wrappedToken.withdraw(amount); // @audit sends ETH to the diamond, which cannot receive it
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "ETH transfer failed");
}
```

The test helper confirms standalone deployment required a dedicated `receive()`:

TestWETHWrappingFacetHelper.sol

```
/// @dev The real diamond's fallback is non-payable, so standalone deploy needs this
///      to accept ETH from WETH.withdraw().
contract TestWETHWrappingFacetHelper is WETHWrappingFacet {
    receive() external payable {} // @audit - confirms the diamond needs receive() payable to enable unwrapETH()
}
```

This issue has a low impact because it does not let an attacker steal third-party funds, but it renders the ETH wrapping utility non-functional. This issue has a high likelihood because every `wrapETH()` and `unwrapETH()` call reverts deterministically on the production proxy.

Recommendations

Add a payable `receive()` to the diamond and make the proxy entrypoint payable for facets that accept native ETH. If the architecture must remain non-payable, remove `WETHWrappingFacet` or redesign `wrapETH()/unwrapETH()` so the WETH contract and the caller, rather than the diamond, handle native ETH.

Resolution

A `payable` keyword has been added to the `fallback` function, and a `payable receive()` function has been introduced in the `TermDiamond` contract in [PR#1600](#).

TRF-40 Removal Of Minimum Tender Amount Check Allows Dust Rollover Bids			
Assets	contracts/TermAuctionBidLocker.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Rollover bids bypass the minimum tender amount validation that applies to regular bids, allowing dust amount bids to be rolled over to auctions where they will never be matched, creating auction inefficiencies and wasted gas during clearing operations.

The `_lockRolloverBid()` function has its `minimumTenderAmount` check removed:

TermAuctionBidLocker.sol::_lockRolloverBid()

```
// reject rollover bid if auction is full
if (bidCount >= MAX_BID_COUNT) {
    revert MaxBidCountReached();
}

if (!bid.isRollover) {
    revert NonRolloverBid(bid.id);
}

// @audit - minimum tender amount check is removed

bool bidExists = bids[bid.id].amount != 0;

if (purchaseToken != bid.purchaseToken) {
    revert InvalidPurchaseToken();
}

// ... check and event
```

When users submit rollover elections through the `TermRepoRolloverManager`, they can specify any `rolloverAmount` greater than 0:

TermRepoRolloverManager.sol::electRollover()

```
if (termRepoRolloverElectionSubmission.rolloverAmount == 0) {
    revert InvalidParameters("Rollover amount cannot be 0");
}
```

The rollover manager calculates `bidAmount` and creates rollover bids without considering minimum tender requirements:

TermRepoRolloverManager.sol::_processRollover()

```
// ... function logic
bidAmount = truncate(
    div_(
        Exp({mantissa: rolloverElection.rolloverAmount * expScale}),
        Exp({mantissa: expScale - servicingFeeProRatedMantissa})
    )
);
// ... loading memory
TermAuctionBid memory termAuctionBid = TermAuctionBid({
    amount: bidAmount, // @audit Can be dust amount that is < minimumTenderAmount
    isRollover: true
    // ... other data
});
// ... calling lock
```

A borrower could submit a rollover election for 1 wei of their repurchase obligation, creating an unmatchable dust bid that bypasses minimum tender validation. These dust amount rollover bids occupy bid slots but will never be matched during auction clearing since they fall below the economic viability threshold, creating inefficiency in the auction system.

Recommendations

Do not remove the `minimumTenderAmount` check, but instead add a method to change its value, so that it can be adjusted based on the tendency of rollover amounts that are in the active terms. Another solution would be to separate the `minimumTenderAmount` for normal locks and for rollovers, allowing admins to adjust both.

Resolution

The development team has addressed this vulnerability in [PR#1463](#) by reintroducing minimum tender amount validation for rollover bids. The fix implements a `MINIMUM_ROLLOVER_TENDER_DIVISOR = 10` constant that requires rollover bid amounts to be at least `minimumTenderAmount / 10`. This approach provides a lower threshold for rollover bids while still preventing dust amount elections from creating unmatchable bids that would occupy auction slots without economic viability.

TRF-41	ERC20 Token Decimals Handling Vulnerabilities In <code>usdValueOfTokens()</code>		
Assets	<code>contracts/TermPriceConsumerV3.sol</code>		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The `usdValueOfTokens()` function in `TermPriceConsumerV3` contains faulty logic that could cause transaction reverts for non-compliant tokens and zero price calculations for high-decimal tokens.

The function unconditionally calls `decimals()` without error handling:

TermPriceConsumerV3.sol::usdValueOfTokens()

```
// ...
IERC20MetadataUpgradeable tokenInstance = IERC20MetadataUpgradeable(
    token
);
uint8 tokenDecimals = tokenInstance.decimals(); // @audit - this could revert
// ...
```

This will cause reverts for tokens that do not implement the optional `decimals()` function as per the ERC20 specification.

Additionally, extremely high decimal values (18+) could lead to `usdValueOfTokens()` returning 0 because of Solidity's integer division:

TermPriceConsumerV3.sol::usdValueOfTokens()

```
function usdValueOfTokens(
    address token,
    uint256 amount
) external view returns (Exp memory) {
    // ... function logic
    return
        mul_(
            Exp({mantissa: (amount * expScale) / 10 ** tokenDecimals}), // @audit - this could truncate down to 0
            Exp({mantissa: (latestPrice * expScale) / 10 ** priceDecimals})
        );
}
```

For instance, with a 30-decimal token and `amount = 1000`, the result becomes $(1000 * 1e18) / 1e35 = 0$, making USD valuation return zero regardless of actual price.

Recommendations

Implement safe decimal handling:

TermPriceConsumerV3.sol::usdValueOfTokens()

```
function _safeDecimals(address token) internal view returns (uint8) {  
    try IERC20MetadataUpgradeable(token).decimals() returns (uint8 decimals) {  
        return decimals > 30 ? 18 : decimals; // Cap high decimals  
    } catch {  
        return 18; // Default for non-compliant tokens  
    }  
}
```

This prevents both DoS scenarios and financial miscalculations affecting core protocol operations.

Resolution

The development team has acknowledged this vulnerability and confirmed that appropriate risk mitigation measures are in place. Rather than implementing the recommended code-level safeguards, the team relies on a comprehensive token onboarding process that includes simulated deployments and thorough vetting procedures.

The development team has provided the following response:

"We agree with this assessment. We have a thorough process for onboarding tokens/price feeds to TermPriceConsumerV3 that includes a simulated deploy on a fork with a price feed before the real addition. So the risk of this issue is mitigated."

This administrative control approach effectively mitigates the risk by ensuring only compliant tokens with verified decimal implementations are approved for use in the protocol.

TRF-42	Fixed Precision Does Not Cater For Various Tokens		
Assets	contracts/TermRepoServicer.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

Current tolerance logic using `10 ** 4` is not compatible with tokens of varying decimals, leading to inconsistent behaviour across assets.

TermRepoServicer.sol::_isTermRepoBalanced()

```

if (shortfallHaircutMantissa == 0) {
    uint256 totalLiquidity = totalOutstandingRepurchaseExposure + totalRepurchaseCollected;
    uint256 totalRedemptionValue = termRepoToken.totalRedemptionValue();
    if (totalLiquidity >= totalRedemptionValue) {
        return totalLiquidity - totalRedemptionValue <= 10 ** 4;
    } else {
        return totalRedemptionValue - totalLiquidity <= 10 ** 4;
    }
}
// ... function logic

```

In this code, the variables `totalLiquidity` and `totalRedemptionValue` have the precision of the `TermRepo` token. However, the system currently uses a hardcoded value of `10 ** 4` for tolerances or scaling without accounting for the number of decimals a token uses. This creates an implicit assumption that all tokens have the same precision – typically 18 decimals – which is not always the case.

When dealing with tokens that use fewer decimals (e.g., 6-decimal stablecoins like USDC), applying the same fixed scaling factor results in disproportionate tolerances or thresholds. This inconsistency can cause critical logic (such as comparisons, calculations, or validations) to behave incorrectly, especially in cross-token or multi-asset environments.

Recommendations

To ensure correctness and maintainability, this fixed value should be replaced with a dynamically calculated constant based on each token's `decimals()` property. This allows the system to apply a consistent and meaningful scale or tolerance relative to the precision of each token.

Resolution

The development team has addressed this vulnerability in [PR#1465](#) by introducing a new state variable `termRepoBalancedThreshold` that dynamically calculates precision tolerances based on the specific token's decimal places. This replaces the hardcoded `10 ** 4` value with a token-aware threshold that ensures consistent behaviour across assets with varying decimal precision.

TRF-43	Incorrect closureAmount Emitted In Repo Token Liquidation Events		
Assets	contracts/TermRepoCollateralManager.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `batchLiquidationWithRepoToken()` and `batchDefaultWithRepoToken()` functions in `TermRepoCollateralManager` emit incorrect `closureAmount` values in liquidation events, passing repo token amounts instead of the calculated closure values from debt repayment.

Both functions correctly calculate the actual closure value by calling `liquidatorCoverExposureWithRepoToken()` in the `termRepoServicer` contract, but then incorrectly pass the repo token amount to `_transferLiquidationCollateral()`, resulting in misleading event data for offchain monitoring and analytics.

TermRepoCollateralManager.sol::batchLiquidationWithRepoToken()

```
// ... function logic
closureValue = termRepoServicer.liquidatorCoverExposureWithRepoToken(borrower, msg.sender, closureRepoTokenAmounts[i]);

(collateralSeizureAmount, collateralSeizureProtocolShare) = _collateralSeizureAmounts(closureValue, collateralTokens[i]);

_transferLiquidationCollateral(
    borrower,
    msg.sender,
    collateralTokens[i],
    closureRepoTokenAmounts[i], // @audit - should be closureValue, not repo token amount
    collateralSeizureAmount,
    collateralSeizureProtocolShare,
    false
);
// ... function logic
```

TermRepoCollateralManager.sol::batchDefaultWithRepoToken()

```
// ... function logic
closureValue = termRepoServicer.liquidatorCoverExposureWithRepoToken(borrower, msg.sender, closureRepoTokenAmounts[i]);

(collateralSeizureAmount, collateralSeizureProtocolShare) = _collateralSeizureAmounts(closureValue, collateralTokens[i]);

_transferLiquidationCollateral(
    borrower,
    msg.sender,
    collateralTokens[i],
    closureRepoTokenAmounts[i], // @audit - should be closureValue, not repo token amount
    collateralSeizureAmount,
    collateralSeizureProtocolShare,
    false
);
// ... function logic
```

This causes `emitter.emitLiquidation()` to record the repo token amount as the closure amount instead of the actual debt repayment value:

TermRepoCollateralManager.sol::_transferLiquidationCollateral()

```
emitter.emitLiquidation(  
    termRepoId,  
    borrower,  
    liquidator,  
    closureAmount, // @audit - this receives repo token amount instead of closure value  
    collateralAddress,  
    collateralSeizureAmount,  
    collateralSeizureProtocolShare,  
    isDefault  
);
```

Offchain systems monitoring liquidation events receive incorrect closure amounts, potentially affecting analytics, accounting, and risk management systems that rely on accurate liquidation data.

Recommendations

Change `closureRepoTokenAmounts[i]` to `closureValue` in the `_transferLiquidationCollateral()` calls for both repo token batch functions.

Resolution

The development team has addressed this vulnerability in [PR#1459](#) by modifying the `batchLiquidationWithRepoToken()` and `batchDefaultWithRepoToken()` functions to pass the calculated `closureValue` parameter instead of the `closureRepoTokenAmounts[i]` to the `_transferLiquidationCollateral()` function. This ensures that liquidation events emit the correct debt closure amounts rather than the repo token amounts used for liquidation.

TRF-44	Leftover Collateral From Batch Default Functions Becomes Inaccessible After <code>mintOpenExposure</code>		
Assets	<code>contracts/TermRepoCollateralManager.sol</code>		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `batchDefault()` and `batchDefaultWithRepoToken()` functions in `TermRepoCollateralManager` do not automatically unlock remaining collateral after full debt repayment, causing a minor issue where borrowers who call `mintOpenExposure()` before manually withdrawing their leftover collateral will be unable to access that collateral until they repay the full debt from `mintOpenExposure()`.

The batch default functions call `_unencumberRemainingBorrowerCollateralOnZeroObligation(borrower)` but omit the automatic `_unlockCollateral()` loop present in batch liquidation functions. When borrowers call `mintOpenExposure()` before withdrawing leftover collateral, the leftover becomes mixed with new collateral in `lockedCollateralLedger[borrower][token]` and becomes inaccessible due to maintenance ratio requirements.

Borrowers in this situation cannot access their leftover collateral until they call `submitRepurchasePayment()` to repay the full `mintOpenExposure()` debt, which triggers `unlockCollateralOnRepurchase()` to return all collateral.

Recommendations

The batch default function should call `_unlockCollateral()` the same way batch liquidation functions do.

Resolution

The development team has clarified that this scenario cannot occur due to the protocol's operational timeline. Batch default operations are only executed after the repayment period ends, which occurs after maturity when `mintOpenExposure()` is already disabled. Since borrowers cannot call `mintOpenExposure()` after batch defaults have been processed, the described collateral mixing issue is not possible within the normal protocol flow. Therefore, no code changes are required to address this finding.

TRF-45	mintExposureCap Is Defined Per Chain Causing Incorrect Accounting		
Assets	contracts/TermRepoToken.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The `mintExposureCap` variable is native to the `TermRepoToken` contract and is set per chain. When tokens are debited through `_debit()`, the cap on the source chain is not updated nor is it updated on the destination chain.

This leads to inconsistent mint tracking across different chains. Consider the following scenario:

- Chain A and Chain B both have `mintExposureCap` set to 1000.
- Chain A currently has 1000 tokens minted (it is at capacity).
- Alice sends 500 tokens from Chain A to Chain B via `_debit()`.
- The 500 tokens are burnt on Chain A, but `mintExposureCap` remains at 1000.
- Chain A incorrectly appears at capacity despite only having 500 tokens remaining.

Recommendations

Update the `mintExposureCap` variable during credit and debit actions such that as tokens exit from one chain to another the variable is updated accordingly.

Resolution

The development team has advised that they decided not to implement the OFT feature and, as such, the finding is now closed.

TRF-46	Incomplete Facet Cleanup On Last Selector Removal Risks EIP2535 Noncompliance		
Assets	contracts/LibDiamond.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When removing the final function selector for a facet, the diamond implementation clears the `facetAddressPosition` and removes the facet address from the array, but fails to delete the corresponding `facetFunctionSelectors[_facetAddress]` entry.

This leaves stale struct data that can later accept new selectors if the same facet address is reused, potentially creating inconsistent state and risking EIP-2535 compliance violations during subsequent add or replace operations that require proper admin configuration.

During the selector removal process in `LibDiamond.sol`, when the final selector for a facet is removed, the implementation performs partial cleanup:

libraries/LibDiamond.sol::removeFunction()

```
// @audit When removing final selector:
if (lastSelectorPosition == 0) {
    uint256 lastFacetAddressPosition = ds.facetAddresses.length - 1;
    uint256 facetAddressPosition = ds.facetFunctionSelectors[_facetAddress].facetAddressPosition;

    if (facetAddressPosition != lastFacetAddressPosition) {
        // @audit Move last address to deleted position
        address lastFacetAddress = ds.facetAddresses[lastFacetAddressPosition];
        ds.facetAddresses[facetAddressPosition] = lastFacetAddress;
        ds.facetFunctionSelectors[lastFacetAddress].facetAddressPosition = facetAddressPosition;
    }
    ds.facetAddresses.pop();
    delete ds.facetFunctionSelectors[_facetAddress].facetAddressPosition;
}
```

The stale entry retains the previous `facetAddressPosition` value, which may become invalid and cause inconsistencies if the same facet address is later re-added through admin operations.

Recommendations

Delete the entire facet entry when its last selector is removed:

libraries/LibDiamond.sol::removeFunction()

```
- delete ds.facetFunctionSelectors[_facetAddress].facetAddressPosition;
+ delete ds.facetFunctionSelectors[_facetAddress];
```

Resolution

The recommended fix has been applied and can be found in the following commit: [0bbbe1c](#).

TRF-47 Unnecessary Payable Entry Points Allow Stuck ETH			
Assets	contracts/TermDiamond.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The diamond accepts native ether despite the protocol not handling it, which enables accidental ETH deposits that become stuck.

Allowing `payable` on the constructor and `fallback`, and exposing an empty `receive()` function, permits ETH to be sent to the diamond without any withdrawal path. This can lead to stuck funds and unexpected onchain behaviours if any facet later relies on `msg.value`.

Recommendations

- Remove `payable` from the constructor and `fallback()`.
- Remove the `receive()` function, or make it revert with a clear error if ETH is sent.

Resolution

The recommended fix has been applied and can be found in the following commit: [f64d019](#).

TRF-48	Permissionless Redemption Enables Front-Run Griefing		
Assets	contracts/TermRouterFacet.sol contracts/TermRepoServicer.sol contracts/TermRepoToken.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `redeemTermRepoTokens()` function in `TermRouterFacet` allows permissionless redemption on behalf of any address without authorisation, enabling adversaries to front-run legitimate redemption transactions and cause them to revert through balance manipulation. An attacker can monitor pending redemption transactions, burn a small amount of the victim's tokens first, then cause the victim's transaction to fail due to insufficient balance when their transaction executes with stale parameters.

Attack Scenario:

1. Victim submits a transaction to redeem their full balance (e.g., 1000 tokens).
2. Adversary monitors the mempool and front-runs with `redeemTermRepoTokens(victim, 1)`.
3. Victim's balance reduces from 1000 to 999 tokens.
4. Victim's original transaction executes with a stale amount (1000) and reverts due to the insufficient balance.
5. Adversary can repeat this process indefinitely to prevent victim's redemption.

Recommendations

Do not allow redemptions on behalf of other addresses or add a "redeem all" endpoint that reads the current balance onchain to avoid stale amount underflows.

Resolution

The development team has acknowledged the front-running possibility but has decided risk is outweighed by the benefit of allowing redemptions on behalf of users, providing the following response:

"This is a rare scenario, and its risk is outweighed by the benefit of allowing redemptions on behalf of users—particularly when the user is a smart contract or when their redemption is blocking another user's redemption, as in the rounding-error case addressed in part 1 of the audit."

TRF-49 NatSpec Claims SETTLER_ROLE Restriction Which Is Not Implemented			
Assets	contracts/ERC4626InterfaceFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `ERC4626InterfaceFacet` contract documentation states, "This facet allows `SETTLER_ROLE` to interact with ERC4626 compliant vaults." However, it does not implement any access control mechanisms.

All functions within the facet are permissionless, allowing any user to call vault interaction functions despite the explicit claim that these operations are restricted to the `SETTLER_ROLE`.

The facet's functions lack access control modifiers:

facets/ERC4626InterfaceFacet.sol::depositToVault()

```
function depositToVault(
    address vault,
    address asset,
    uint256 assets,
    address receiver
) external returns (uint256 shares) {
    // No access control modifier - should have onlyRole(SETTLER_ROLE)
}
```

facets/ERC4626InterfaceFacet.sol::withdrawFromVault()

```
function withdrawFromVault(
    address vault,
    address asset,
    uint256 assets,
    address receiver,
    address owner
) external returns (uint256 shares) {
    // No access control modifier - should have onlyRole(SETTLER_ROLE)
}
```

The documentation explicitly indicates role-based restrictions, but the implementation permits unrestricted access, creating a mismatch between intended design and actual behaviour.

The absence of access controls creates several security and operational concerns:

- **Unauthorised Vault Operations:** Any address can deposit or withdraw from ERC4626 vaults using the protocol's approvals and balances.
- **Protocol Asset Risk:** Unauthorised users could potentially manipulate vault positions that should be controlled by protocol settlers.
- **Governance Bypass:** The intended role-based governance model is completely circumvented for vault operations.
- **Integration Assumptions:** External systems expecting role-restricted access may be compromised by the permissionless nature.

Recommendations

Implement the documented access controls by adding appropriate role modifiers.

Alternatively, if these functions are intended to be permissionless, update the documentation to reflect the actual access model and carefully review the security implications of unrestricted vault access.

Resolution

The development team updated the NatSpec and removed the use of `SETTLER_ROLE`.

The fix can be found in the following commit: [1c07922](#).

TRF-50 Missing Oracle Validation			
Assets	contracts/TermMorphoInterfaceFacet.sol contracts/TermAaveInterfaceFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `availableBorrow()` function in the `TermMorphoInterfaceFacet.sol` and `TermAaveInterfaceFacet.sol` facets calls an oracle without validating the oracle's legitimacy or price sanity, potentially returning incorrect borrowing capacity calculations based on manipulated or stale price data.

facets/external/TermMorphoInterfaceFacet.sol::availableBorrow()

```
MarketParams memory marketParams = morpho.idToMarketParams(marketId);
uint256 collateralPrice = IOracle(marketParams.oracle).price(); // No validation
```

facets/external/TermAaveInterfaceFacet.sol::availableBorrow()

```
// Get the asset price in base currency
uint256 assetPrice = oracle.getAssetPrice(asset); // No validation
```

This could lead to incorrect borrowing capacity calculations, though impact is limited since this is a view function that does not directly move funds.

Recommendations

Add basic oracle validation and price sanity checks, e.g.:

Recommended Oracle Validation

```
function validateOraclePrice(address oracle, uint256 price) internal view {
    require(oracle != address(0), "Invalid oracle address");
    require(price > 0, "Invalid price");
    require(price < MAX_REASONABLE_PRICE, "Price exceeds maximum threshold");

    // Additional checks for freshness if oracle supports it
    if (IOracle(oracle).supportsInterface(type(IAggregatorV3Interface).interfaceId)) {
        (, , , uint256 updatedAt, ) = IAggregatorV3Interface(oracle).latestRoundData();
        require(block.timestamp - updatedAt < STALE_PRICE_THRESHOLD, "Price data is stale");
    }
}
```

Alternatively, implement oracle whitelisting for known trusted oracle implementations.

Resolution

The development team implemented partial validation of the oracle address and price in the commit [d75e29c](#).

With regards to other fixes suggested, the development team provided the following response:

"Morpho oracles do not follow the AggregatorV3Interface, so don't think the supportsInterface call is going to work here. Staleness checks might be arbitrary since different oracle source feeds have different heart-

beats. Since this is a read function and we whitelist morpho markets for use from our web interface that works with our intents contracts, we think the 2 validations are sufficient."

TRF-51 Unsafe uint160 Casting In Permit2 Transfers Enables Accounting Manipulation			
Assets	contracts/GasTokenWrappingFacet.sol contracts/TermMorphoInterfaceFacet.sol contracts/TermAaveInterfaceFacet.sol contracts/TermLoanIntentFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Multiple facets use unsafe `uint160` casting for `Permit2` transfers while using the original `uint256` amount for protocol operations, creating truncation vulnerabilities that allow attackers to drain facet funds.

Several facets across the codebase perform unsafe casting from `uint256` to `uint160` when calling `Permit2`'s `transferFrom()` function.

This casting silently truncates values larger than `type(uint160).max` ($\approx 1.46 \times 10^{48}$) instead of reverting, creating a critical mismatch between the amount transferred via `Permit2` and the amount used in subsequent protocol operations.

Affected functions include:

- `TermAaveInterfaceFacet.aaveSupply()`
- `TermMorphoInterfaceFacet.morphoSupplyCollateral()`
- `TermMorphoInterfaceFacet.morphoSupply()`
- `GasTokenInterfaceFacet.unwrapGasToken()`
- `ERC4626InterfaceFacet.depositToVault()`
- `ERC4626InterfaceFacet.withdrawFromVault()`
- `ERC4626InterfaceFacet.erc4626Withdraw()`
- `ERC4626InterfaceFacet.erc4626Redeem()`
- `TermLoanIntentFacet._settleLendOrder()`
- `TermLoanIntentFacet._executeTokenTransfer()`

Consider the following attack scenario:

1. Attacker calls function with amount = `type(uint160).max + X` where X is desired profit
2. `Permit2` transfer only moves truncated amount (`type(uint160).max`) from attacker
3. Protocol operation uses full untruncated amount, consuming facets existing token balance
4. Attacker receives benefit of full amount while only paying truncated amount

This could be carried out with `aaveSupply()` as follows (Assumptions: `type(uint160).max = 1,000`, facet has 5,000 tokens from previous operations):

1. Attacker calls `aaveSupply(amount: 6,000)`
2. `Permit2` transfers only 1,000 tokens from attacker (truncated)
3. `aaveSupply()` uses full 6,000 tokens (1,000 from attacker + 5,000 from facet)
4. User receives 6,000 aTokens but only paid 1,000 tokens

This could lead to the following:

- **Critical fund drainage:** Attackers can steal accumulated facet tokens
- **Amplified by `payoutToUser=false`:** Functions that retain tokens in facet increase available balance for exploitation
- **Cross-function vulnerability:** Multiple entry points enable various attack vectors
- **Bypasses economic constraints:** Attacker needs minimal capital to drain large amounts

Recommendations

Replace the unsafe casting with OpenZeppelin's `SafeCast` to ensure the function reverts on overflow rather than silently truncating.

Resolution

The development team updated all `Permit2` transfer calls to use `SafeCast.toUint160()` instead of direct `uint160()` casting, ensuring transactions revert rather than silently truncate when amounts exceed the `uint160` maximum value.

The fix can be found in the commit [e416926](#).

TRF-52	Use Of Deprecated <code>transfer()</code> Method		
Assets	contracts/GasTokenWrappingFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `unwrapGasToken()` function uses the deprecated `.transfer()` method to send ETH, which can cause transactions to fail for smart contract wallets and other contracts due to the 2300 gas limit.

The `.transfer()` method forwards only 2300 gas to the recipient, which is insufficient for many smart contract wallets (like Gnosis Safe, Argent, etc.) that need to execute logic in their fallback functions. This gas limit was originally intended as a reentrancy protection measure, but modern best practices recommend using `.call()` with proper reentrancy guards instead. When `.transfer()` fails due to gas limitations, legitimate users with smart contract wallets will be unable to unwrap their tokens, creating a poor user experience and potentially locking funds.

Recommendations

Replace `payable(msg.sender).transfer(amount)` with:

```
facets/GasTokenWrappingFacet.sol::unwrapGasToken()
(bool success, ) = payable(msg.sender).call{value: amount}("");
require(success, "ETH transfer failed");
```

This change maintains the same functionality while ensuring compatibility with all wallet types and following current Solidity best practices.

Resolution

The development team has correctly updated the ETH transfer method in the commit [72c6b49](#).

TRF-53	Missing Input Validation Enables DoS and Unexpected Behaviour		
Assets	contracts/TermFlashLoanCentralReceiverFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `TermFlashLoanCentralReceiverFacet.executeOperation()` function lacks comprehensive input validation for flash loan parameters, which can lead to denial-of-service attacks, gas griefing, and unexpected behaviour in target facets that assume valid parameters.

The function accepts flash loan parameters without validation, trusting that all inputs are well-formed and within expected bounds:

facets/flashloan/TermFlashLoanCentralReceiverFacet.sol::executeOperation()

```
function executeOperation(
    address[] calldata assets,          // No validation
    uint256[] calldata amounts,         // No validation
    uint256[] calldata premiums,        // No validation
    address initiator,                  // Validated via modifier
    bytes calldata data                 // No validation beyond decoding
) external override validateCallback(flashLoanAggregatorContract, initiator) returns (bool success) {
    // No parameter validation before processing (TermFlashLoanCallback memory callback, ) =
    //   abi.decode(data, (TermFlashLoanCallback, bytes));
    // ... proceeds to delegateCall with unvalidated data
}
```

Recommendations

Add the necessary parameter validation to the `executeOperation()` function, e.g.:

facets/flashloan/TermFlashLoanCentralReceiverFacet.sol

```
// Missing check - arrays could have mismatched lengths
if (assets.length != amounts.length || amounts.length != premiums.length) {
    ArrayLengthMismatch();
}

// Missing checks for zero addresses and invalid contracts
for (uint i = 0; i < assets.length; i++) {
    if (assets[i] == address(0)) revert InvalidAssetAddress();
}

// Missing checks for zero amounts and excessive values
for (uint i = 0; i < amounts.length; i++) {
    if (amounts[i] == 0) revert ZeroAmount();
    if (amounts[i] > MAX_FLASH_LOAN_AMOUNT) revert AmountTooLarge();
    if (premiums[i] > amounts[i]) revert PremiumExceedsAmount();
}

// Missing check for excessively large calldata that could cause DoS
if (data.length > MAX_CALLDATA_SIZE) revert CalldataTooLarge();
```

Resolution

The development team resolved this issue by implementing robust input validation throughout the `executeOperation()` function.

The fix includes array length validation to prevent mismatched parameters, zero address and zero amount checks, configurable calldata size limits to prevent DoS attacks, and premium bounds validation to catch configuration errors. These comprehensive validations eliminate the DoS and unexpected behaviour risks while preserving legitimate flash loan functionality.

The fix can be found in commits [802d5c0](#) and [0e1e377](#).

TRF-54	Pro-Rata Rounding Creates Permanently Stale Encumbered Collateral		
Assets	contracts/TermAuction.sol contracts/TermRepoServicer.sol contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Integer rounding in the auction's pro-rata marginal-fill path can assign a bid a purchase amount of zero, yet the code still calls `fulfillBid()` with the bid's full collateral basket. That journals collateral as encumbered while recording zero debt. When the borrower later withdraws, the encumbered amount is never cleared.

`_assignBids()` allocates partial fills across the marginal price group using a pro-rata ratio:

TermAuction.sol::_assignBids()

```
// @audit pro-rata partial assignment
uint256 bidAmount = sortedBids[i - innerIndex].amount;
Exp memory partialAssignmentRatio = div_(
  Exp({
    mantissa: (maxAssignable - totalAssignedBids) *
      10 ** (18 - purchaseTokenDecimals)
  }),
  Exp({
    mantissa: priceGroupAmount *
      10 ** (18 - purchaseTokenDecimals)
  })
);
uint256 assignedAmount = mul_(
  partialAssignmentRatio,
  Exp({
    mantissa: bidAmount *
      10 ** (18 - purchaseTokenDecimals)
  })
).mantissa / 10 ** (18 - purchaseTokenDecimals); // @audit rounds to 0 for small bids in large price groups

totalAssignedBids += _partiallyAssignBid(
  sortedBids[i - innerIndex],
  assignedAmount // @audit can be 0
);
```

With a 6-decimal token such as USDC, a small bid in a large marginal price group can produce `assignedAmount = 0`. `_partiallyAssignBid()` has no zero-amount guard and still forwards the bid's full collateral arrays:

TermAuction.sol::_partiallyAssignBid()

```
function _partiallyAssignBid(
    TermAuctionRevealedBid memory bid,
    uint256 assignedAmount
) internal nonReentrant returns (uint256) {
    uint256 repurchaseAmount = _calculateRepurchasePrice(assignedAmount);
    if (!bid.isRollover) {
        termRepoServicer.fulfillBid(
            bid.bidder,
            assignedAmount,          // @audit 0
            repurchaseAmount,         // @audit 0
            bid.collateralTokens,     // @audit full collateral basket
            bid.collateralAmounts,
            dayCountFractionMantissa
        );
    }
    // ... snip ...
}
```

fulfillBid() calls journalBidCollateralToCollateralManager() first, incrementing both lockedCollateralLedger and encumberedCollateralBalances by the full collateral amounts, and then adds repurchaseExposureLedger[bidder] += 0. The borrower now has encumbered collateral with zero debt.

When the borrower withdraws via externalUnlockCollateral(), _externalUnlockCollateralInternal() checks the obligation to decide whether to decrement encumberedCollateralBalances:

TermRepoCollateralManager.sol::_externalUnlockCollateralInternal()

```
if (termRepoServicer.getBorrowerRepurchaseObligation(borrower) != 0) {
    decrementEncumberedCollateral = true; // @audit skipped when obligation is 0
}
_unlockCollateral(borrower, collateralToken, amount, decrementEncumberedCollateral);
```

Because the obligation is zero, decrementEncumberedCollateral remains false. The collateral is returned but encumberedCollateralBalances retains the stale amount permanently. No path exists to clear it, since _unencumberRemainingBorrowerCollateralOnZeroObligation() operates on lockedCollateralLedger which is already zero after the unlock.

The downstream effect is in redeemTermRepoTokens(), which checks encumberedCollateralRemaining() before allowing pro-rata redemption. Even a dust-level stale entry causes EncumberedCollateralRemaining(), blocking pro-rata redemption for all lenders in that term. This only matters in shortfall scenarios, so the impact is medium. The likelihood is low because the trigger needs a pro-rata marginal assignment, a small enough bid to round to zero, and a later shortfall.

Recommendations

Skip zero-allocation bids in _assignBids() and treat them as unassigned instead of calling _partiallyAssignBid().

Resolution

The development team has resolved this issue in [PR#1636](#). The auction now reverts with AssignedZeroAmount when a pro-rata assignment rounds to zero, preventing the stale encumbered collateral state from ever being created. In the unlikely event this revert is triggered, the operational response is to call cancelAuction().

TRF-55	Rollover Approval Boundary Schedule Makes Rollover Assignment Impossible		
Assets	contracts/TermRepoRolloverManager.sol contracts/TermAuction.sol contracts/TermAuctionBidLocker.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When `approveRolloverAuction()` is called with a new auction whose end time equals the old term's repurchase window end, approval succeeds even though rollover assignment is impossible. By the first timestamp at which the auction can be completed, every rollover bid has already crossed the expiry boundary and is treated as expired.

`approveRolloverAuction()` validates the new auction's end time against the current term's repurchase window using a strict greater-than comparison:

TermRepoRolloverManager.sol::approveRolloverAuction()

```
if (
  auctionBidLocker.auctionEndTime() > // @audit strict >; passes when == endOfRepurchaseWindow
  termRepoServicer.endOfRepurchaseWindow()
) {
  revert AuctionEndsAfterRepayment();
}
```

The boundary case `auctionEndTime == endOfRepurchaseWindow` therefore passes without revert. However, two independent time guards create a deadlock at this value.

Auction completion requires `block.timestamp` to be strictly **greater than** `auctionEndTime`:

TermAuction.sol::onlyWhileAuctionClosed()

```
modifier onlyWhileAuctionClosed() {
  if (block.timestamp <= auctionEndTime) { // @audit requires block.timestamp > auctionEndTime
    revert AuctionNotClosed();
  }
  _;
}
```

Rollover bid validity during `_processRevealedBidsForValidity()` requires `block.timestamp` to be **not greater than** `endOfRepurchaseWindow`:

TermAuctionBidLocker.sol::_processRevealedBidsForValidity()

```
if (
  block.timestamp > pairOffServicer.endOfRepurchaseWindow() // @audit rollover bid is expired when block.timestamp >
  ↪ endOfRepurchaseWindow
) {
  revert RolloverBidExpired(revealedBid.id);
}
```

When `auctionEndTime == endOfRepurchaseWindow`, these two conditions are mutually exclusive. `completeAuction()` can only execute when `block.timestamp > auctionEndTime`, which equals `block.timestamp > endOfRepurchaseWindow`, the exact condition that marks all rollover bids as expired. Any rollover bid in the `revealedBids` array causes an immediate revert; they must instead be placed in `expiredRolloverBids`, where they are deleted without assignment. No rollover can ever be assigned.

The failure is silent: approval succeeded and rollover elections were accepted, but every rollover participant is treated as expired rather than assigned. By that point the repurchase window has ended, so borrowers cannot submit a replacement rollover election and face cash repayment or liquidation/default risk.

This issue has a medium impact because all rollover participants in the affected auction are silently denied assignment, leaving them with no rollover escape and forcing them towards cash repayment, liquidation, or default. This issue has a low likelihood because triggering this outcome requires an admin to configure the new auction's end time at precisely the repurchase window boundary, which is a plausible edge case when the intent is to maximise the rollover auction window.

Recommendations

Change the strict greater-than comparison in `approveRolloverAuction()` to greater-than-or-equal (`>=`) so that the boundary case `auctionEndTime == endOfRepurchaseWindow` is rejected at approval time.

Resolution

The recommendation has been implemented in [PR#1638](#).

TRF-56 Rollover Revocation Does Not Unwind Granted Roles			
Assets	contracts/TermRepoRolloverManager.sol contracts/TermRepoServicer.sol contracts/TermRepoCollateralManager.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Admin revocation of a rollover auction via `revokeRolloverApproval()` only clears the local allowlist flag without revoking the three privileged roles granted during approval. The revoked auction retains full authority to mark rollovers as processed, bricking affected borrowers.

`approveRolloverAuction()` performs four state changes: setting the allowlist flag, granting `ROLLOVER_TARGET_AUCTIONEER_ROLE` on `TermRepoServicer` and `TermRepoCollateralManager`, and granting `ROLLOVER_BID_FULFILLER_ROLE` on the rollover manager:

TermRepoRolloverManager.sol::approveRolloverAuction()

```
function approveRolloverAuction(
    ITermAuctionBidLocker auctionBidLocker
) external notPastRepaymentWindow onlyRole(ADMIN_ROLE) {
    // ... snip validation ...

    approvedRolloverAuctionBidLockers[address(auctionBidLocker)] = true;

    address termAuction = address(auctionBidLocker.termAuction());

    termRepoServicer.approveRolloverAuction(termAuction); // @audit - grants ROLLOVER_TARGET_AUCTIONEER_ROLE
    termRepoCollateralManager.approveRolloverAuction(termAuction); // @audit - grants ROLLOVER_TARGET_AUCTIONEER_ROLE
    _grantRole(ROLLOVER_BID_FULFILLER_ROLE, termAuction); // @audit - grants ROLLOVER_BID_FULFILLER_ROLE
    // ... snip ...
}
```

However, `revokeRolloverApproval()` only reverses the first of those four changes:

TermRepoRolloverManager.sol::revokeRolloverApproval()

```
function revokeRolloverApproval(
    ITermAuctionBidLocker auctionBidLocker
) external onlyRole(ADMIN_ROLE) {
    approvedRolloverAuctionBidLockers[address(auctionBidLocker)] = false; // @audit - clears allowlist only; three granted
    ↪ roles remain live

    emitter.emitRolloverTermApprovalRevoked(
        termRepoId,
        auctionBidLocker.termAuctionId()
    );
}
```

The allowlist flag only gates new `electRollover()` calls at `_electRolloverInternal()`:

TermRepoRolloverManager.sol::_electRolloverInternal()

```

if (
    !approvedRolloverAuctionBidLockers[
        termRepoRolloverElectionSubmission.rolloverAuctionBidLocker
    ]
) {
    revert RolloverAddressNotApproved(
        termRepoRolloverElectionSubmission.rolloverAuctionBidLocker
    );
}

```

But the revoked auction still holds `ROLLOVER_BID_FULFILLER_ROLE` and can call `fulfillRollover()` to set `processed = true`. The auction exercises this through `_markRolloverAsProcessed()`, reached via `completeAuction()`, `cancelAuction()`, and `cancelAuctionForWithdrawal()`:

TermAuction.sol::_markRolloverAsProcessed()

```

function _markRolloverAsProcessed(
    address rolloverPairOffTermRepoServicer,
    address borrower
) internal {
    ITermRepoServicer termRepoServicer_ = ITermRepoServicer(
        rolloverPairOffTermRepoServicer
    );
    ITermRepoRolloverManager rolloverManager = termRepoServicer_
        .termRepoRolloverManager();
    rolloverManager.fulfillRollover(borrower); // @audit - succeeds because ROLLOVER_BID_FULFILLER_ROLE was never revoked
}

```

If governance revokes a rollover approval as an emergency measure, because the target auction or term is deemed unsafe, the revocation has no practical effect on existing elections. The auction proceeds to completion, marks rollovers as processed, and permanently blocks affected borrowers from re-electing rollover, potentially forcing them into cash repayment or liquidation.

This issue has a medium impact because the intended emergency safety mechanism is ineffective, leaving borrowers exposed to the scenario revocation was meant to prevent. This issue has a low likelihood because it requires governance to invoke `revokeRolloverApproval()` and the revoked auction to subsequently complete.

Recommendations

Have `fulfillRollover()` check the allowlist flag before marking an election as processed, and silently skip (rather than revert) when the originating auction's bid locker is no longer approved. This prevents the revoked auction from bricking borrowers while allowing `completeAuction()` to finish without reverting. Directly revoking the granted roles is insufficient on its own because `_markRolloverAsProcessed()` is called inside `completeAuction()`, a role-check revert would brick the new auction's completion entirely.

Additionally, provide an admin-callable path that resets `rolloverElections[borrower].processed` for borrowers whose elections pointed at the revoked auction, so borrowers already marked processed before revocation are not permanently bricked.

Resolution

The development team has acknowledged the finding with the following comment:

"revokeRolloverApproval() is not intended to serve as the primary emergency response for an unsafe auction or term. The operational response would be to cancel the auction, delist the affected auction or term from

the subgraph, and call `unmarkTermDeployed()` so it is no longer treated as an active venue. `revokeRolloverApproval()` is used as an additional measure to block new rollover elections into that auction, not as the primary mechanism for neutralising an unsafe auction after users have already elected into it."

TRF-57	Strategy Facet Rejects Factory-Deployed Repos		
Assets	contracts/facets/TermStrategyFacet.sol contracts/factory/TermRepoDeployerFactory.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

`_validateRepoServicer()` only accepts servicers marked with `isTermDeployed()`. Repos deployed through `TermRepoDeployerFactory` are registered with `markTermFactoryDeployed()` instead, so they fail this check even though they are valid factory deployments. As a result, `mintAndSellRepoToken()` and `mintAndSellRepoTokenHook()` revert with `InvalidRepoId()` for factory-created repos.

`_registerWithController()` registers each deployed contract as factory-deployed:

factory/TermRepoDeployerFactory.sol::`_registerWithController()`

```
function _registerWithController(DeployedTermRepo memory deployed) internal {
    controller.markTermFactoryDeployed(deployed.termRepoServicer);
    controller.markTermFactoryDeployed(deployed.termRepoCollateralManager);
    controller.markTermFactoryDeployed(deployed.termRepoLocker);
    controller.markTermFactoryDeployed(deployed.termRepoToken);
    controller.markTermFactoryDeployed(deployed.rolloverManager);
}
```

`_validateRepoServicer()` checks only `isTermDeployed()`:

facets/TermStrategyFacet.sol::`_validateRepoServicer()`

```
function _validateRepoServicer(ITermRepoServicer servicer) private view {
    TermStorage storage s = LibTermStorage.termStorage();
    ITermController termController = servicer.termController();
    if (!s.approvedTermControllers[address(termController)]) {
        revert InvalidTermController();
    }
    if (!termController.isTermDeployed(address(servicer))) {
        revert InvalidRepoId();
    }
    if (block.timestamp > servicer.maturityTimestamp()) {
        revert AfterMaturity();
    }
}
```

The sister hook facet `TermLoanIntentHookFacet._validateRepoServicer()` checks both registration types:

facets/TermLoanIntentHookFacet.sol

```
if (
    !termController.isTermDeployed(address(servicer)) ||
    !termController.isFactoryDeployed(address(servicer))
) {
    revert InvalidRepoId();
}
```

This is an integration gap rather than a lasting protocol break. The controller admin can recover existing factory repos by calling `markTermDeployed()` on the relevant servicer, without redeploying. The impact is therefore limited to the strategy entry points that rely on this validation, and no funds are put at risk.

The likelihood is medium because any factory-created repo hits the revert until it is manually marked as term-deployed, but the issue has a straightforward administrative workaround.

Recommendations

Add an `isFactoryDeployed()` fallback to `_validateRepoServicer()`, mirroring `TermLoanIntentHookFacet._validateRepoServicer()`. Revert only when the servicer satisfies neither registration check.

Resolution

The recommendation has been implemented in [PR#1601](#).

The function `validateRepoServicer()` checks now if the servicer is `isFactoryDeployed()`.

TRF-58	TermFlashLoanExecutorFacet Requires Multi-Action Plans		
Assets	contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

Users cannot submit execution plans containing only a single action.

In `TermFlashLoanExecutorFacet`, a single-action plan is effectively unsupported due to conflicting validation requirements. On one hand, the first action should match the flashloaned token, and the final action should return the same token:

flashloan/TermFlashLoanExecutorFacet.sol

```
if (previewAction.expectedInputToken != flashExecuteRequest.flashLoanToken) {
    revert InvalidInputToken(0);
}

if (flashExecuteRequest.actions[flashExecuteRequest.actions.length - 1].outputToken !=
    ↪ flashExecuteRequest.flashLoanToken){
    revert FinalOutputTokenMismatch();
}
```

On the other hand, each action is explicitly forbidden from having identical input and output tokens:

flashloan/TermFlashLoanExecutorFacet.sol

```
if (executionPlan.actions[i].inputToken == executionPlan.actions[i].outputToken) {
    revert InputTokenMatchesOutputToken(i);
}
```

For a plan containing only one action, these conditions are incompatible: that action must both start and end with the flash loan token, while also being prohibited from using the same token as both input and output. As a result, successful execution implicitly requires more than one action.

This constraint is not documented or enforced at the protocol interface level, which can cause integrators to construct execution plans that appear valid but are guaranteed to revert.

Recommendations

If it is intended that the number of actions should be greater than one, make sure to document this case. If not, consider treating the one single action in the implementation separately.

Resolution

A comment indicating that the `Action` list of the struct `FlashExecuteRequest` should contain at least 2 actions has been added in [PR#1640](#). However, the function `flashExecute()` still does not have a proper error when the number of actions is equal to 1.

TRF-59 Cached EIP-712 Domain Separator Can Become Stale			
Assets	contracts/facets/TermLoanIntentFacet.sol contracts/facets/TermRepoTokenIntentFacet.sol contracts/facets/ERC4626InterfaceFacet.sol contracts/lib/Versionable.sol term-finance/package.json		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The intent-order and vault-approval signature system caches a single EIP-712 domain separator during initialisation and never recomputes it. If the chain ID changes or a future release alters the version string, old signatures remain valid under the stale domain while newly signed messages become unverifiable, breaking settlement and approval flows.

`TermLoanIntentFacet.initializeTermIntentFacet()` initialises `s.eip712DomainSeparator` exactly once, and `termIntentInitialized` prevents it from ever being refreshed:

TermLoanIntentFacet.sol::initializeTermIntentFacet()

```
function initializeTermIntentFacet(
    ITermEventEmitter emitter_
) external {
    TermStorage storage s = LibTermStorage.termStorage();

    if (s.termIntentInitialized){
        revert AlreadyInitialized();
    }

    uint256 chainId;
    assembly {
        chainId := chainid()
    }
    s.eip712DomainSeparator = keccak256(
        abi.encode(
            keccak256(
                "EIP712Domain("
                "string name,"
                "string version,"
                "uint256 chainId,"
                "address verifyingContract"
                ")"
            ),
            keccak256("TermFinance"),
            keccak256(bytes(version())),
            chainId,
            address(this)
        )
    );
    // @audit the cached separator is never recomputed after initialisation
}
```

All later signature checks reuse this cached value. This affects intent orders and ERC4626 vault approvals:

TermLoanIntentFacet.sol::_getEIP712Hash()

```
function _getEIP712Hash(bytes32 structHash) private view returns (bytes32) {
    TermStorage storage s = LibTermStorage.termStorage();
    return keccak256(abi.encodePacked(hex"1901", s.eip712DomainSeparator, structHash)); // @audit - reads stale cached
    ↪ separator; not recomputed from live chainId/version
}
```

TermRepoTokenIntentFacet.sol::_getEIP712Hash()

```
function _getEIP712Hash(bytes32 structHash) private view returns (bytes32) {
    TermStorage storage s = LibTermStorage.termStorage();
    return keccak256(abi.encodePacked(hex"1901", s.eip712DomainSeparator, structHash)); // @audit - reads stale cached
    ↪ separator; not recomputed from live chainId/version
}
```

ERC4626InterfaceFacet.sol

```
bytes32 digest = keccak256(abi.encodePacked(
    hex"1901",
    ts.eip712DomainSeparator, // @audit - reads stale cached separator; ERC4626 approvals affected
    structHash
));
```

Release builds explicitly rewrite the `Versionable.sol` string before compilation, making the cached `version()` sensitive to release changes:

package.json

```
"version:update": "shx sed -i \"s/development/${RELEASE_VERSION}/g\" contracts/lib/Versionable.sol",
"build:release": "shx sed -i \"s/development/${RELEASE_VERSION}/g\" contracts/lib/Versionable.sol && hardhat compile",
```

If an upgraded frontend starts using the new release version while the live diamond still verifies against the original cached separator, all new signatures fail. Similarly, after a chain ID change the contract continues verifying against the old chain ID, creating an inconsistent domain boundary exactly when EIP-712 domain separation is meant to protect against replay and misbinding.

This issue has a low impact because it requires an operational event outside normal usage, but when triggered it can invalidate legitimate offchain signatures or preserve signatures that should have been scoped to the old domain. This issue has a low likelihood because chain ID changes are rare and version drift requires a future release that changes the expected EIP-712 version without refreshing the cached separator onchain.

Recommendations

Recompute the EIP-712 domain separator from the live `block.chainid` and current version whenever a digest is built, or implement an explicit admin-controlled separator refresh that must run as part of any upgrade which changes the intended signing domain. The same domain-construction logic should be shared across intent settlement and vault-approval paths so offchain signers and onchain verification cannot drift.

Resolution

This issue has been addressed in [PR#1642](#) by adding a new admin-controlled function `updateEIP712DomainSeparator()` which recomputes the domain separator using the `block.chainid` and a `version`.

TRF-60	Caller-Sensitive vault.asset() Enables Undeclared Diamond Balance Spend		
Assets	contracts/facets/base/TermFlashHookFacet.sol contracts/facets/ERC4626InterfaceFacet.sol contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

depositToVaultHook() trusts the asset returned during preview, but preview and execution query vault.asset() from different caller contexts. A malicious ERC4626 vault can implement asset() so it returns different token addresses depending on msg.sender, causing the executor to validate one asset during preview and consume a different asset during execution.

previewDeposit() records the vault's live asset() response as the expected input token:

facets/ERC4626InterfaceFacet.sol::previewDeposit()

```
function previewDeposit(ActionHookInput calldata actionHookInput) external view returns (PreviewAction memory){
    IERC4626 vault = IERC4626(actionHookInput.targetAddress);
    return PreviewAction({
        expectedInputToken: vault.asset(),
        expectedInputAmount: vault.previewMint(actionHookInput.minOutputAmount),
        expectedOutputToken: address(vault),
        expectedOutputAmount: actionHookInput.minOutputAmount,
        isDeterministic: true
    });
}
```

Execution reads vault.asset() again from the diamond context:

facets/ERC4626InterfaceFacet.sol::_depositToVault()

```
function _depositToVault(
    address vault_,
    uint256 assets_,
    address user_,
    address receiver_
) internal returns (uint256) {
    IERC4626 vaultContract = IERC4626(vault_);
    IERC20 asset = IERC20(vaultContract.asset());
    asset.approve(vault_, assets_);
    uint256 shares = vaultContract.deposit(assets_, receiver_);
    asset.approve(vault_, 0);
    return shares;
}
```

The executor only compares balances for the previewed tokenIn and declared tokenOut. It does not re-check that the execution-time asset still matches the previewed asset:

facets/flashloan/TermFlashLoanExecutorFacet.sol

```

uint256 tokenInBalanceBefore = tokenInERC20.balanceOf(address(this));
uint256 tokenOutBalanceBefore = tokenOutERC20.balanceOf(address(this));

Address.functionCall(address(this), encodedCalldata);

uint256 tokenInBalanceAfter = tokenInERC20.balanceOf(address(this));
uint256 tokenOutBalanceAfter = tokenOutERC20.balanceOf(address(this));

uint256 actualOutputGained = tokenOutBalanceAfter - tokenOutBalanceBefore;
uint256 actualInputSpent = tokenInBalanceAfter < tokenInBalanceBefore ? tokenInBalanceBefore - tokenInBalanceAfter : 0;

```

If the previewed `tokenIn` is not consumed, this path does not revert. It simply reports `actualInputSpent = 0`, while `_depositToVault()` separately validates consumption of whatever asset the second `vault.asset()` call returned during execution.

An attacker can exploit this by:

1. Deploying a malicious ERC4626 vault whose `asset()` returns `primaryAsset` when called from the preview facet and `secondaryAsset` when called from the diamond during execution.
2. Approving that vault through `userApproveVault()` or using a controller-approved vault.
3. Submitting a flash plan with `depositToVaultHook()` against the vault. Preview records `primaryAsset` as the expected input token.
4. During execution, the vault consumes `secondaryAsset` from the diamond while the executor only measures `primaryAsset` and the vault share token.
5. If the diamond already holds `secondaryAsset`, the step can spend an undeclared token while still completing successfully.

The issue is low severity because it depends on the diamond holding a token balance, which is not part of the intended design.

Recommendations

Bind execution-time asset identity to the preview result. Cache the previewed asset in the hook calldata and revert if `vault.asset()` at execution does not match it.

Resolution

The development team has addressed this issue in [PR#1643](#) by checking in the function `depositToVaultHook()` that the previewed asset in the hook calldata matches with the `vault.asset()` as follows:

facets/ERC4626InterfaceFacet.sol::depositToVaultHook()

```

function depositToVaultHook(
    ActionHookInput calldata input
) external onlyFlashLoanContext(input.user) nonReentrant approvedERC4626VaultOnly(input.targetAddress, input.user) returns
    (uint256) {
    if (input.inputToken != IERC4626(input.targetAddress).asset()) // <-- new check {
        revert InvalidVaultAsset();
    }
    //... snip ...
}

```

TRF-61 Preview Hooks Hardcode Primary Collateral Index, Breaking Multi-Collateral Flash Plans			
Assets	contracts/facets/TermLoanIntentHookFacet.sol contracts/facets/TermRouterFacet.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Two preview functions hardcode `collateralTokens(0)` instead of respecting the caller-selected collateral token. In `TermLoanIntentHookFacet.previewSettleLimitLend()`, the hardcoded token is returned as `expectedInputToken`. In `TermRouterFacet.previewSubmitRepurchasePayment()`, the hardcoded token is returned as `expectedOutputToken`. For repos that accept more than one collateral token, any attempt to build a flash loan plan using a non-primary collateral index causes a revert, making those collateral slots unreachable through the flash loan pipeline.

`generateActionCalldata()` (in `TermFlashHookFacet`) rebuilds the `ActionHookInput` from the previewed tokens. A user who passes `collateralTokens(k)` for $k > 0$ as `inputToken` to `settleLimitLendHook()` gets `collateralTokens(0)` back from the preview, so execution reverts with `InputTokenMismatch`. Symmetrically, `submitRepurchasePaymentHook()` reverts with `OutputTokenMismatch`. The transaction fails outright rather than silently misusing the wrong collateral, but the functional consequence is the same: non-primary collateral is unreachable.

TermLoanIntentHookFacet.sol

```
function previewSettleLimitLend(
    ActionHookInput calldata actionHookInput
) external view returns (PreviewAction memory) {
    (, LimitLendOrder[] memory orders, , ) = abi.decode(
        actionHookInput.additionalCalldata,
        (bool, LimitLendOrder[], Signature[], uint256[])
    );
    if (orders.length == 0) {
        revert EmptyOrderBatch();
    }
    address termRepoServicer = orders[0].repoServicer;

    ITermRepoServicer _termRepoServicer = ITermRepoServicer(termRepoServicer);
    address purchaseToken = _termRepoServicer.purchaseToken();
    ITermRepoCollateralManager collateralManager = _termRepoServicer.termRepoCollateralManager();
    address collateralToken = collateralManager.collateralTokens(0); // @audit - always index 0; ignores
    ↪ actionHookInput.inputToken

    return PreviewAction({
        expectedInputToken: collateralToken, // @audit - user's chosen collateral overwritten
        expectedInputAmount: actionHookInput.maxInputAmount,
        expectedOutputToken: purchaseToken,
        expectedOutputAmount: actionHookInput.minOutputAmount,
        isDeterministic: true
    });
}
```

The same hardcoding pattern exists in `TermRouterFacet.previewSubmitRepurchasePayment()`, but there it corrupts the previewed output token rather than the input token:

TermRouterFacet.sol

```
function previewSubmitRepurchasePayment(
    ActionHookInput calldata actionHookInput
) external view returns (PreviewAction memory) {
    // ...
    address collateralToken = collateralManager.collateralTokens(0); // @audit - same hardcoded index 0
    // ...
}
```

`generateActionCalldata()` rebuilds the hook calldata from the preview output:

TermFlashHookFacet.sol

```
ActionHookInput memory previewActionHookInput = ActionHookInput({
    user: user,
    inputToken: previewAction.expectedInputToken,
    maxInputAmount: previewAction.expectedInputAmount,
    outputToken: previewAction.expectedOutputToken,
    // ... snip ...
});
bytes memory encodedCalldata = abi.encodeWithSelector(selector, previewActionHookInput); // @audit - calldata built with
↳ corrupted preview token; wrong collateral propagates to execution
```

The recent multi-order hook refactor did not change this root cause. Other preview functions correctly respect the caller's chosen token, for example, `previewMintAndSellRepoToken()` uses `actionHookInput.inputToken` directly, confirming the hardcoded index is an oversight rather than a design choice.

For single-collateral repos (the common case) this has no effect. For multi-collateral repos, users cannot select a non-primary collateral for use with `settleLimitLendHook()` or `submitRepurchasePaymentHook()` in flash loan plans.

This issue has a low impact because no fund loss occurs; only non-primary collateral selection in multi-collateral repos is affected. This issue has a low likelihood because it requires a multi-collateral repo configuration, which is uncommon in current deployments.

Recommendations

Replace the hardcoded token selection with the caller's chosen collateral token:

```
address collateralToken = actionHookInput.inputToken;
```

For `previewSubmitRepurchasePayment()`, the equivalent fix is:

```
address collateralToken = actionHookInput.outputToken;
```

Then validate that the chosen token is a supported collateral for the repo before returning the preview, and revert with `InvalidCollateralToken` if it is not.

Resolution

The recommendation has been implemented in [PR#1644](#).

TRF-62	Shared Salt In Deterministic Deployment Enables Address Griefing		
Assets	contracts/factory/TermDiamondFactory.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`deployDiamondDeterministic()` uses the same caller-supplied `salt` for both `CREATE2` deployments, `DiamondCutFacet` first and then `TermDiamond`. The salt is not namespaced by `msg.sender` or any other caller-specific value.

Because the function is permissionless and the salt is mempool-visible, a frontrunner can deploy `DiamondCutFacet` with the victim's salt before the victim's transaction executes, occupying the facet address and reverting the victim's deployment. The victim's pre-computed `TermDiamond` address, produced by `computeTermDiamondAddress()` for offchain pre-authorisation workflows, becomes permanently unreachable for that salt.

factory/TermDiamondFactory.sol::deployDiamondDeterministic()

```
function deployDiamondDeterministic(
    address devopsWallet,
    address adminWallet,
    bytes32 salt // @audit raw, unnamespaced, mempool-visible
) external returns (address diamond, address diamondCutFacet) {
    // Deploy DiamondCutFacet with CREATE2
    bytes memory bytecode = type(DiamondCutFacet).creationCode;
    assembly {
        diamondCutFacet := create2(0, add(bytecode, 0x20), mload(bytecode), salt)
        // @audit address depends only on factory + salt + DiamondCutFacet bytecode
        // anyone can deploy this ahead of the intended caller
    }
    require(diamondCutFacet != address(0), "DiamondCutFacet deployment failed");
    // @audit reverts here if frontrunner already burned the salt

    // Deploy TermDiamond with CREATE2 using the same salt
    bytes memory diamondBytecode = abi.encodePacked(
        type(TermDiamond).creationCode,
        abi.encode(devopsWallet, adminWallet, diamondCutFacet)
    );
    assembly {
        diamond := create2(0, add(diamondBytecode, 0x20), mload(diamondBytecode), salt)
    }
    // ... snip ...
}
```

The grieving sequence proceeds as follows:

1. Victim calls `computeTermDiamondAddress(devops, admin, facetAddr, salt_V)` offchain and pre-authorises the returned diamond address in a multisig or another contract.
2. Victim submits `deployDiamondDeterministic(devops, admin, salt_V)` to the network.
3. Attacker observes the pending transaction and submits `deployDiamondDeterministic(attacker_devops, attacker_admin, salt_V)` with higher priority.
4. Attacker's transaction executes first. `DiamondCutFacet` is deployed at `keccak256(0xff || factory || salt_V || keccak256(DiamondCutFacet.creationCode))`, the address the victim predicted for the facet.
5. Victim's transaction executes. The first `create2` call returns `address(0)` because the facet address is already occupied. `require(diamondCutFacet != address(0))` reverts.

6. The victim's diamond address is permanently unreachable under `salt_v`. Any pre-authorisation referencing that address is invalid.

The attack costs only gas. The victim must select a new salt and redo any offchain authorisation work. `computeTermDiamondAddress()` exists specifically to support pre-authorisation workflows, confirming this is an intended use case that the attack directly defeats.

This issue has a low impact because no funds are at risk and the harm is limited to friction and breakage of offchain workflows that pre-compute and pre-authorise diamond addresses. This issue has a low likelihood because the attack costs only gas with no financial gain, making it unattractive without a motivated griefing party.

Recommendations

Namespace the salt by `msg.sender` to make it caller-specific and non-squattable, e.g. `bytes32 namespacedSalt = keccak256(abi.encode(msg.sender, salt))`, and use the namespaced salt in both `create2` calls. Update `computeDiamondCutFacetAddress()` and `computeTermDiamondAddress()` to accept a `deployer` parameter and apply the same namespacing so pre-computation remains accurate.

Resolution

The development team has decided to not use `create2` and has removed the functions `deployDiamondDeterministic()`, `computeDiamondCutFacetAddress()` and `computeTermDiamondAddress()` in [PR#1639](#).

TRF-63 No Default Admin Role For Permission Management After Deployment	
Assets	contracts/TermRepoToken.sol
Status	Closed: See Resolution
Rating	Informational

Description

The `TermRepoToken` contract grants roles during deployment but never grants the `DEFAULT_ADMIN_ROLE` or sets admin roles:

```
_grantRole(ADMIN_ROLE, adminWallet_);  
_grantRole(MINTER_ROLE, termRepoServicer_);  
_grantRole(BURNER_ROLE, termRepoServicer_);  
_grantRole(DEVOPS_ROLE, devopsMultisig_);
```

Without admin roles, no account can grant/revoke roles after deployment. If any role account is compromised or lost, the only recovery is complete contract redeployment.

Note, a single admin role does also present a single point of failure and multiple admin roles do increase complexity.

Recommendations

Consider creating admin roles:

```
_setRoleAdmin(MINTER_ROLE, MINTER_ADMIN_ROLE);  
_setRoleAdmin(BURNER_ROLE, BURNER_ADMIN_ROLE);
```

Alternatively, ensure that the recovery procedure is well known, established and protected in the multisig contract.

Resolution

The development team has advised that they decided not to implement the OFT feature and, as such, the finding is now closed.

TRF-64 Generic Custom Error Handling Obscures Underlying Revert Reason		
Assets	contracts/MulticallFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The diamond architecture's `delegatecall` mechanism collapses custom errors from facet functions into generic string messages, obscuring the original error selector and encoded arguments. When facet functions revert with custom errors containing specific data, the diamond's error handling converts these structured errors into basic string representations, eliminating valuable debugging information for developers, monitoring systems, and end users.

```
facets/MulticallFacet.sol::_getRevertMsg()
function _getRevertMsg(bytes memory returnedData) private view returns (string memory) {
    // ...
    // Handle custom errors or other cases
    return "multicall: subcall failed with custom error"; // @audit
}
```

Recommendations

Return or propagate the raw revert data for custom errors instead of a generic string.

Resolution

The development team has replaced `_getRevertMsg()` with `_revert()`. This allows for more concise error messages and easier debugging. This change preserves the return data instead of overriding it with a generic message.

The fix can be found in the following commit: [be0b5ad](#).

TRF-65	Missing Duplicate Selector Check With Misleading Replace Revert	
Assets	contracts/LibDiamond.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The diamond cut mechanism does not explicitly detect duplicate function selectors within a single replace operation, resulting in misleading revert messages that complicate debugging.

When developers accidentally include duplicate selectors in `replaceFunctions()`, the system fails with generic error messages rather than clearly indicating the root cause of the duplicate selector conflict.

libraries/LibDiamond.sol::replaceFunctions()

```
for (uint256 selectorIndex; selectorIndex < _functionSelectors.length; selectorIndex++) {
    bytes4 selector = _functionSelectors[selectorIndex];
    address oldFacetAddress = ds.selectorToFacetAndPosition[selector].facetAddress;
    require(oldFacetAddress != _facetAddress, "LibDiamondCut: Can't replace function with same function"); // @audit
    ↪ misleading if duplicates supplied
    // ... no duplicate detection across _functionSelectors ...
}
```

Recommendations

Add a local set to detect duplicates within `_functionSelectors` and revert with a clear message to distinguish “same facet” reverts from “duplicate selector supplied”.

Resolution

The development team has added a for loop to separate the error logic inside the `replaceFunctions()` function. This allows for more concise error messages and easier debugging.

The fix can be found in the following commit: [3cf9f05](#).

TRF-66	Diamond Lacks Pausing Mechanism	
Assets	contracts/LibDiamond.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The diamond core library does not implement a pause mechanism, creating an operational inconsistency with other protocol contracts that expose pause controls. This absence removes the possibility of a global circuit breaker at the diamond level, requiring administrators to pause individual facets rather than having a unified emergency stop mechanism across the entire diamond proxy.

Recommendations

Consider adding a diamond-level pause mechanism in the core library.

Resolution

The development team has added pause and unpause functions to the `DiamondCutFacet.sol` to allow for global pausing ability.

The fixes were implemented in:

- [f89fc36](#).
- [ce86ae5](#).

TRF-67	Owner Not Externally Queryable Via Loupe	
Assets	contracts/LibDiamond.sol contracts/DiamondLoupeFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The diamond owner information is only accessible through an internal function `LibDiamond.contractOwner()`, with no external view function exposed by any facet to query ownership onchain. This creates a transparency gap where users, tools, and integrators cannot programmatically determine the diamond's owner, which is standard practice for most ownable contracts and expected by many external systems.

libraries/LibDiamond.sol::contractOwner()

```
function contractOwner() internal view returns (address contractOwner_) {  
    contractOwner_ = diamondStorage().contractOwner;  
}
```

Recommendations

Implement an external getter function querying the storage to return the diamond contract owner.

Resolution

The development team has removed the use of a `contractOwner` variable in the contract. The decision was made to implement OpenZeppelin's `AccessControl` library for a role based system. The fix has been implemented in [PR#1476](#).

The development team has provided the following response:

"This aligns with our contract admin permissioning pattern on our protocol contracts, where we give the aforementioned role to an ownerless Gnosis Safe that is under our governance system. There is no direct queryable function to read the DEVOPS wallet. However, users can use `hasRole` to confirm who it is and off chain can also read the event emitted when the role is granted in constructor."

TRF-68	Add Only Cuts Can Execute Unintended Initialisers Via <code>_init</code>	
Assets	contracts/LibDiamond.sol contracts/DiamondCutFacet.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `diamondCut()` function conflates two distinct operational concerns by allowing administrators to simultaneously manage function selectors and execute arbitrary initialisation logic through a `delegatecall` mechanism.

When an administrator intends to perform routine function selector updates, they must provide `_init` and `_calldata` parameters that, if misconfigured, can trigger unintended state mutations within the diamond's storage context.

This design creates an operational hazard where routine maintenance operations carry the risk of storage corruption, access control bypass, or business logic alteration, potentially leading to fund loss or permanent contract impairment.

The function unconditionally accepts `_init` and `_calldata` parameters alongside the function cuts:

facets/DiamondCutFacet.sol::diamondCut()

```
function diamondCut(FacetCut[] calldata _diamondCut, address _init, bytes calldata _calldata) external {
    LibDiamond.enforceIsContractOwner();
    LibDiamond.diamondCut(_diamondCut, _init, _calldata);
}
```

Within `LibDiamond.sol`, any non-zero `_init` address triggers a `delegatecall` in the diamond's storage context:

libraries/LibDiamond.sol::initializeDiamondCut()

```
function initializeDiamondCut(address _init, bytes memory _calldata) internal {
    if (_init == address(0)) { return; }
    enforceHasContractCode(_init, "... no code");
    (bool success, bytes memory error) = _init.delegatecall(_calldata);
    // ... bubble revert
}
```

This architectural decision creates a significant operational hazard where routine function management operations carry the risk of unintended state modifications.

An administrator performing routine function selector updates could inadvertently cause severe state corruption through misconfiguration of the `_init` parameter. Potential consequences include:

- **Storage Corruption:** Critical diamond storage variables could be overwritten, compromising contract integrity.
- **Access Control Bypass:** Administrative roles and permissions could be modified unintentionally.
- **Business Logic Alteration:** Core protocol state variables could be corrupted, affecting operational logic.
- **Fund Loss Risk:** State mutations could impact financial calculations or token balances.
- **Operational Disruption:** Contract functionality could become permanently impaired.

This issue is significantly mitigated by the requirement for the transaction in question to be executed by an administrator, and for the incorrectly submitted parameters to specifically affect stored data.

Recommendations

Consider implementing separate functions to decouple function management from initialisation logic, making administrative intent explicit.

Resolution

The development team has acknowledged the risks associated with the double initialisation of `_init` and `_calldata` and has provided the following response.

"We want to keep the EIP-2535 function exactly as it is. diamondCut will be secured to a DEVOPS_ROLE administrator (see TRF-05). There will also be initialisation booleans in the storage state for each initialisation function to prevent double initialisations from occurring."

TRF-69 No Ownership Transfer Function In Diamond	
Assets	contracts/LibDiamond.sol contracts/TermDiamond.sol
Status	Closed: See Resolution
Rating	Informational

Description

The diamond owner is only set during deployment and cannot be changed thereafter, as `LibDiamond.setContractOwner()` is internal and no external ownership transfer function exists. This prevents key rotation and creates operational security risks if the owner's private key is compromised or if ownership needs to be transferred for governance or operational reasons.

The diamond ownership is established in the constructor and stored in diamond storage:

TermDiamond.sol::constructor()

```
constructor(address _contractOwner, address _diamondCutFacet) payable {
    LibDiamond.setContractOwner(_contractOwner);
    // ...
}
```

However, `LibDiamond.setContractOwner` is marked as internal:

libraries/LibDiamond.sol::setContractOwner()

```
function setContractOwner(address _newOwner) internal {
    DiamondStorage storage ds = diamondStorage();
    address previousOwner = ds.contractOwner;
    ds.contractOwner = _newOwner;
    emit OwnershipTransferred(previousOwner, _newOwner);
}
```

No facet exposes an external function to change ownership, making the owner address permanently fixed after deployment.

The immutable ownership creates several operational and security concerns:

- **Key Compromise Risk:** If the owner's private key is compromised, there is no mechanism to transfer ownership to a secure address.
- **Operational Inflexibility:** Cannot transfer ownership for legitimate governance changes, multisig upgrades, or organisational transitions.
- **Single Point of Failure:** The protocol becomes permanently dependent on a single address for critical administrative functions.
- **Recovery Impossibility:** Loss of owner private key results in permanent loss of administrative control over the diamond.

Recommendations

Implement an owner management function externally.

Resolution

The development team has acknowledged the finding and provided the following response:

"To align with our contract admin permissioning pattern on our protocol contracts, we give the DEVOPS_ROLE to the ownerless Gnosis Safe from our governance system. Therefore there will be no transfers of ownership."

TRF-70	Misleading NatSpec In TermRouterFacet	
Assets	contracts/TermRouterFacet.sol contracts/TermRepoServicer.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `submitRepurchasePayment()` function in `TermRouterFacet` contains NatSpec documentation stating it enables users to “submit repurchase payment on behalf of a borrower”, but the implementation only supports self payment by forwarding `msg.sender` to the servicer.

This creates a contradiction between documented functionality and actual behaviour, potentially misleading integrators and users about the function's capabilities.

The function documentation promises functionality on behalf of borrowers:

facets/TermRouterFacet.sol::submitRepurchasePayment()

```

/// @notice Submit repurchase payment on behalf of a borrower << @audit Documentation claims on behalf of
/// @param termRepoServicer The address of the TermRepoServicer contract
/// @param amount The amount of purchase token to submit for repurchase
function submitRepurchasePayment(
    address termRepoServicer,
    uint256 amount
) external nonReentrant {
    // ...
    _termRepoServicer.submitRepurchasePayment(msg.sender, amount); // @audit Routes as self, not on behalf of borrower
}

```

The underlying servicer supports both self-payment and payment on behalf of others:

facets/TermRepoServicer.sol::submitRepurchasePayment()

```

// TermRepoServicer interfaces
function submitRepurchasePayment(uint256 amount) external; // @audit Self-payment
function submitRepurchasePayment(address borrower, uint256 amount) external onlyRole(SETTLER_ROLE); // @audit On behalf of
    ↪ borrower

```

However, the router implementation hardcodes `msg.sender` as the borrower, making it functionally identical to the self-payment variant whilst claiming to support repayment on behalf of borrowers.

Recommendations

Either implement the documented functionality to repay on behalf of borrowers or update the documentation to reflect the functionality which has been implemented.

Resolution

The development team has acknowledged the finding and provided the following response:

"The borrower invokes submitRepurchasePayment on TermDiamond, and TermDiamond then calls the servicer contract using delegatecall, forwarding the borrower as msg.sender. Because the servicer is being called in the borrower's context rather than directly by a Term protocol contract, this is not a case where SETTLER_ROLE – which protects functions on Term protocol contracts intended to be called only by TermDiamond—applies."

TRF-71	Borrowers' Excess Collateral Remains Locked On Partial Bid Fills	
Assets	contracts/TermAuction.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

Note: Although this finding is outside the defined scope of this engagement, it was discovered during our testing and is included in this report for completeness.

When partially assigning bids during auction completion, the implementation passes the bidder's full collateral arrays to the fulfilment function, causing all pledged collateral to remain locked regardless of the assignment ratio. This contrasts with the offer assignment logic, which explicitly unlocks the unassigned portion immediately, creating an inconsistency that unnecessarily locks borrower capital and reduces protocol capital efficiency.

TermAuction.sol::_partiallyAssignBid()

```
function _partiallyAssignBid(TermAuctionRevealedBid memory bid, uint256 assignedAmount)
    internal nonReentrant returns (uint256)
{
    uint256 repurchaseAmount = _calculateRepurchasePrice(assignedAmount);

    if (!bid.isRollover) {
        termRepoServicer.fulfillBid(
            bid.bidder,
            assignedAmount,
            repurchaseAmount,
            bid.collateralTokens,
            bid.collateralAmounts, // @audit full collateral locked despite partial assignment
            dayCountFractionMantissa
        );
    }
    // ...
    return assignedAmount;
}
```

In contrast, partial offer assignment immediately unlocks the unassigned portion:

TermAuction.sol::_partiallyAssignOffer()

```
function _partiallyAssignOffer(TermAuctionRevealedOffer memory offer, uint256 assignedAmount)
    internal nonReentrant returns (uint256)
{
    uint256 repurchaseAmount = _calculateRepurchasePrice(assignedAmount);
    termRepoServicer.fulfillOffer(offer.offeror, assignedAmount, repurchaseAmount, offer.id);

    // Unlock remaining.
    termAuctionOfferLocker.unlockOfferPartial(
        offer.id,
        offer.offeror,
        offer.amount - assignedAmount // @audit partial
    );
    return assignedAmount;
}
```

This asymmetry means borrowers have their full collateral locked even when only partially filled, whilst lenders receive immediate access to their unassigned funds.

Recommendations

Pass proportional collateral amounts alongside `assignedAmount` when fulfilling a partial bid, and immediately unlock the unused collateral.

Resolution

The development team has determined that the current behaviour is intentional and will remain unchanged. The decision to keep all pledged collateral locked during partial bid fills serves several important purposes:

- **Liquidation Risk Management:** Automatically unlocking proportional collateral could immediately push borrowers below the minimum collateralization threshold, triggering unwanted liquidations
- **Gas Optimization:** Avoiding complex collateral calculations and unlock operations during auction completion reduces gas costs for all participants
- **Borrower Control:** Users retain full control over their collateral management strategy and can manually unlock excess collateral when appropriate for their risk profile

This approach prioritizes borrower safety by preventing automatic actions that could compromise their collateralization position, while allowing sophisticated users to optimize their capital efficiency through manual collateral management post-auction.

The asymmetry with offer handling is justified because lenders (offer makers) face different risk profiles compared to borrowers (bid makers) who must maintain adequate collateralization ratios throughout their loan terms.

The development team considers the current behaviour to be by design, based on risk management and user experience considerations.

TRF-72 Potential Overlapping Offer Reveal And Borrower Bid Commit Times	
Assets	contracts/TermAuctionBidLocker.sol contracts/TermAuctionOfferLocker.sol
Status	Closed: See Resolution
Rating	Informational

Description

Note: Although this finding is outside the defined scope of this engagement, it was discovered during our testing and is included in this report for completeness.

The auction mechanism allows timing configurations where reveal windows overlap with commit windows for the opposite side of the market, enabling participants to observe revealed pricing information before finalising their own commitments. If the offer reveal window overlaps with the bid commit window, bidders can observe revealed offer prices before committing their bids; conversely, if the bid reveal window overlaps with the offer commit window, offerors can observe revealed bid prices before committing offers. This timing overlap fundamentally undermines the sealed-bid auction model by leaking critical pricing intelligence.

Both lockers initialise with the same three timing parameters but do not enforce non-overlap across sides:

TermAuctionBidLocker.sol::initialize()

```
function initialize( .. ) external initializer {
    UUPSUpgradeable.__UUPSUpgradeable_init();
    AccessControlUpgradeable.__AccessControl_init();
    ReentrancyGuardUpgradeable.__ReentrancyGuard_init();

    _grantRole(INITIALIZER_ROLE, termInitializer_);

    termRepoId = keccak256(abi.encodePacked(termRepoId_));
    termAuctionId = keccak256(abi.encodePacked(auctionId_));

    if (auctionStartTime_ > revealTime_) { // @audit only validation check on time parameters
        revert AuctionStartsAfterReveal(auctionStartTime_, revealTime_);
    }

    auctionStartTime = auctionStartTime_;
    revealTime = revealTime_;
    auctionEndTime = auctionEndTime_;
    minimumTenderAmount = minimumTenderAmount_;
    dayCountFractionMantissa =
        ((redemptionTimestamp_ - auctionEndTime_) * expScale) /
        THREESIXTY_DAYCOUNT_SECONDS;
    purchaseToken = purchaseToken_;
    for (uint8 i = 0; i < collateralTokens_.length; ++i) {
        collateralTokens[collateralTokens_[i]] = true;
    }

    bidCount = 0;
    termContractPaired = false;
}
```

The only validation check is that the commit starts before the reveal. There is a similar check in `TermAuctionOfferLocker`.

If bid and offer lockers are initialised with different `auctionStartTime` or `revealTime` values, the reveal window of one side can overlap the commit window of the other, enabling strategic timing exploitation.

Recommendations

Consider enforcing, at initialisation, that:

- The offer reveal time must not overlap the borrower bid commit time.
- The bid reveal time must not overlap the lender offer commit time.

Add explicit validation in initialisation to reject overlapping schedules and include tests to ensure the constraints do not overlap.

Resolution

The development team has acknowledged the finding but have advised that the scheduling overlaps are taken care of in the deploy script which was out of scope.

The development team has provided the following response:

"Overlapping schedules right now are prevented in our deploy script but not on the smart contract."

TRF-73	Bid Slot Exhaustion Enables Single Borrower Manipulation And Auction Cancellation	
Assets	contracts/TermAuctionBidLocker.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

Note: Although this finding is outside the defined scope of this engagement, it was discovered during our testing and is included in this report for completeness.

An adversary can exploit the auction's bid limit mechanism by submitting `MAX_BID_COUNT` minimum sized bids, then strategically revealing only selected bids or none at all to manipulate auction outcomes. Since unrevealed bids are removed without penalty during auction processing, this enables two primary attack vectors: monopolising borrower access by becoming the sole revealed bidder, or forcing auction cancellation by revealing no bids whilst blocking legitimate participants from submitting.

The attack exploits several auction mechanism characteristics:

- Bid Slot Limitation: The auction enforces `MAX_BID_COUNT` to limit total bid count.
- No Revelation Penalty: Unrevealed bids are simply discarded without cost to the bidder.
- Minimum Bid Requirements: An adversary can submit many small bids with minimal capital commitment.

Attack Scenario 1: Monopolisation

1. An adversary submits `MAX_BID_COUNT` minimum sized bids, blocking legitimate bidders.
2. Legitimate bidders cannot participate due to slot exhaustion.
3. The adversary reveals only one strategic bid, becoming the sole borrower.
4. The adversary obtains financing at potentially favourable rates due to lack of competition.

Attack Scenario 2: Auction Cancellation

1. An adversary submits `MAX_BID_COUNT` minimum-sized bids.
2. Legitimate bidders cannot participate due to slot exhaustion.
3. The adversary reveals no bids during the reveal phase.
4. The auction fails due to the absence of valid bids, wasting lender time and gas.

The adversary's unrevealed bids are simply removed during processing without financial penalty, making this a low cost grieving mechanism.

The attack requires minimal capital (just enough for minimum bid sizes) but can significantly impact auction outcomes.

Recommendations

Add a pre-reveal blackout for unlocks (e.g. `require(block.timestamp < revealTime - unlockBuffer)`).

Introduce a refundable deposit and slash a percentage for non-revealed bids.

Resolution

The development team has acknowledged the finding and provided the following response:

"Agree that the price caps don't directly mitigate the DoS issue. But it does make it very expensive to exploit as we continue to up the minimum tender. As admins, Term also has the ability to cancel auctions so in the worst case we can cancel the auction that is being spammed."

TRF-74	Offer Slot Exhaustion Enables Single Lender Manipulation	
Assets	contracts/TermAuctionOfferLocker.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

Note: Although this finding is outside the defined scope of this engagement, it was discovered during our testing and is included in this report for completeness.

An adversary can monopolise the auction by filling `MAX_OFFER_COUNT` with minimum size offers, then either reveal only one offer to become the sole lender, or reveal none to force cancellation. Unrevealed offers are removed without penalty during `getAllOffers()`, enabling griefing and price manipulation.

facets/TermAuctionOfferLocker.sol

```
uint256 public constant MAX_OFFER_COUNT = 150;
uint256 public offerCount;
```

facets/TermAuctionOfferLocker.sol::_lock()

```
function _lock(TermAuctionOfferSubmission memory offerSubmission, address authenticatedUser)
    internal
    onlyOfferor(offerSubmission.offeror, authenticatedUser)
    returns (TermAuctionOffer storage)
{
    if (offerCount >= MAX_OFFER_COUNT) { // @audit fills global slots; blocks others
        revert MaxOfferCountReached();
    }
    // ... stores offer and increments offerCount ...
}
```

facets/TermAuctionOfferLocker.sol::getAllOffers()

```
uint256 k = 0;
for (uint256 i = 0; i < unrevealedOffers.length; ++i) {
    TermAuctionOffer storage unrevealedOffer = offers[unrevealedOffers[i]];
    if (unrevealedOffer.amount == 0) revert NonExistentOffer(unrevealedOffers[i]);
    if (unrevealedOffer.isRevealed) revert OfferRevealed(unrevealedOffers[i]);

    unrevealed[k] = unrevealedOffer;
    ++k;
    _processOfferForAuction(unrevealedOffer.id); // @audit deletes offer without penalty, enabling griefing
}
```

- Vector A: Submit 150 offers, reveal only one. The 149 unrevealed offers are deleted during processing, leaving the adversary as the sole lender and able to manipulate clearing.
- Vector B: Submit 150 offers, reveal none. All are deleted and the auction cancels, producing a DOS at low cost.

Recommendations

Introduce an economic penalty for unrevealed offers (e.g. stake a small deposit at lock time; slash on non-reveal; refund on reveal).

Resolution

The development team has acknowledged the finding and provided the following response:

"Agree that the price caps don't directly mitigate the DoS issue. But it does make it very expensive to exploit as we continue to up the minimum tender. As admins, Term also has the ability to cancel auctions so in the worst case we can cancel the auction that is being spammed."

TRF-75 Weak Commit Hash Allows Price Guessing With Low Entropy Nonces	
Assets	contracts/TermAuctionOfferLocker.sol contracts/TermAuctionBidLocker.sol
Status	Closed: See Resolution
Rating	Informational

Description

Note: Although this finding is outside the defined scope of this engagement, it was discovered during our testing and is included in this report for completeness.

Offers and bids commit as `keccak256(abi.encode(price, nonce))`, relying entirely on a user chosen nonce for secrecy. Low entropy or reused nonces enable offline guessing of prices (small search space) and practical pre-computation across repeated auctions.

TermAuctionOfferLocker.sol::_revealOffer()

```
if (keccak256(abi.encode(price, nonce)) != offers[id].offerPriceHash) { // @audit weak domain separation and nonce reliance
    revert OfferPriceModified();
}
```

A similar pattern exists in `_revealBid()`. Domain separation is absent (no binding to offeror/bidder, auction id, or contract), making rainbow tables more feasible across auctions if users pick predictable nonces.

Recommendations

Document the risk for users and ensure that there is a clear warning of the need to pick fresh, cryptographically secure nonces for each submission.

Resolution

The development team has stated this issue is prevented by implementing cryptographically secure nonce generation in their user interface. Rather than relying on users to manually generate secure nonces, the UI automatically generates randomized nonces using the browser's cryptographically secure random number generator.

This approach ensures that:

- Nonces are generated with high entropy (256 bits of randomness)
- Users cannot accidentally choose weak or predictable nonces
- Each auction submission receives a cryptographically secure, unique nonce
- The risk of price guessing through brute force or rainbow table attacks is effectively eliminated

TRF-76	Code Quality Issues In BSC Flashloan Implementation	
Assets	flashloan-aggregator/contracts/aggregator/bsc/flashloan/main.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The BSC flashloan implementation contains development artifacts and syntax errors that should be cleaned up. Two specific issues were identified:

- **Unused Development Import** line [6] imports `hardhat/console.sol` but never uses console functions, creating unnecessary dependency and deployment bloat.
- **Duplicated Function Call** line [241] contains duplicated function parameters `routePancakeSwap(_tokens, _amounts, _data, _instadata);(_tokens, _amounts, _data, _instadata);` which appears to be a copy-paste error.

Recommendations

Remove unused development imports and fix syntax errors:

- Delete `import "hardhat/console.sol";` from line [6]
- Fix line [241] to single function call: `routePancakeSwap(_tokens, _amounts, _data, _instadata);`

Resolution

The finding has been resolved - the unused import and duplicate code were removed in the commit [6937f75](#).

TRF-77 Protocol Route Numbering System Issues	
Assets	flashloan-aggregator/contracts/aggregator/mainnet/flashloan/main.sol flashloan-aggregator/contracts/aggregator/arbitrum/flashloan/main.sol flashloan-aggregator/contracts/aggregator/avalanche/flashloan/main.sol flashloan-aggregator/contracts/aggregator/base/flashloan/main.sol flashloan-aggregator/contracts/aggregator/bsc/flashloan/main.sol flashloan-aggregator/contracts/aggregator/plasma/flashloan/main.sol flashloan-aggregator/contracts/aggregator/sepolia/flashloan/main.sol
Status	Closed: See Resolution
Rating	Informational

Description

The flash loan aggregator uses inconsistent hardcoded route numbers across networks.

Only route 9 (AaveV3) is universal - other routes vary significantly (mainnet has 10 routes: 1-7, 9-11; arbitrum has 3: 5, 8, 9).

This creates developer confusion where route 5 means Balancer on mainnet/arbitrum but does not exist on other networks. Users cannot discover available protocols per network, requiring hardcoded mappings for each integration.

Recommendations

Replace magic route numbers with named protocol identifiers:

- Implement Protocol Registry contract per network
- Use descriptive names instead of route numbers.
- Provide dynamic protocol discovery API
- Maintain backward compatibility during migration

Resolution

The development team has acknowledged the finding with the following comment:

"We would like to keep the same enumerations that Instadapp is using before we forked their repo."

TRF-78	Incorrect Comments In TermMorphoInterfaceFacet.sol	
Assets	contracts/TermMorphoInterfaceFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `generateCalldata()` function in `TermMorphoInterfaceFacet` contains incorrect comments that were copy-pasted from `TermAaveInterfaceFacet`.

lines [318-324] have comments stating "Default to variable rate mode (2)" but Morpho protocol does not use Aave's rate mode concepts (stable vs variable rates).

```
facets/external/TermMorphoInterfaceFacet.sol::generateCalldata()
else if (selector == TermMorphoInterfaceFacet.morphoWithdrawCollateral.selector) {
    // Default to variable rate mode (2) <-- INCORRECT
    return abi.encodeWithSelector(selector, morphoPool, marketParams, amount, user, payoutUser);
} else if (selector == TermMorphoInterfaceFacet.morphoBorrow.selector) {
    // Default to variable rate mode (2) <-- INCORRECT
    return abi.encodeWithSelector(selector, morphoPool, marketParams, amount, user, payoutUser);
} else if (selector == TermMorphoInterfaceFacet.morphoRepay.selector) {
    // Default to variable rate mode (2) <-- INCORRECT
    return abi.encodeWithSelector(selector, morphoPool, marketParams, amount, user);
}
```

Recommendations

Remove the misleading comments.

Resolution

The finding has been resolved - the comments were removed in the commit [16dc5a4](#).

TRF-79	Unused Local Variable	
Assets	contracts/TermAaveInterfaceFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The local variable `baseCurrencyUnit` in the function `availableBorrow()` is never used.

```
facets/external/TermAaveInterfaceFacet.sol::availableBorrow()
```

```
// Get base currency unit (1e8 for USD, 1e18 for ETH)
uint256 baseCurrencyUnit = oracle.BASE_CURRENCY_UNIT();
```

Recommendations

Remove the variable as it is not needed.

Resolution

The variable was removed in the commit [fe0b263](#).

TRF-80	Documentation Claims Generic Gas Token Support But Code Is WETH-Specific	
Assets	contracts/GasTokenWrappingFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The facet documentation claims to support "*native gas tokens (e.g., ETH to WETH)*" and "*wrapped token facets (e.g., WETH)*" suggesting generic functionality, but the implementation hardcodes WETH-specific interface methods that may not exist on other wrapped gas tokens.

The facet's NatSpec comments and parameter names suggest it can work with any wrapped gas token across different chains (WMATIC on Polygon, WBNB on BSC, etc.), but the implementation specifically uses the IWETH interface with `deposit()` and `withdraw(uint256)` methods. While many wrapped gas tokens follow this same interface standard, this is not guaranteed for all chains or future tokens.

The mismatch between documentation and implementation could lead to integration errors where developers assume broader compatibility than actually exists, potentially causing failed transactions or deployment issues when used with non-WETH-compatible wrapped tokens. Additionally, the generic naming (`wrappedTokenAddr`, `GasTokenWrappingFacet`) reinforces the false impression of universal compatibility.

Recommendations

Choose one approach for consistency:

- Option 1 - Make it WETH-specific, e.g.:

```
facets/GasTokenWrappingFacet.sol::wrapETH()
/// @title WETHWrappingFacet
/// @notice Provides functionality to wrap and unwrap ETH to WETH
/// @param wethAddr The address of the WETH contract
function wrapETH(address wethAddr, bool payoutUser) external payable
```

- Option 2 - Add sufficient validation, e.g.:

```
facets/GasTokenWrappingFacet.sol::validWrappedToken()
/// @notice Validates that the wrapped token supports required interface
modifier validWrappedToken(address wrappedTokenAddr) {
    // Add interface validation here
    _;
}
```

Resolution

The development team renamed the `GasTokenWrappingFacet` facet to `WETHWrappingFacet`. This, combined with the updated function names and updated Natspec has provided a sufficient fix to this issue.

The fix can be found in the commit [63ce181](#).

TRF-81 Insufficient Validation In <code>permit</code> Function		
Assets	<code>contracts/PermitFacet.sol</code>	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The `permit()` function lacks proper input validation for critical parameters, allowing invalid signature components and zero addresses to be processed without verification.

The function in `PermitFacet.sol` does not validate essential parameters before making the external call to the token's `permit()` function. Specifically, it fails to verify that the asset address is non-zero, that the ECDSA signature components `r` and `s` are valid non-zero values, and that the recovery parameter `v` is within the expected range (27 or 28).

This lack of validation can result in wasted gas when invalid parameters are passed, and more importantly, may lead to unexpected behaviour if the underlying token contract does not properly validate these parameters itself.

Recommendations

Validate that the asset address is non-zero and that the `r`, `s` and `v` values are valid to ensure no wasted gas.

Resolution

The issue has been resolved in the commit [17077eb](#) by implementing validation checks on the `permit` function parameters.

TRF-82 Factory Does Not Explicitly Reject Fee-On-Transfer Tokens	
Assets	contracts/TermRepoLocker.sol contracts/TermRepoCollateralManager.sol contracts/TermRepoServicer.sol contracts/factory/TermRepoDeployerFactory.sol contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol
Status	Closed: See Resolution
Rating	Informational

Description

If a fee-on-transfer or other short-crediting token is admitted into a repo, Term will over-credit collateral, repayments, and flash top-ups against nominal requested amounts rather than actual received balances. The development team has indicated such tokens are unsupported, but the repo-factory path does not enforce that exclusion.

`TermRepoDeployerFactory.deployTermRepo()` is public and `_validateToken()` only checks zero address, `decimals()`, and oracle support:

TermRepoDeployerFactory.sol

```
function deployTermRepo(
    TermRepoDeployParams calldata deployParams
) external whileDeployingNotPaused returns (DeployedTermRepo memory deployed) {
    _validateDeployParams(deployParams);
    // ...
}

function _validateToken(
    address token
) internal view {
    if (token == address(0)) {
        revert TokenZeroAddress();
    }
    uint8 decimals;
    try IERC20Metadata(token).decimals() returns (uint8 d) {
        decimals = d;
    } catch {
        revert InvalidTokenDecimals();
    }
    if (decimals > 18) {
        revert InvalidTokenDecimals();
    }
    _validateTokenOraclePrice(token, decimals); // @audit no check for fee-on-transfer behaviour; oracle support alone is
    ↪ insufficient
}
```

This does not reject fee-on-transfer, burn-on-transfer, transfer-tax, or other short-crediting behaviour. Admission depends on deliberate protocol configuration: the factory must be paired and the token must have a configured price feed. However, once those steps are taken, a repo deployer can select the token because no further semantic check is applied during deployment.

The downstream accounting is incompatible with short-crediting transfers. `TermRepoLocker.transferTokenFromWallet()` forwards the requested amount without measuring the locker balance delta, and `TermRepoCollateralManager._lockCollateral()` credits the nominal amount into the collateral ledger:

TermRepoLocker.sol::transferTokenFromWallet()

```
function transferTokenFromWallet(
    address originWallet,
    address token,
    uint256 amount
) external override whileTransfersNotPaused whileTermContractsNotPaused(termRepoServicer.termController())
    ↪ onlyRole(SERVICER_ROLE) {
    IERC20 tokenInstance = IERC20(token);
    tokenInstance.safeTransferFrom(originWallet, address(this), amount); // @audit no balance-delta check; nominal amount
    ↪ assumed equal to received amount
}
```

TermRepoCollateralManager::_lockCollateral()

```
function _lockCollateral(
    address borrower,
    address sender,
    address collateralToken,
    uint256 amount
) internal {
    uint256 currentBalance = lockedCollateralLedger[borrower][collateralToken];
    if (termRepoServicer.getBorrowerRepurchaseObligation(borrower) == 0 && currentBalance > 0) {
        encumberedCollateralBalances[collateralToken] += currentBalance;
    }
    lockedCollateralLedger[borrower][collateralToken] += amount; // @audit nominal amount credited before actual receipt is
    ↪ verified
    encumberedCollateralBalances[collateralToken] += amount;
    termRepoLocker.transferTokenFromWallet(sender, collateralToken, amount);
}
```

The same nominal-accounting assumption appears in `TermRepoServicer._repay()` and `TermFlashLoanExecutorFacet.flashExecuteCallback()`, which book requested amounts without measuring the actual transfer-in delta. Regression tests confirm that a fee-on-transfer token can create phantom collateral and nominally complete repayment while delivering less value to custody.

Recommendations

Add a deployment-time token allowlist or configuration flag that explicitly excludes fee-on-transfer tokens.

Resolution

The development team has acknowledged the finding with the following comment:

“A mandatory validation step has been added to the token onboarding workflow, requiring manual verification that the token does not implement transfer fees or other non-standard transfer mechanics. This validation is a blocking requirement: tokens cannot progress through onboarding unless this check is completed and approved. Only tokens that successfully pass this process are added to the oracle feed, which is a prerequisite for being enabled within the Term protocol.”

TRF-83	Inefficient Zero-Value Approvals in Collateral Flow	
Assets	contracts/facets/TermStrategyFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

In the function `TermStrategyFacet.mintAndSellRepoTokenHook()`, the function `_buildCollateralAmountsArray()` constructs an array sized to the total number of accepted collateral tokens, which may be greater than one, but populates only a single index with a non-zero value while leaving all other entries as zero.

```
uint256[] memory collateralAmounts = new uint256[](numCollateralTokens);

bool collateralSupported;
for (uint256 i = 0; i < numCollateralTokens; i++) {
    if (collateralManager.collateralTokens(i) == collateralToken) {
        collateralAmounts[i] = collateralAmount;
        collateralSupported = true;
        break;
    }
}
```

This array is later passed to `_mintAndSellRepoTokenInternal()`, which processes the full array and may perform approval operations for zero-value entries.

```
// ... snip ...
address collateralToken;
uint256 amount;
uint8 index;

for (index = 0; index < collateralAmounts.length; ++index){
    collateralToken = termRepoCollateralManager.collateralTokens(index);
    amount = collateralAmounts[index];
    IERC20(collateralToken).approve(address(termRepoLocker), amount);
}
// ... snip ...
```

Because the number of accepted collateral tokens may be greater than one, the current implementation can issue unnecessary external calls for zero-value approvals on unused collateral assets.

Recommendations

In the function `_mintAndSellRepoTokenInternal()`, consider checking that the collateral amount is non-zero before approving to `TermRepoLocker`.

Resolution

The recommendation has been implemented in [PR#1628](#).

TRF-84	settleLimitLendHook() Omits nonReentrant, Diverging From The Hook Safety Pattern	
Assets	contracts/facets/TermLoanIntentHookFacet.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

`settleLimitLendHook()` carries no `nonReentrant` modifier despite being a flash-loan hook that makes external token calls after invoking another guarded entrypoint. This diverges from the hook safety pattern used elsewhere in the codebase and increases maintenance risk, even though no exploitable authority leak from this omission alone has been identified.

TermLoanIntentHookFacet.sol::settleLimitLendHook()

```
function settleLimitLendHook(
    ActionHookInput calldata input
) external onlyFlashLoanContext(input.user) {
    // @audit - still no nonReentrant after the multi-order refactor

    // ... snip ...
    for (uint256 i = 0; i < len; ++i) {
        TermLoanIntentFacet(address(this)).settleLimitLend(...);
        // @audit - nested self-call acquires + releases OZ lock per iteration
    }

    if (usePermit2) {
        Permit2Lib.PERMIT2.transferFrom(
            taker, address(this), totalBorrowAmount.toUint160(), purchaseToken
        );
        // @audit - external call after the nested nonReentrant calls have returned
    } else {
        IERC20(purchaseToken).safeTransferFrom(taker, address(this), totalBorrowAmount);
    }
}
```

This is a consistency and defence-in-depth issue:

- other hook entrypoints such as `mintAndSellRepoTokenHook()` do use `nonReentrant`,
- `settleLimitLendHook()` performs an external token transfer after returning from nested `settleLimitLend()` calls,
- future changes could make this gap security-relevant.

The remaining live context is the current flash-loan borrower's own context, not an independently leaked victim context, so the missing modifier is a hardening issue rather than a proven exploit path.

Recommendations

Do not add `nonReentrant` directly to `settleLimitLendHook()`: the function self-calls into `TermLoanIntentFacet` through the diamond, and a second acquisition of the shared OZ lock would deadlock.

The safer remediation is to remove this hook's dependence on ambient flash-loan borrower authority. The nested `settleLimitLend()` path should only be reachable through an explicit executor-controlled context, such as a

short-lived hook-specific flag or nonce that is set immediately before the internal self-call and cleared immediately after. That makes the authorization model match the actual execution plan instead of relying on a global "active borrower" bit that remains visible to unrelated external callers.

As defence in depth, ensure no privileged context remains live across the post-settlement `transferFrom / safeTransferFrom` call. If the broader flash-loan context model is redesigned as recommended in issue [TRF-24](#), this hook should be migrated to that tighter execution-scoped authorization scheme rather than relying on `nonReentrant` alone.

Resolution

The development team has resolved this issue in [PR#1641](#).

The hook now sets `activeAtomicTxSettlementTaker` before each nested `settleLimitLend()` call and clears it immediately after, replacing the ambient flash-loan borrower authority with an explicit, short-lived context flag as recommended. The `TermMultiContextAuth` modifier has been tightened to only validate this scoped flag.

TRF-85	Unsafe Permit2 Cast In RepoToken Swap Silently Truncates Transfer Amount	
Assets	contracts/facets/TermRepoTokenIntentFacet.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

`TermRepoTokenIntentFacet._executeTokenTransfer()` uses raw `uint160()` casts when calling `Permit2's transferFrom()`, silently truncating transfer amounts that exceed `type(uint160).max` instead of reverting, while the protocol's swap accounting continues to use the full `uint256` value.

`_executeTokenTransfer()` is the settlement helper called by `_settleSwap()` for every `swapRepoToken()` execution. When `useTransfer2` is true, it pulls tokens via `Permit2's` allowance-based `transferFrom()`, which accepts a `uint160` amount. The function performs the narrowing conversion with a bare cast:

TermRepoTokenIntentFacet.sol

```
if (useTransfer2) {
    Permit2Lib.PERMIT2.transferFrom(from, to, uint160(amount - fee), token); // @audit raw cast, silently truncates
    if (fee > 0) {
        Permit2Lib.PERMIT2.transferFrom(from, feeRecipient, uint160(fee), token); // @audit raw cast, silently truncates
    }
}
```

The identical helper in `TermLoanIntentFacet` uses the checked extension from OpenZeppelin's `SafeCast` library, which is already imported via `using SafeCast for uint256` in both facets:

TermLoanIntentFacet.sol

```
if (useTransfer2) {
    Permit2Lib.PERMIT2.transferFrom(from, to, (amount - fee).toUint160(), token);
    if (fee > 0) {
        Permit2Lib.PERMIT2.transferFrom(from, feeRecipient, fee.toUint160(), token);
    }
}
```

If `amount` exceeds `type(uint160).max`, the raw cast truncates silently: `Permit2` pulls the lower 160 bits while swap accounting records the full `uint256` value. This finding is informational because exploitation requires token amounts exceeding `type(uint160).max` ($\sim 1.46 * 10^{48}$), which no deployed ERC-20 token approaches in total supply, and the inconsistency is a code-quality concern rather than a reachable loss path.

Recommendations

Replace `uint160(amount - fee)` and `uint160(fee)` with `(amount - fee).toUint160()` and `fee.toUint160()` respectively in `TermRepoTokenIntentFacet._executeTokenTransfer()`, consistent with the pattern already used in `TermLoanIntentFacet`.

Resolution

The recommendation has been implemented in [PR#1629](#).

TRF-86 Miscellaneous General Comments	
Assets	All contracts
Status	Resolved: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Hardcoded Value Should Be Replaced By A Named Variable

Related Asset(s): *TermRepoServicer.sol*

The code currently uses `10 ** 4` in multiple places, which reduces readability and maintainability.

TermRepoServicer.sol::redeemTermRepoTokens()

```
...
if (
    termRepoToken.totalRedemptionValue() <=
    totalRepurchaseCollected + 10 ** 4
)
...

```

TermRepoServicer.sol::_isTermRepoBalanced()

```
if (shortfallHaircutMantissa == 0) {
    uint256 totalLiquidity = totalOutstandingRepurchaseExposure + totalRepurchaseCollected;
    uint256 totalRedemptionValue = termRepoToken.totalRedemptionValue();
    if (totalLiquidity >= totalRedemptionValue) {
        return totalLiquidity - totalRedemptionValue <= 10 ** 4;
    } else {
        return totalRedemptionValue - totalLiquidity <= 10 ** 4;
    }
}
else {
    // ... formula
    if (totalRepurchaseCollected >= haircutRedemptionValue) {
        return totalRepurchaseCollected - haircutRedemptionValue <= 10 ** 4;
    } else {
        return haircutRedemptionValue - totalRepurchaseCollected <= 10 ** 4;
    }
}
}

```

The use of `10 ** 4` can lead to poor readability and lack of meaning around the code. It is considered best practice to declare constants when code will be using a magic number repeatedly.

This magic number should be replaced with a named constant, e.g., `PRECISION`, defined at the top of the file. This improves clarity and makes future changes easier.

2. Inconsistent `notPastRepaymentWindow` Modifier Usage In *TermRepoRolloverManager*

Related Asset(s): *TermRepoRolloverManager.sol*

The *TermRepoRolloverManager* contract inconsistently applies the `notPastRepaymentWindow` modifier, with `electRollover()` using the modifier, while `approveRolloverAuction()` duplicates the same logic inline instead of using the existing modifier.

The `notPastRepaymentWindow` modifier provides a standardised check for `block.timestamp >= termRepoServicer.endOfRepurchaseWindow()`, but `approveRolloverAuction()` re-implements this identical logic:

TermRepoRolloverManager.sol

```

modifier notPastRepaymentWindow() {
    // solhint-disable-next-line not-rely-on-time
    if (block.timestamp >= termRepoServicer.endOfRepurchaseWindow()) {
        revert EndOfRepurchaseWindowReached();
    }
    _;
}

function electRollover(
    TermRepoRolloverElectionSubmission calldata termRepoRolloverElectionSubmission
) external notPastRepaymentWindow {
    // ... function logic
}

function approveRolloverAuction(
    ITermAuctionBidLocker auctionBidLocker
) external onlyRole(ADMIN_ROLE) {
    // solhint-disable-next-line not-rely-on-time
    if (block.timestamp >= termRepoServicer.endOfRepurchaseWindow()) { // @audit - duplicate logic
        revert EndOfRepurchaseWindowReached();
    }
    // ... function logic
}

```

This unnecessary repetition creates potential maintenance issues where changes to the repayment window logic would need to be updated in multiple places instead of just the modifier. It also likely increases the contract size.

Apply the `notPastRepaymentWindow` modifier in `approveRolloverAuction()`.

3. Inefficient Double Looping In Batch Liquidation Functions

Related Asset(s): `TermRepoCollateralManager.sol`

Batch liquidation functions perform redundant double looping over collateral tokens when unlocking remaining borrower collateral after full debt repayment, causing unnecessary gas consumption.

Both `batchLiquidation()` and `batchLiquidationWithRepoToken()` in `TermRepoCollateralManager` execute the same pattern:

TermRepoCollateralManager.sol::batchLiquidation()

```

// @audit - first loop - decrements encumbered collateral balances
_unencumberRemainingBorrowerCollateralOnZeroObligation(borrower);

// @audit - second loop - unlocks collateral but skips the encumbered balance decrement
for (uint256 i = 0; i < collateralTokens.length; ++i) {
    if (lockedCollateralLedger[borrower][collateralTokens[i]] > 0) {
        _unlockCollateral(
            borrower,
            collateralTokens[i],
            lockedCollateralLedger[borrower][collateralTokens[i]],
            false // @audit - decrementEncumberedCollateral = false
        );
    }
}

```

TermRepoCollateralManager.sol::batchLiquidationWithRepoToken()

```
// @audit - first loop - decrements encumbered collateral balances
_unencumberRemainingBorrowerCollateralOnZeroObligation(borrower);

// @audit - second loop - unlocks collateral but skips the encumbered balance decrement
for (uint256 i = 0; i < collateralTokens.length; ++i) {
    if (lockedCollateralLedger[borrower][collateralTokens[i]] > 0) {
        _unlockCollateral(
            borrower,
            collateralTokens[i],
            lockedCollateralLedger[borrower][collateralTokens[i]],
            false // @audit - decrementEncumberedCollateral = false
        );
    }
}
```

The `_unencumberRemainingBorrowerCollateralOnZeroObligation()` function loops through all collateral tokens:

TermRepoCollateralManager.sol::_unencumberRemainingBorrowerCollateralOnZeroObligation()

```
for (uint256 i = 0; i < collateralTokens.length; ++i) {
    encumberedCollateralBalances[
        collateralTokens[i]
    ] -= lockedCollateralLedger[borrower][collateralTokens[i]];
}
```

Then the second loop calls `_unlockCollateral()` with `decrementEncumberedCollateral = false`:

TermRepoCollateralManager.sol::_unlockCollateral()

```
// ... requirement
lockedCollateralLedger[borrower][collateralToken] -= amount;
if (decrementEncumberedCollateral) {
    encumberedCollateralBalances[collateralToken] -= amount; // @audit - skipped when false
}
termRepoLocker.transferTokenToWallet(borrower, collateralToken, amount);
// ... event emitted
```

The first loop decrements `encumberedCollateralBalances` for all collateral tokens. This results in a double iteration over the collateral token array.

Remove the call to `_unencumberRemainingBorrowerCollateralOnZeroObligation()` and change the `decrementEncumberedCollateral` parameter to `true` in the `_unlockCollateral()` calls.

4. Misleading Comment

Related Asset(s): `contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol`

In `TermFlashLoanExecutorFacet`, the comment in line [499] states that the code validates the actual output token against the user-defined output token, while the implementation instead in line [500] compares `action.inputToken` with `previewAction.expectedInputToken`:

flashloan/TermFlashLoanExecutorFacet.sol

```
// Validate actual output token matches user defined output token
if (action.inputToken != previewAction.expectedInputToken) {
    revert InputTokenMismatch(i);
}
```

Consider fixing the comment.

5. Unused `user_` Parameter in Internal Functions

Related Asset(s): `contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol`

The internal functions `_depositToVault()` and `_redeemFromVaultInternal()` in both accept a `user_` parameter, but the value is never used in their implementations.

Consider removing the input argument `user_` from `_depositToVault()` and `_redeemFromVaultInternal()`.

6. Non-Standard Interface Naming

Related Asset(s): `contracts/facets/flashloan/TermFlashLoanExecutorFacet.sol`

The interface `FlashLoanAggregator` does not adhere to the conventional Solidity naming standard, which prefixes interfaces with `I`.

Consider renaming the interface to `IFlashLoanAggregator` to follow the naming standard.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team has addressed the above findings through the following pull requests:

- [PR#1446](#) - Replaced hardcoded `10 ** 4` values with named constants to improve code readability and maintainability
- [PR#1455](#) - Applied the `notPastRepaymentWindow` modifier consistently across `TermRepoRolloverManager` functions to eliminate duplicate validation logic
- [PR#1465](#) - Optimized batch liquidation functions to eliminate redundant double looping over collateral tokens, reducing gas consumption
- [PR#1630](#) - Addressed misleading comment, unused parameter, and non-standard interface naming in flash loan executor

These changes improve code quality, consistency, and efficiency without affecting the security properties of the protocol.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'